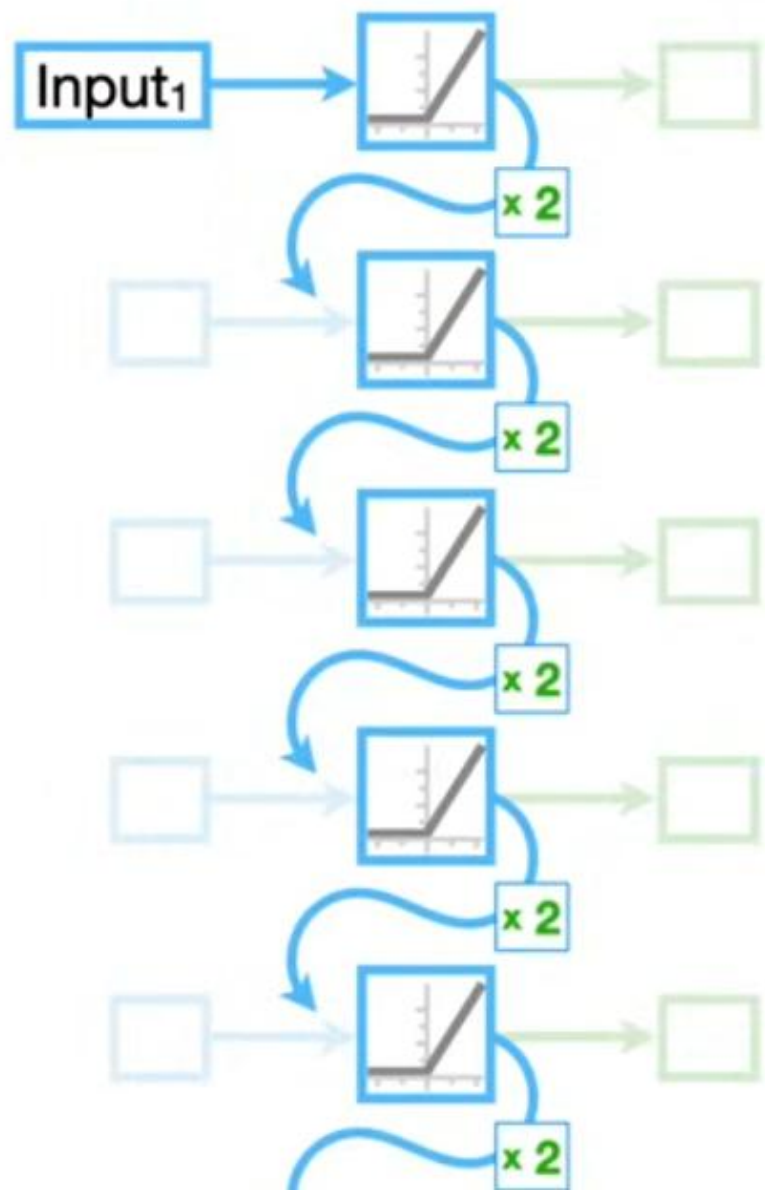




...then, when we do the math,
we end up multiplying the
Input by the **Weight**, which in
this case is **2**, raised to the
number of times we unrolled
the network.

$$= \text{Input}_1 \times 2^{\text{Num. Unroll}}$$





And thus, if we had **50**
sequential data points, like **50**
days of stock market data,
which isn't really that much...

...then we would raise
2 by **50**...

= $\text{Input}_1 \times 2^{50}$





...and **2** to the **50th** power is a
HUGE NUMBER...

$$= \text{Input}_1 \times 2^{50}$$

$$= \text{Input}_1 \times \text{A HUGE NUMBER}$$





...and this **HUGE NUMBER**
would cause the gradient,
which we need for **Gradient
Descent**...

$$= \text{Input}_1 \times 2^{50}$$

$$= \text{Input}_1 \times \text{A HUGE NUMBER}$$





Alternatively, we saw that if the **Weight** on the feedback loop was less than **1**, and now we have it set to **0.5**...

$$= \text{Input}_1 \times 0.5^{50}$$





...then we'll end up
multiplying the **Input** value
by **0.5** raised to the **50th**
power...

$= \text{Input}_1 \times 0.5^{50}$





...and **0.5** raised to the **50th** power is a number super close to **0**...

$$= \text{Input}_1 \times 0.5^{50}$$

$$= \text{Input}_1 \times \text{a number super close to } 0$$





...and this number super close to **0** would cause the gradient, which we need for **Gradient Descent**...

$$= \text{Input}_1 \times 0.5^{50}$$

$$= \text{Input}_1 \times \text{a number super close to } 0$$





...to vanish.

$$= \text{Input}_1 \times 0.5^{50}$$

$$= \text{Input}_1 \times \text{a number super close to } 0$$





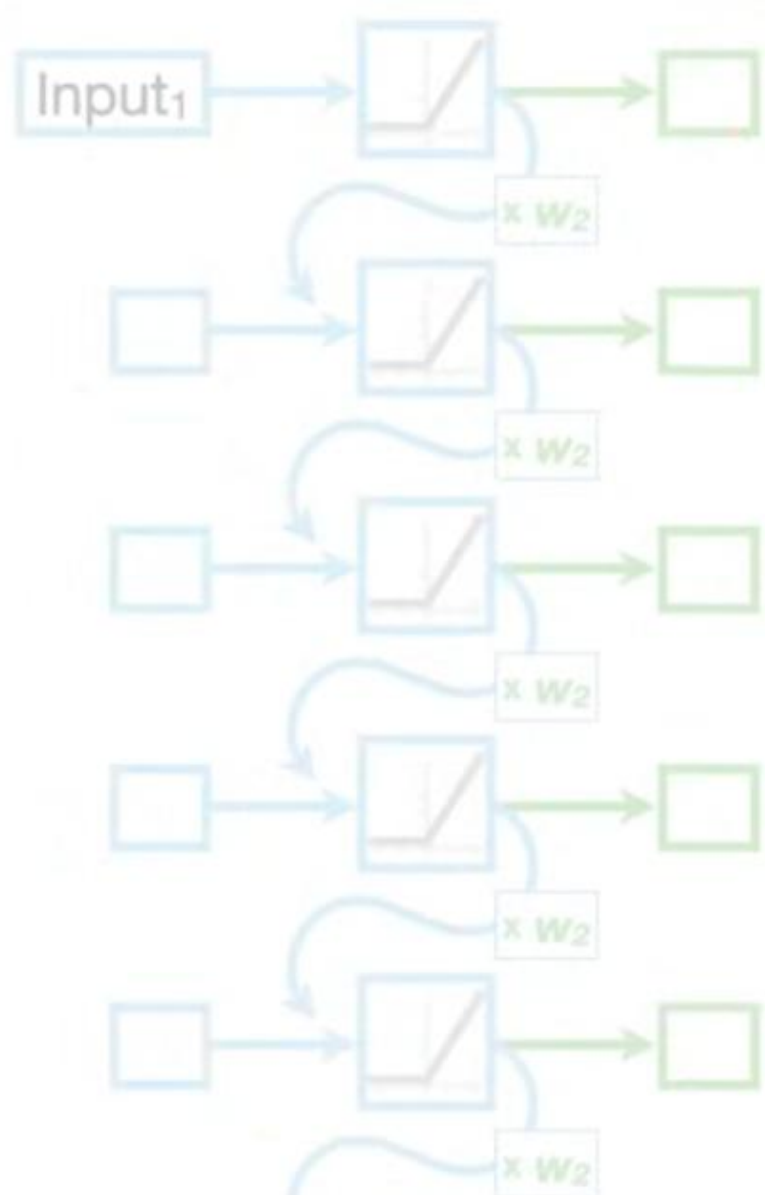
In summary, basic, vanilla **Recurrent Neural Networks** are hard to train because the gradients can **explode**, or **vanish**.

$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$





The good news is that it doesn't take much to extend the basic, vanilla **Recurrent Neural Network** so that we can avoid this problem.





So, today, we're going to talk about **Long Short-Term Memory (LSTM)**, which is a type of **Recurrent Neural Network** that is designed to avoid the **exploding/vanishing** gradient problem.

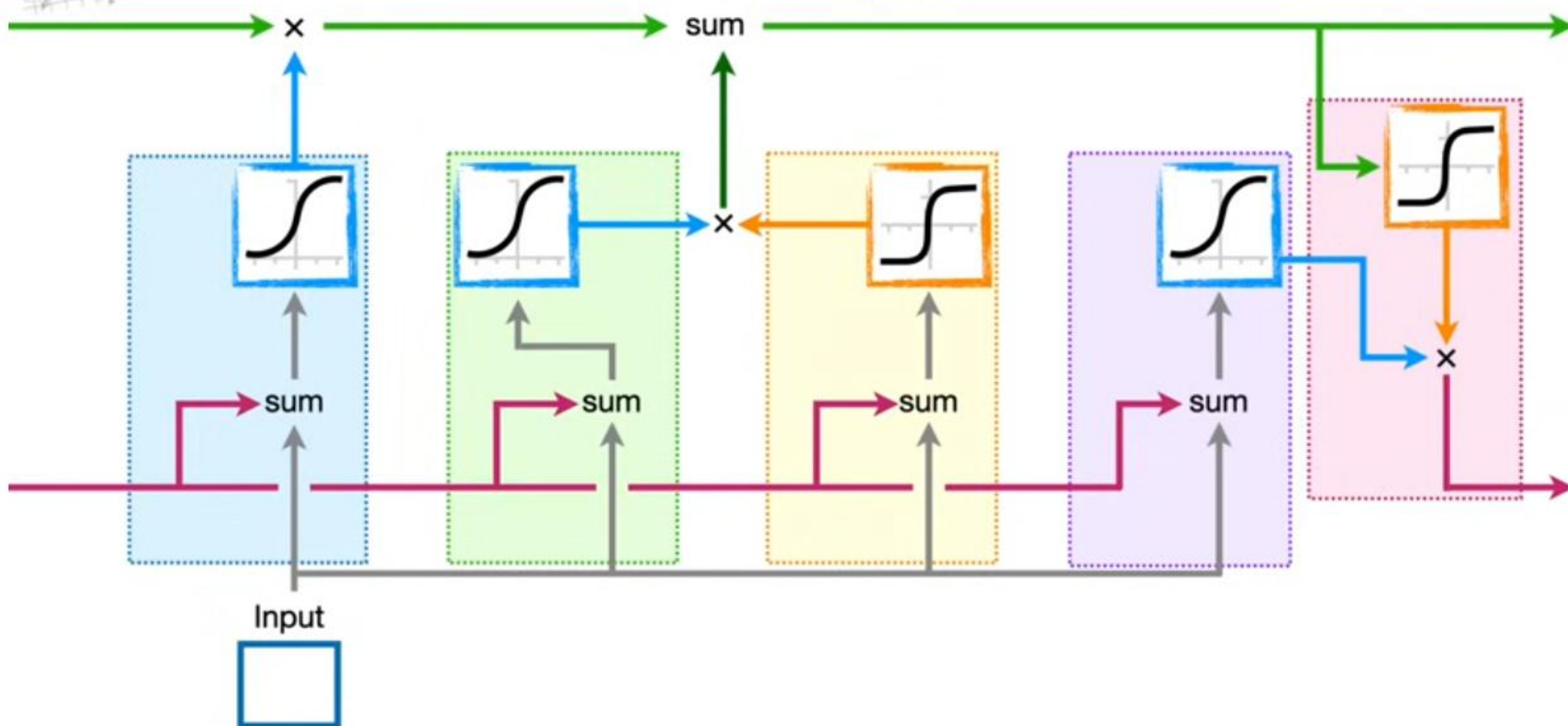




...is that instead of using the
same feedback loop
connection...

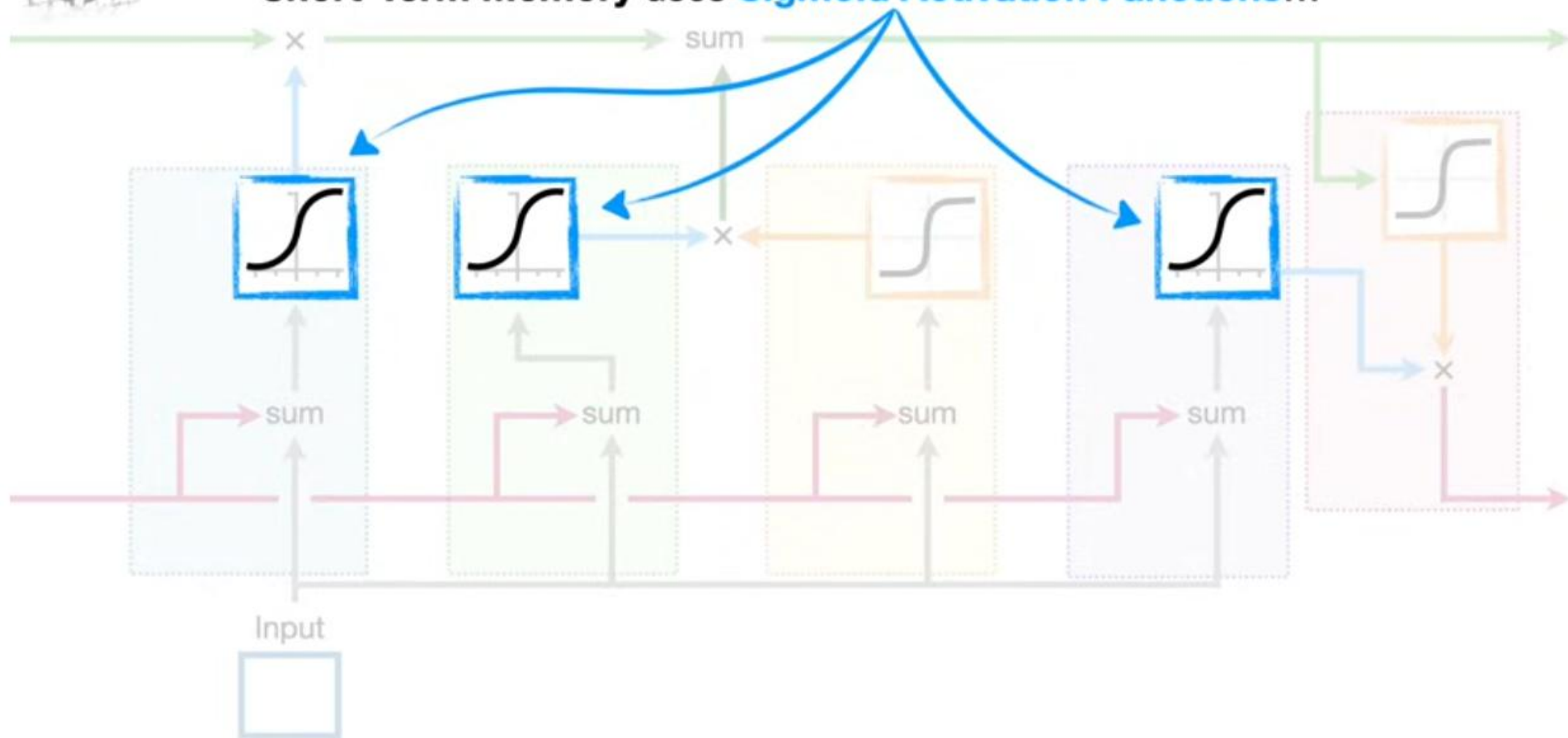


...**Long Short-Term Memory** is based
on a much more complicated unit.



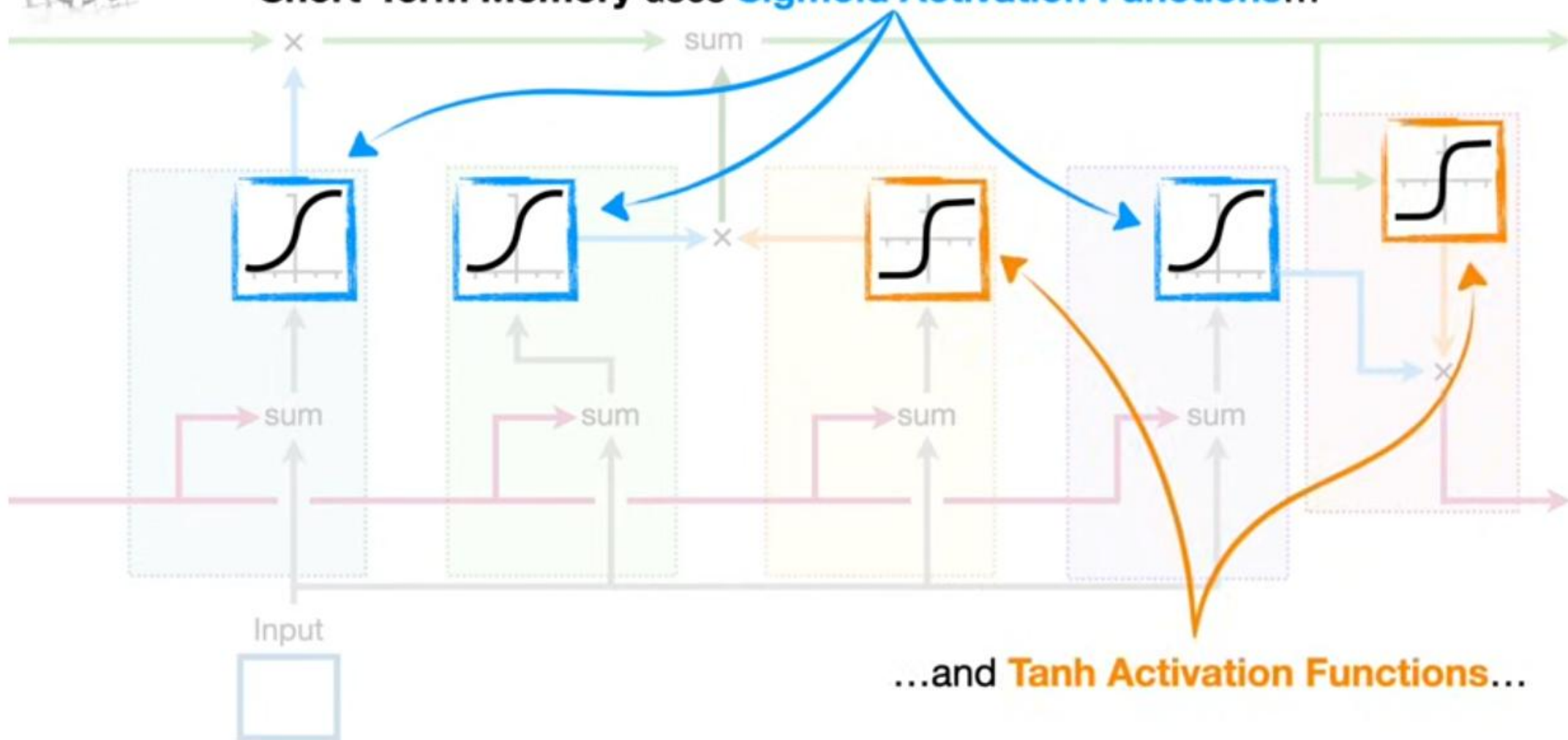


NOTE: Unlike the networks we've used before in this series, **Long Short-Term Memory** uses **Sigmoid Activation Functions...**





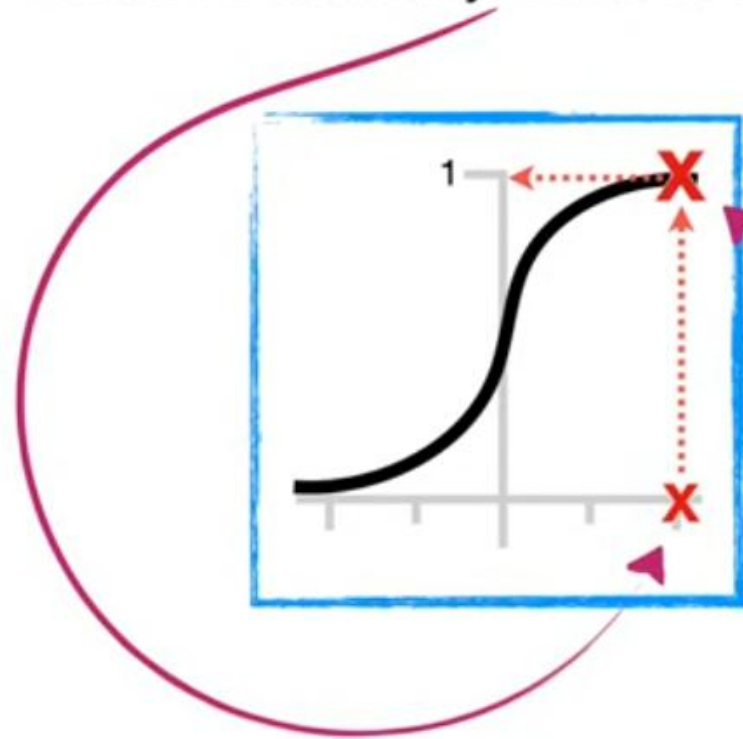
NOTE: Unlike the networks we've used before in this series, **Long Short-Term Memory** uses **Sigmoid Activation Functions...**





In a nutshell, the **Sigmoid Activation Function** takes any x-axis coordinate...

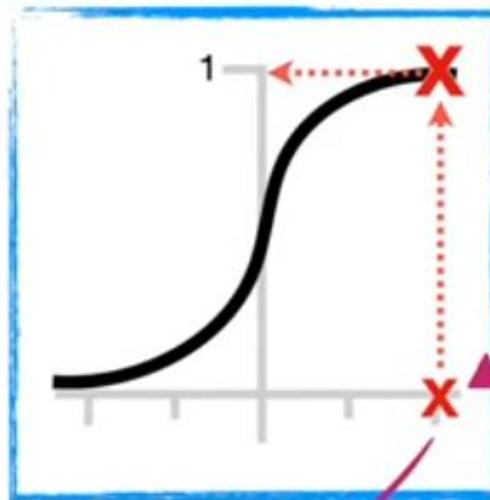
...and turns it into a y-axis coordinate between **0** and **1**.





For example, when we plug in this x-axis coordinate, **10**, into the equation for the **Sigmoid Activation Function...**

$$f(x) = \frac{e^x}{e^x + 1}$$

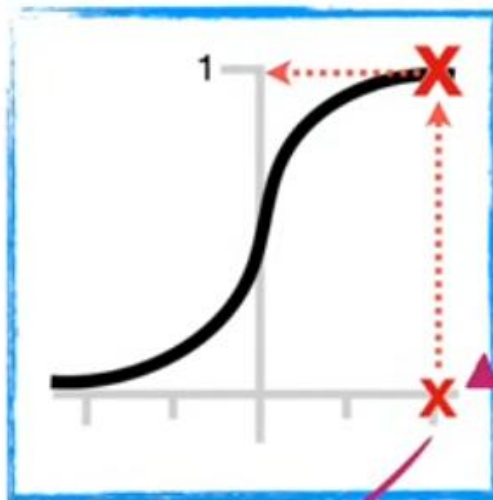




For example, when we plug in this x-axis coordinate, **10**, into the equation for the **Sigmoid Activation Function...**

$$f(x) = \frac{e^x}{e^x + 1}$$

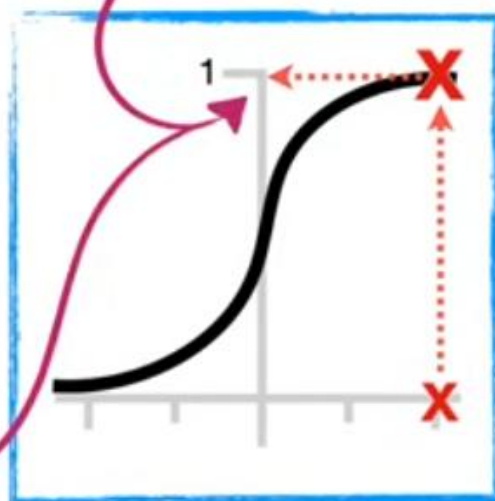
$$f(10) = \frac{e^{10}}{e^{10} + 1}$$





...we get **0.99995** as
the y-axis coordinate.

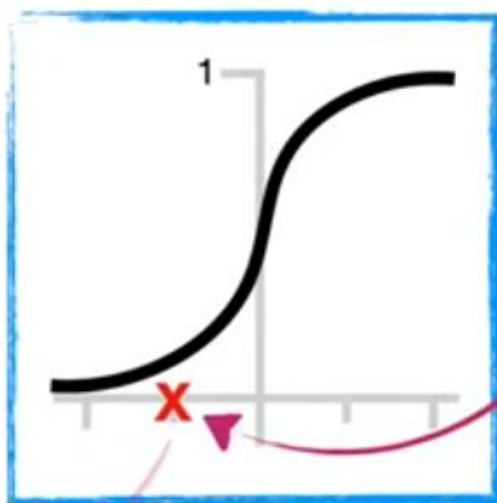
$$f(x) = \frac{e^x}{e^x + 1}$$
$$f(10) = \frac{e^{10}}{e^{10} + 1}$$
$$= 0.99995$$





And if we plug in this
x-axis coordinate, **-5**...

$$f(x) = \frac{e^x}{e^x + 1}$$

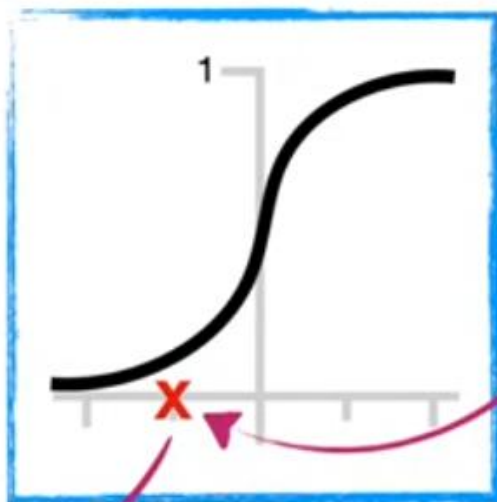




And if we plug in this
x-axis coordinate, **-5**...

$$f(x) = \frac{e^x}{e^x + 1}$$

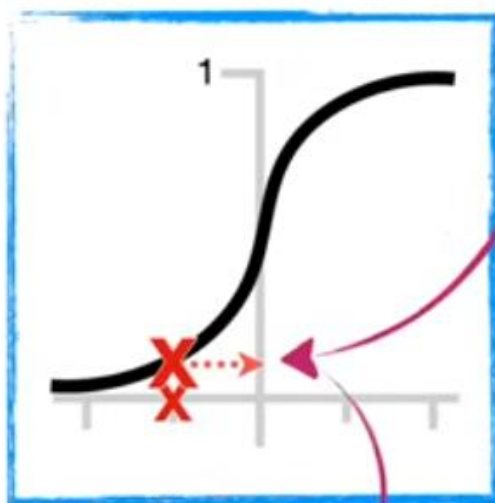
$$f(-5) = \frac{e^{-5}}{e^{-5} + 1}$$





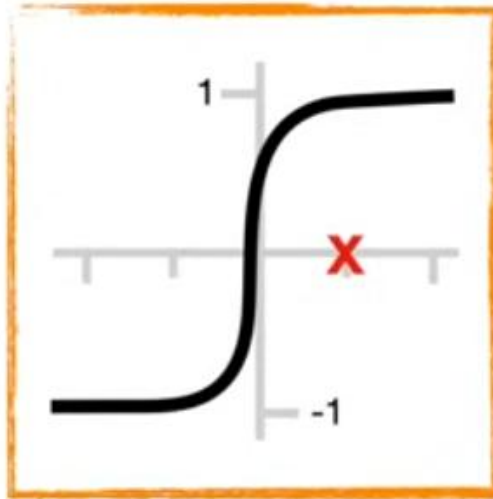
...then we get **0.01** as the y-axis coordinate.

$$f(x) = \frac{e^x}{e^x + 1}$$
$$f(-5) = \frac{e^{-5}}{e^{-5} + 1}$$
$$= 0.01$$



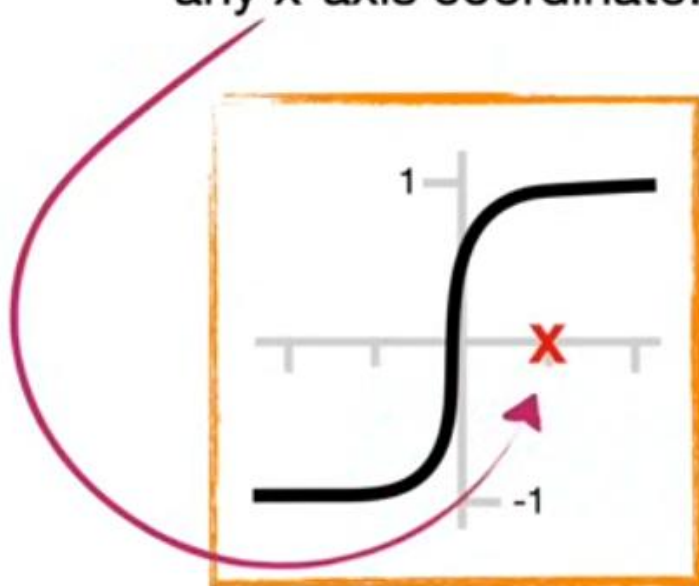


In contrast, the **tanh**, or **Hyperbolic Tangent, Activation Function** takes any x-axis coordinate...





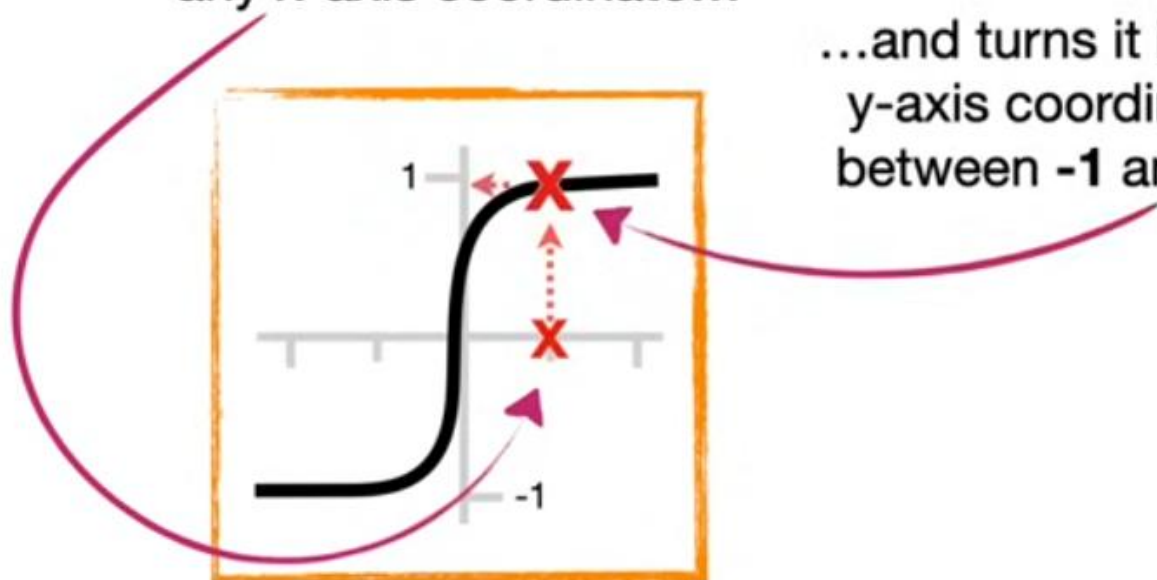
In contrast, the **tanh**, or **Hyperbolic Tangent, Activation Function** takes any x-axis coordinate...





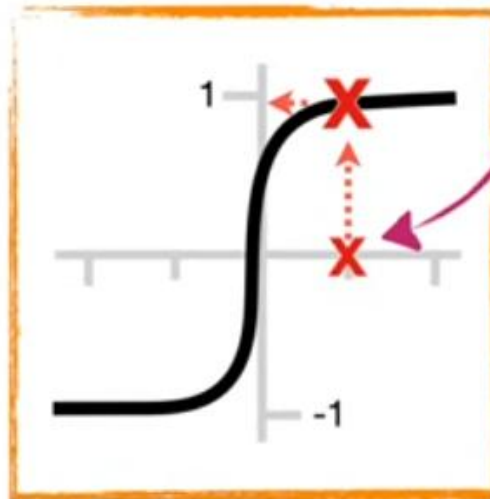
In contrast, the **tanh**, or **Hyperbolic Tangent, Activation Function** takes any x-axis coordinate...

...and turns it into a y-axis coordinate between **-1** and **1**.





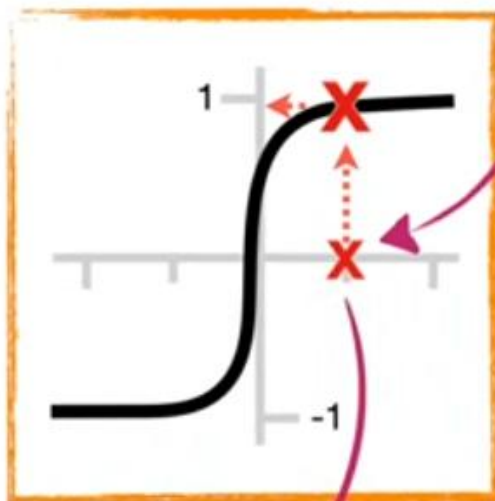
For example, if we plug this x-axis coordinate, **2**, into the equation for the **Tanh Activation Function...**





For example, if we plug this x-axis coordinate, **2**, into the equation for the **Tanh Activation Function...**

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

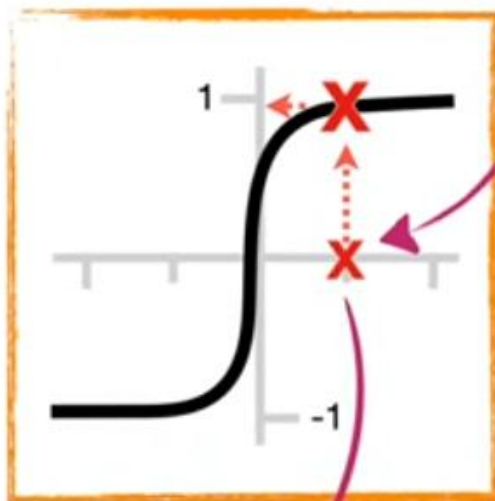




For example, if we plug this x-axis coordinate, **2**, into the equation for the **Tanh Activation Function...**

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(2) = \frac{e^2 - e^{-2}}{e^2 + e^{-2}}$$



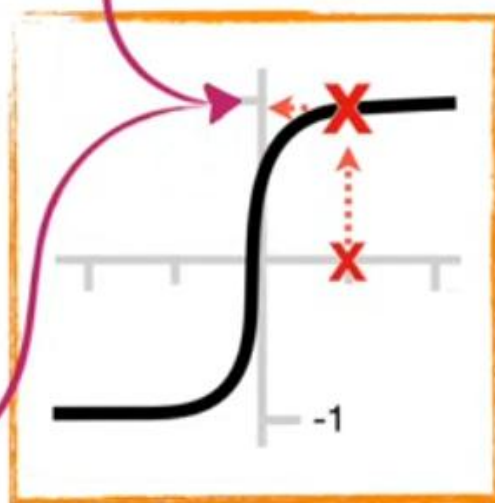


...we get **0.96** as the y-axis coordinate.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(2) = \frac{e^2 - e^{-2}}{e^2 + e^{-2}}$$

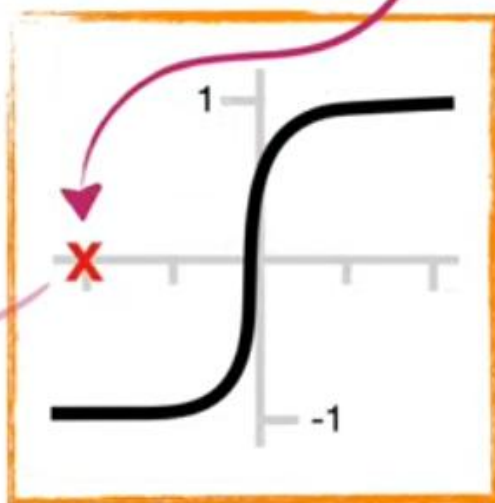
$$= 0.96$$





And if we plug in this
x-axis coordinate, **-5**...

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

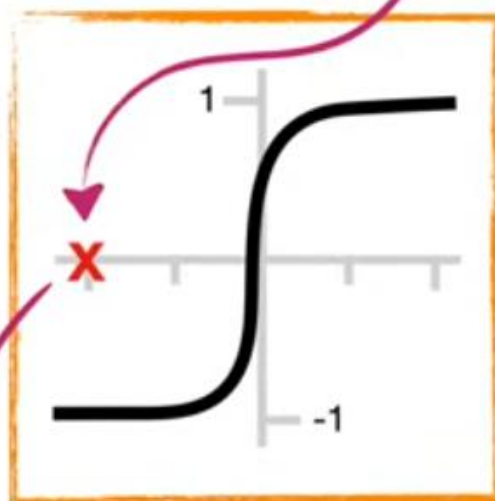




And if we plug in this
x-axis coordinate, **-5**...

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(-5) = \frac{e^{-5} - e^5}{e^{-5} + e^5}$$



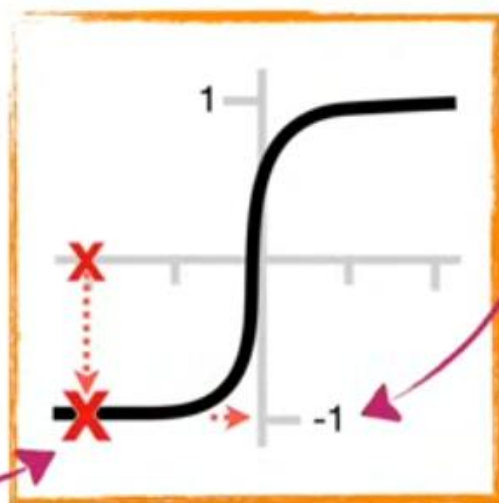


...we get **-1** as the
y-axis coordinate.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

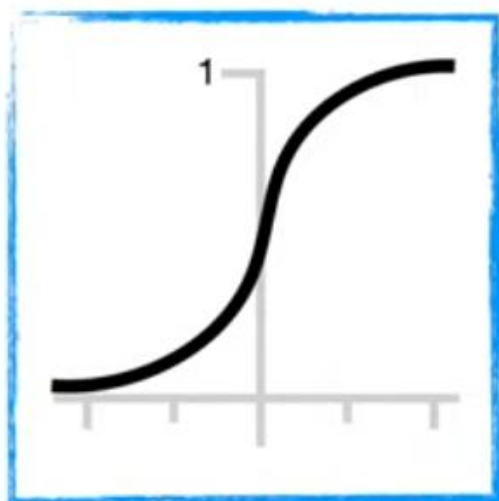
$$f(-5) = \frac{e^{-5} - e^5}{e^{-5} + e^5}$$

$$= -1$$



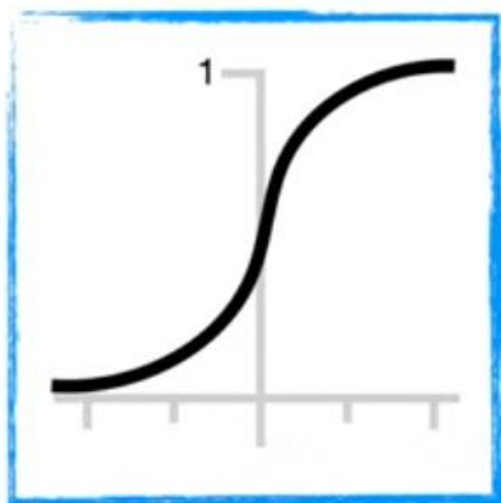


So, now that we know that the **Sigmoid Activation Function** turns any input into a number between **0** and **1**...

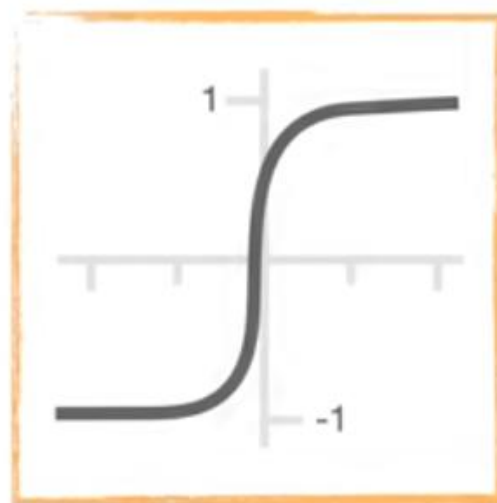




So, now that we know that the **Sigmoid Activation Function** turns any input into a number between **0** and **1**...

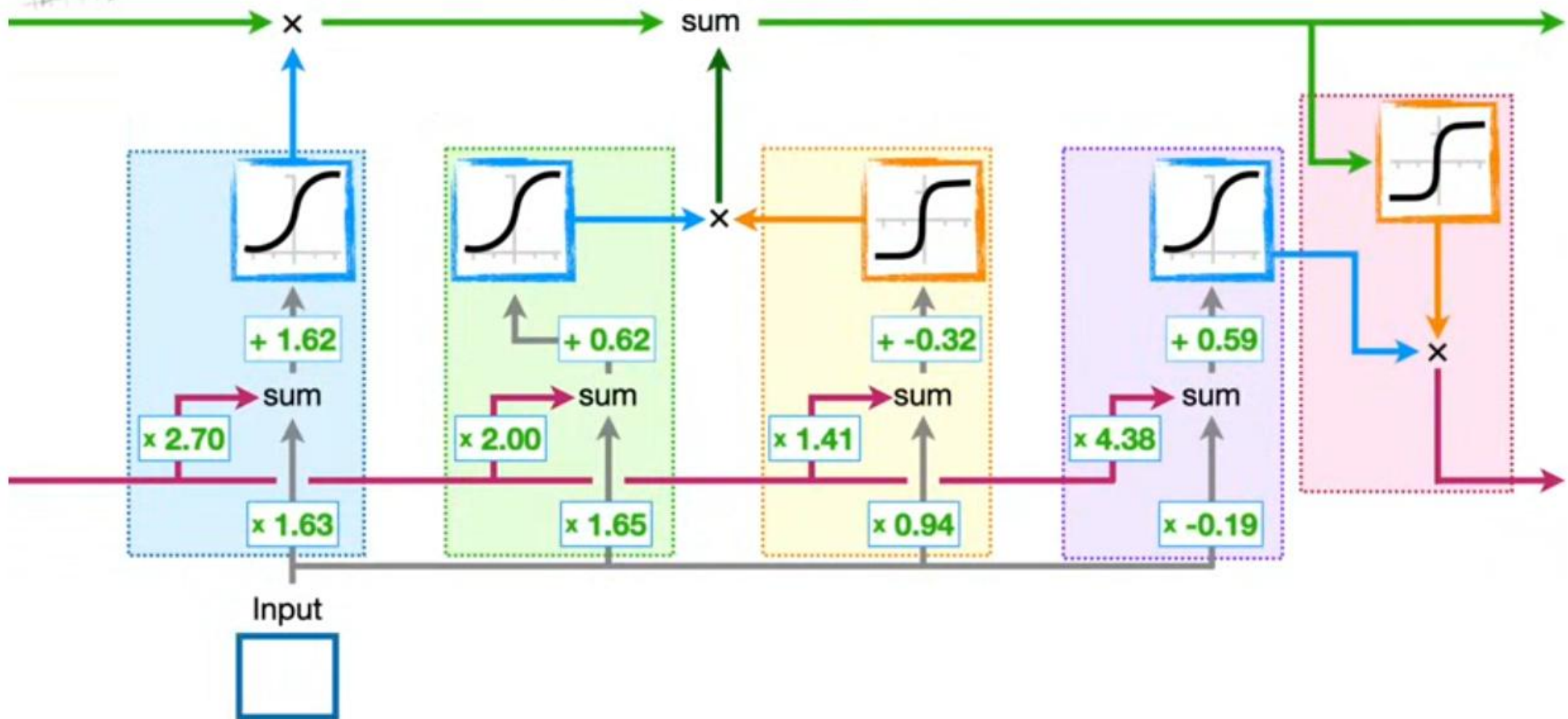


...and the **Tanh Activation Function** turns any input into a number between **-1** and **1**...





...let's talk about how the **Long Short-Term Memory** unit works.





First, this **green line** that runs all the way across the top of the unit is called the **Cell State** and represents the **Long-Term Memory**.

Input



x

sum

Although the **Long-Term Memory** can be modified by this multiplication...





x

sum

...you'll notice that there are
no **Weights** and **Biases** that
can modify it directly.

Input





x

sum

The lack of **Weights** allows the **Long-Term Memories** to flow through a series of unrolled units without causing the gradient to **explode** or **vanish**.

Input





x

sum

Now, this **pink line**, called the **Hidden State**,
represents the **Short-Term Memories**.

x 2.70

x 2.00

x 1.41

x 4.38



And, as we can see, the **Short-Term Memories** are directly connected to **Weights** that can modify them.





x

sum

To understand how the **Long** and **Short-Term Memories** interact and result in predictions, let's run some numbers through this unit.

x 2.70

x 2.00

x 1.41

x 4.38

Input



Long-Term
Memory

2

x

sum

First, for the sake of making the math interesting, let's just assume that the previous **Long-Term Memory** is 2...

x 2.70

sum

x 2.00

sum

x 1.41

sum

x 4.38

sum

Input





...and the previous
Short-Term Memory is 1.





Long-Term
Memory

2

x

sum

And let's also set the
Input value to 1.

Short-Term
Memory

1

x 2.70

sum

x 1.63

Input

1

x 2.00

sum

x 1.65

x 1.41

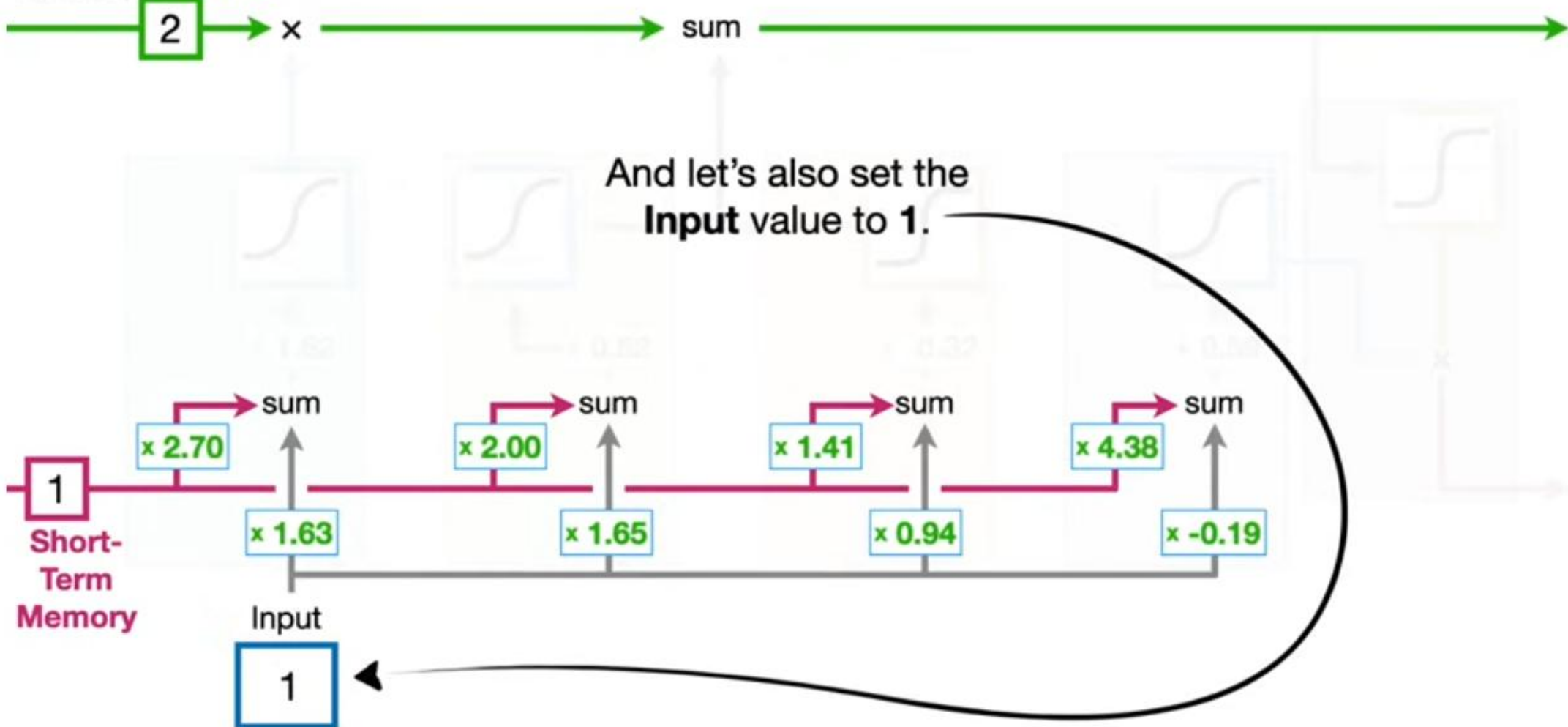
sum

x 0.94

x 4.38

sum

x -0.19





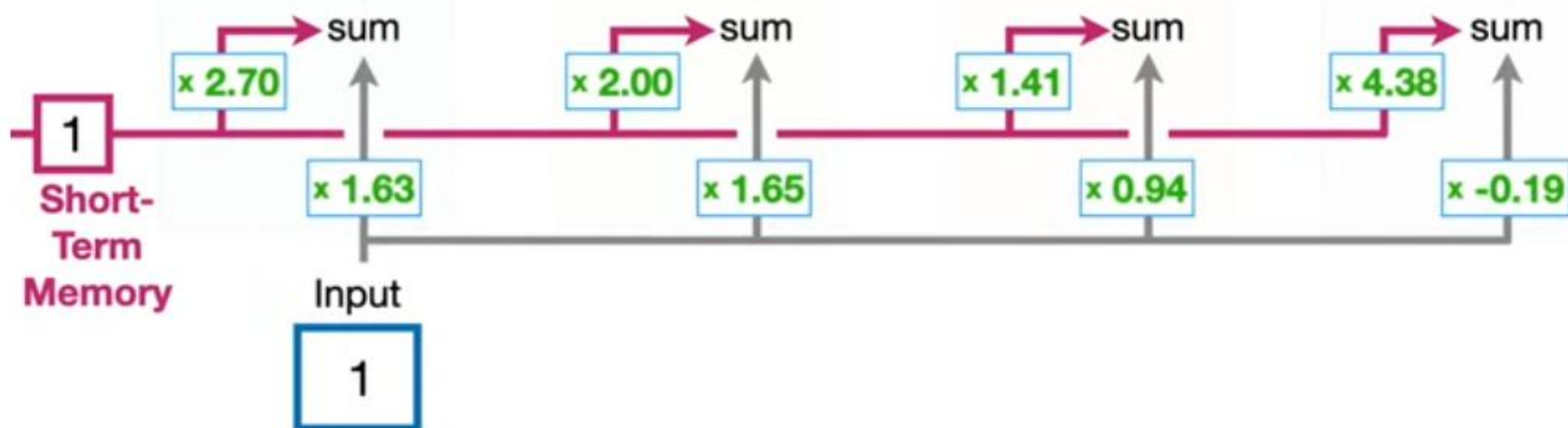
Long-Term
Memory

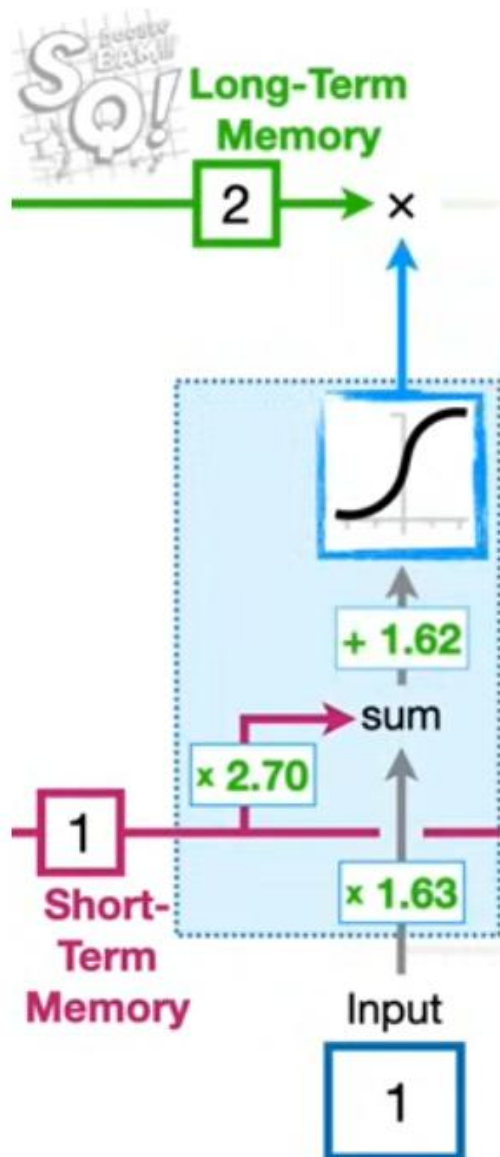
2

x

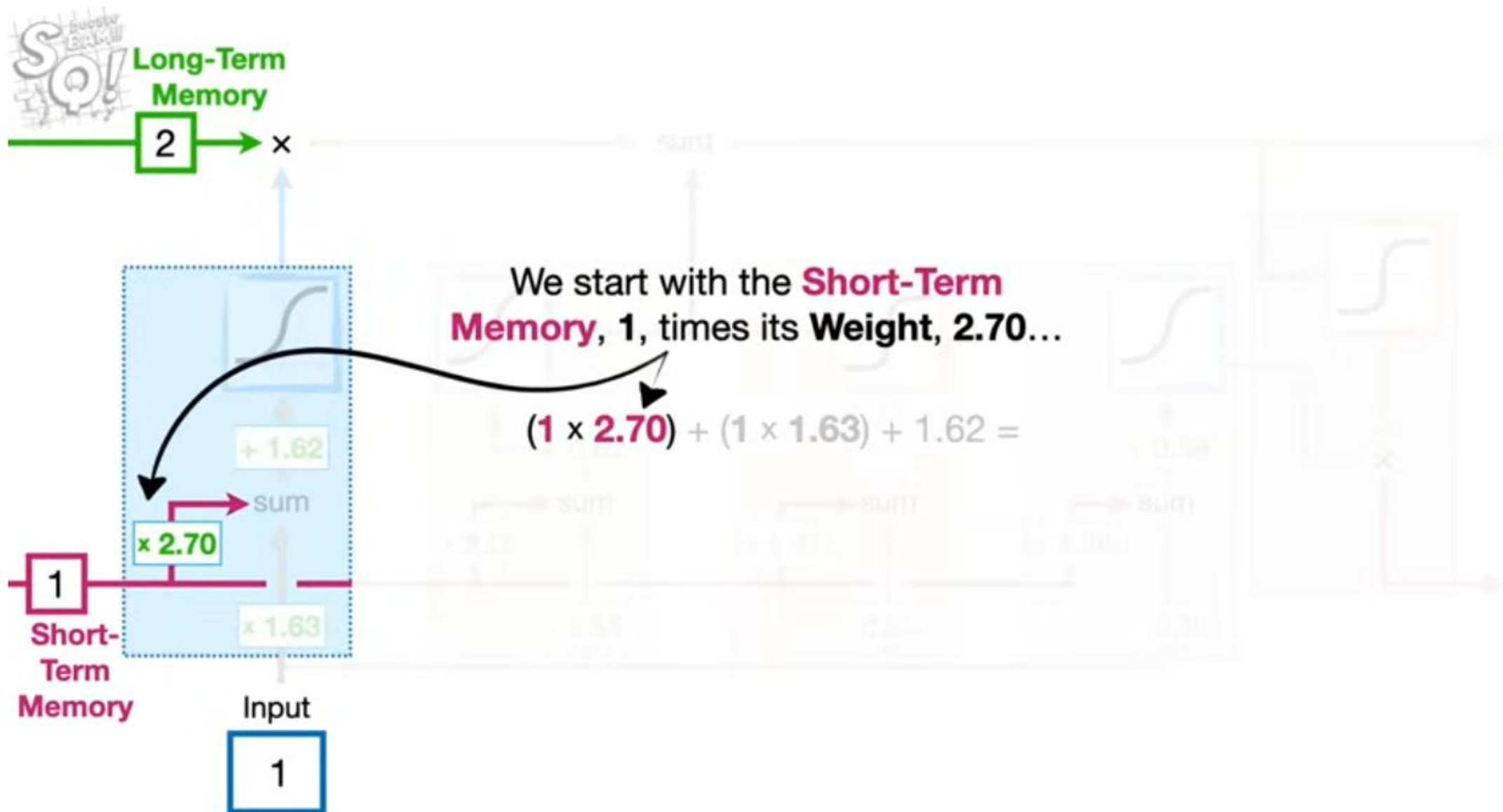
sum

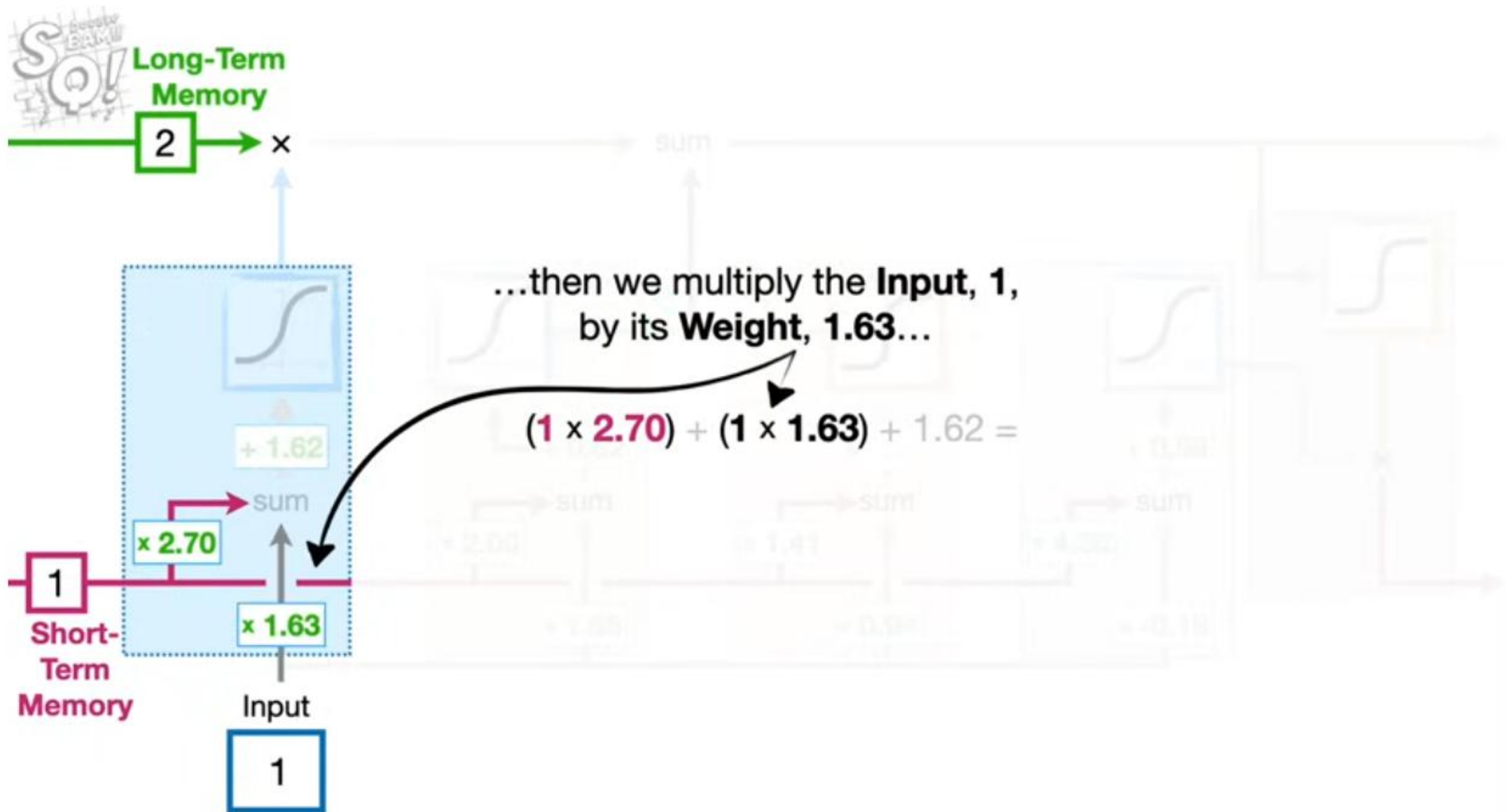
Now that we have plugged in some numbers, let's do the math to see what happens in the first stage of the **Long Short-Term Memory** unit.

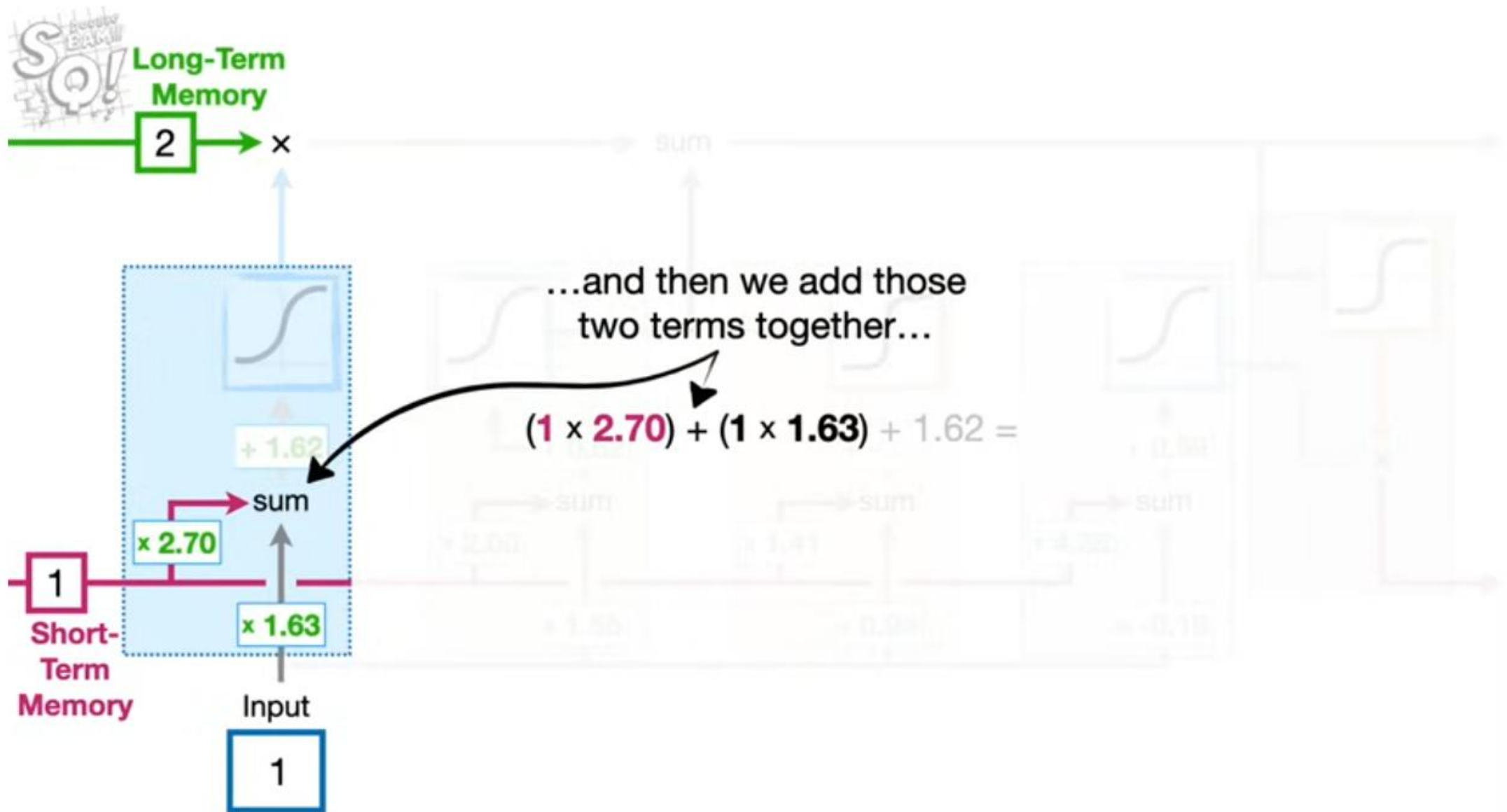


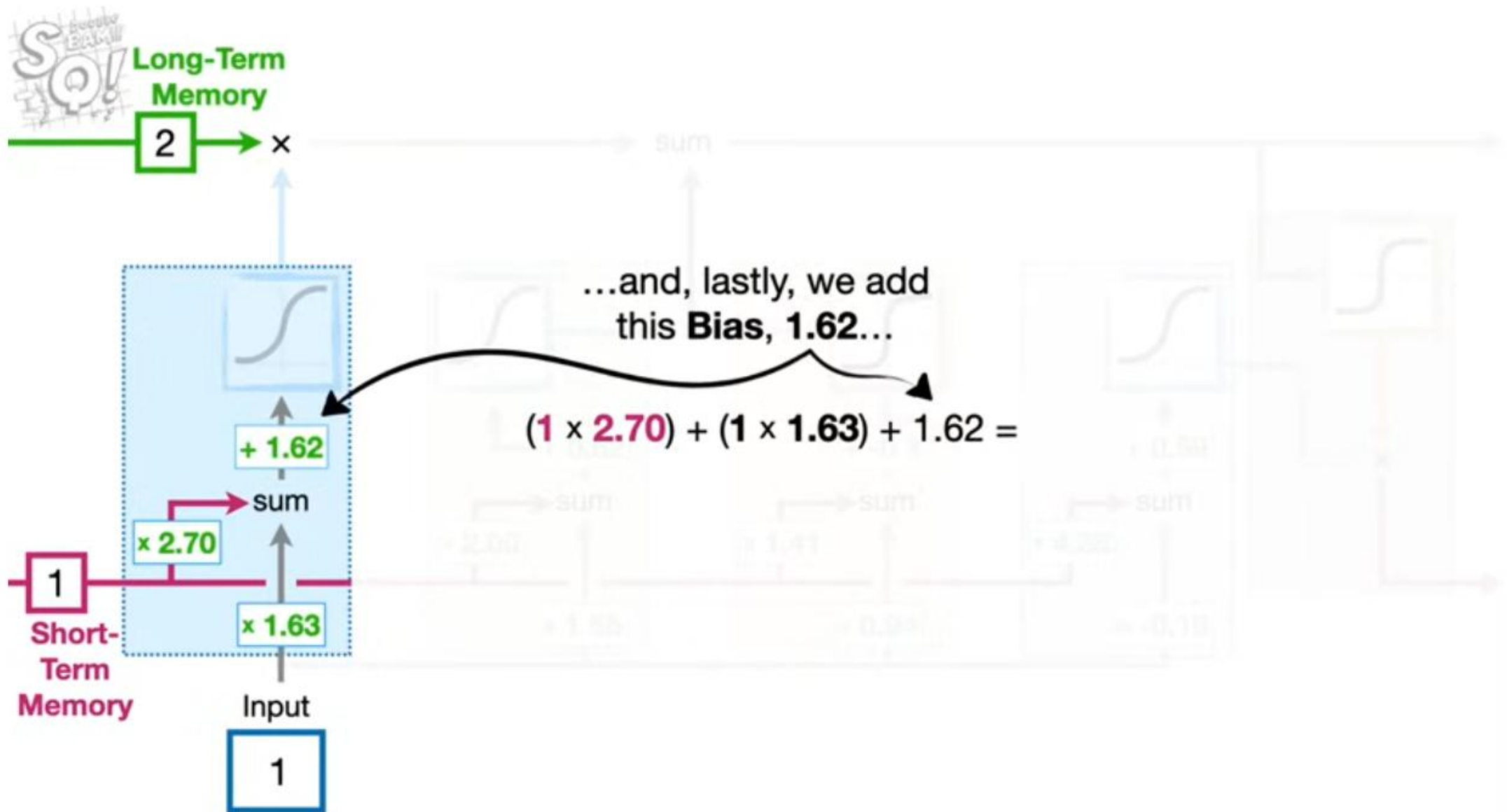


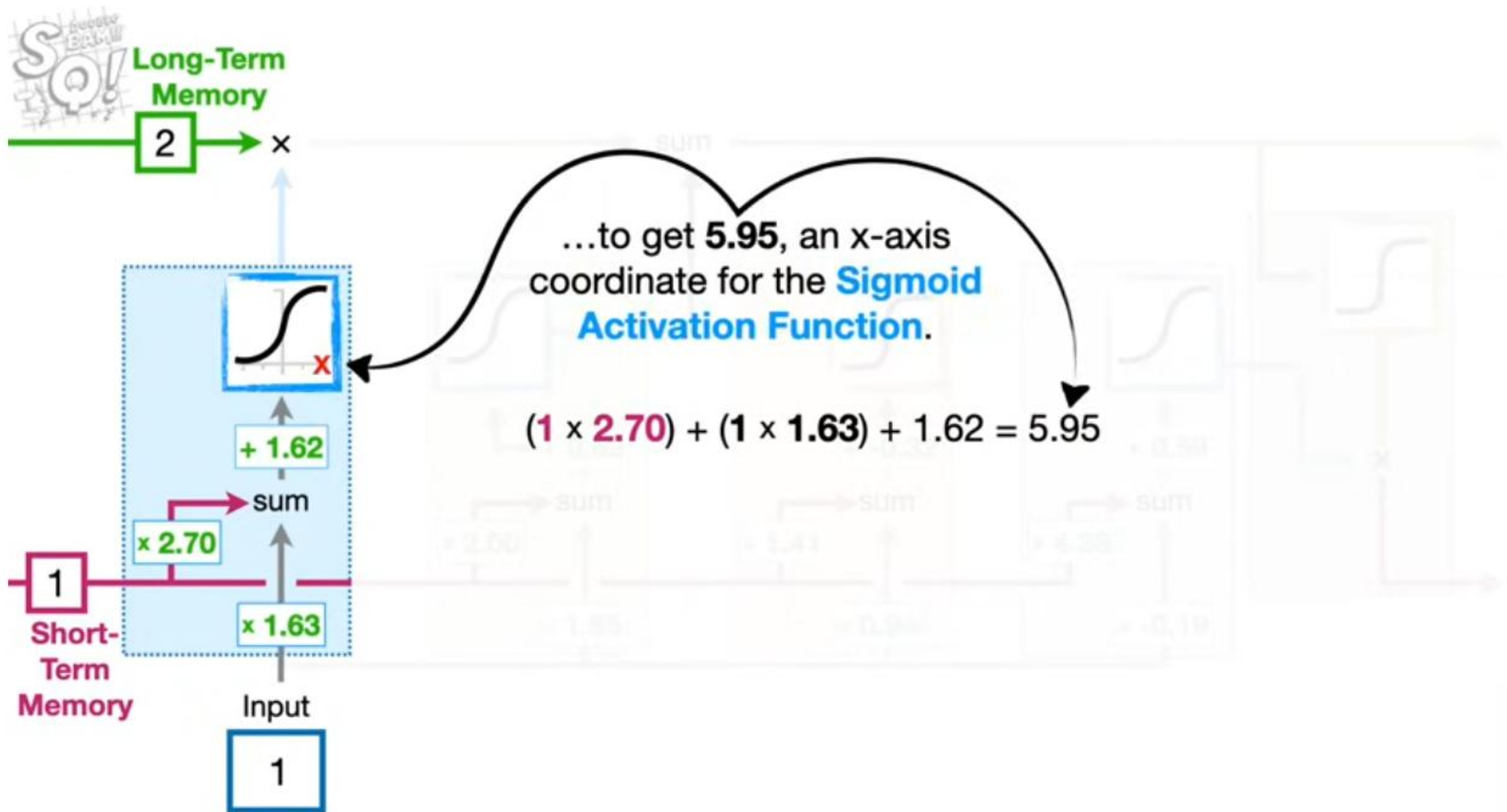
Now that we have plugged in some numbers, let's do the math to see what happens in the first stage of the **Long Short-Term Memory** unit.

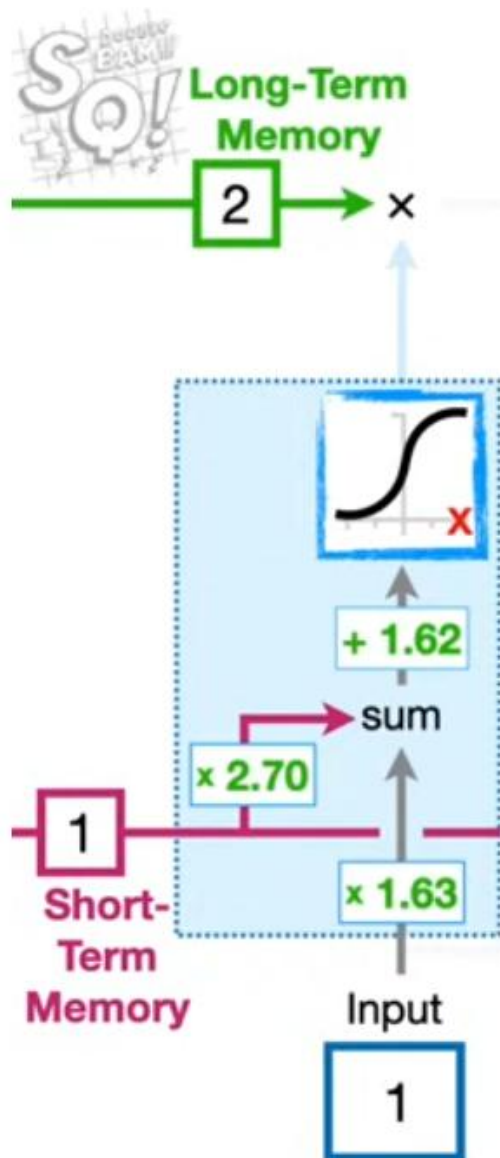








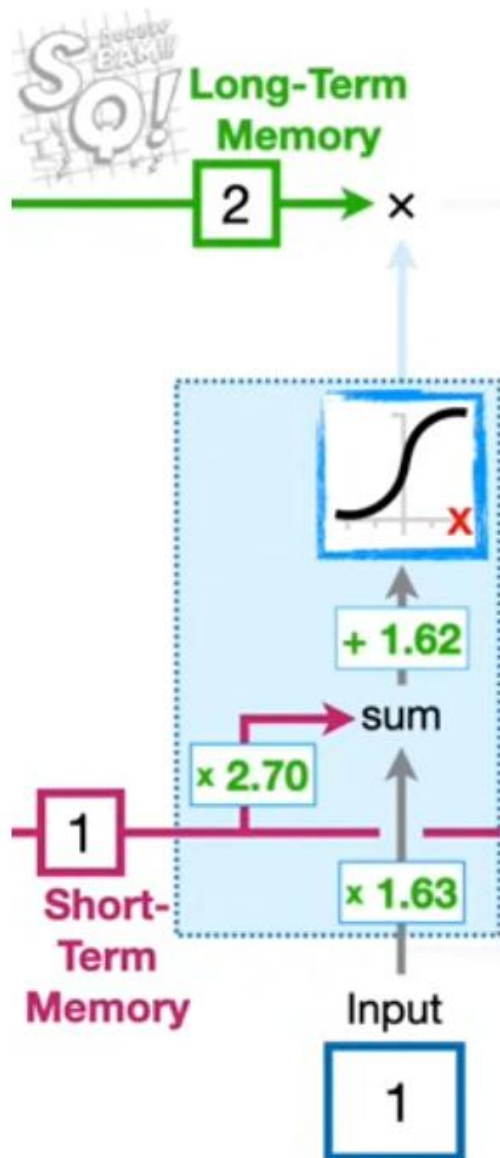




Now we plug the x-axis coordinate into the equation for the **Sigmoid Activation Function**...

$$(1 \times 2.70) + (1 \times 1.63) + 1.62 = 5.95$$

$$f(x) = \frac{e^x}{e^x + 1}$$

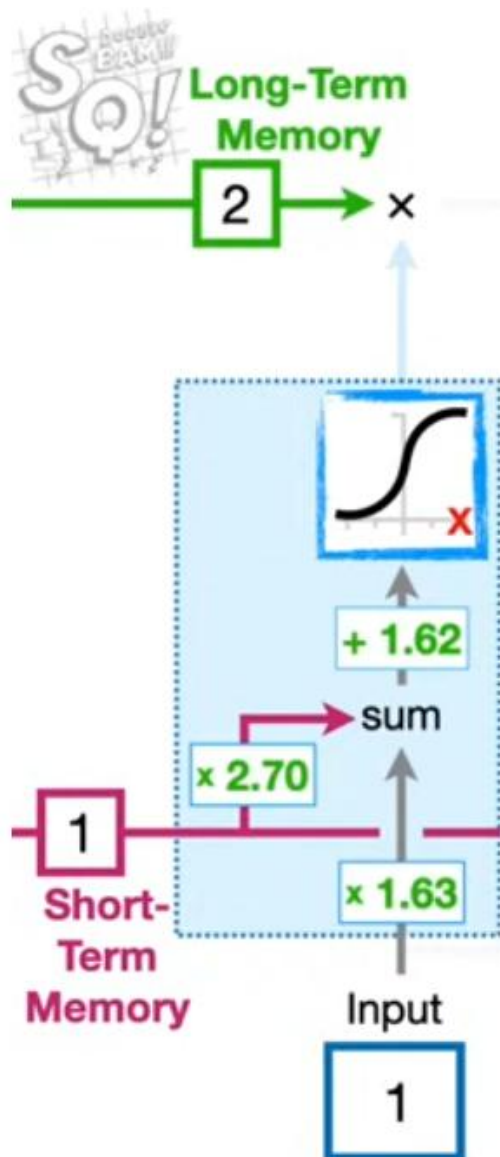


Now we plug the x-axis coordinate into the equation for the **Sigmoid Activation Function...**

$$(1 \times 2.70) + (1 \times 1.63) + 1.62 = 5.95$$

$$f(x) = \frac{e^x}{e^x + 1}$$

$$f(5.95) = \frac{e^{5.95}}{e^{5.95} + 1}$$



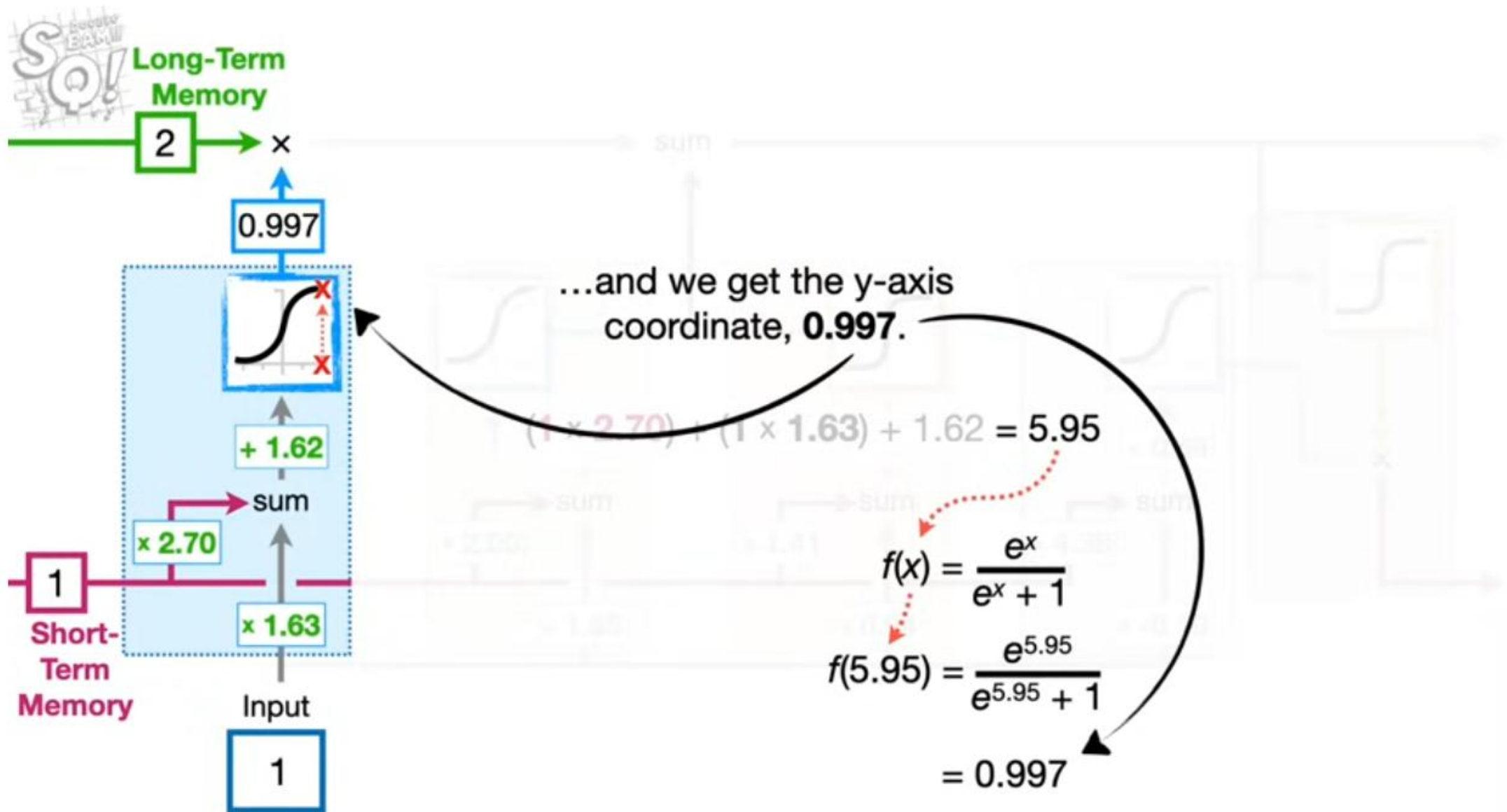
Now we plug the x-axis coordinate into the equation for the **Sigmoid Activation Function**...

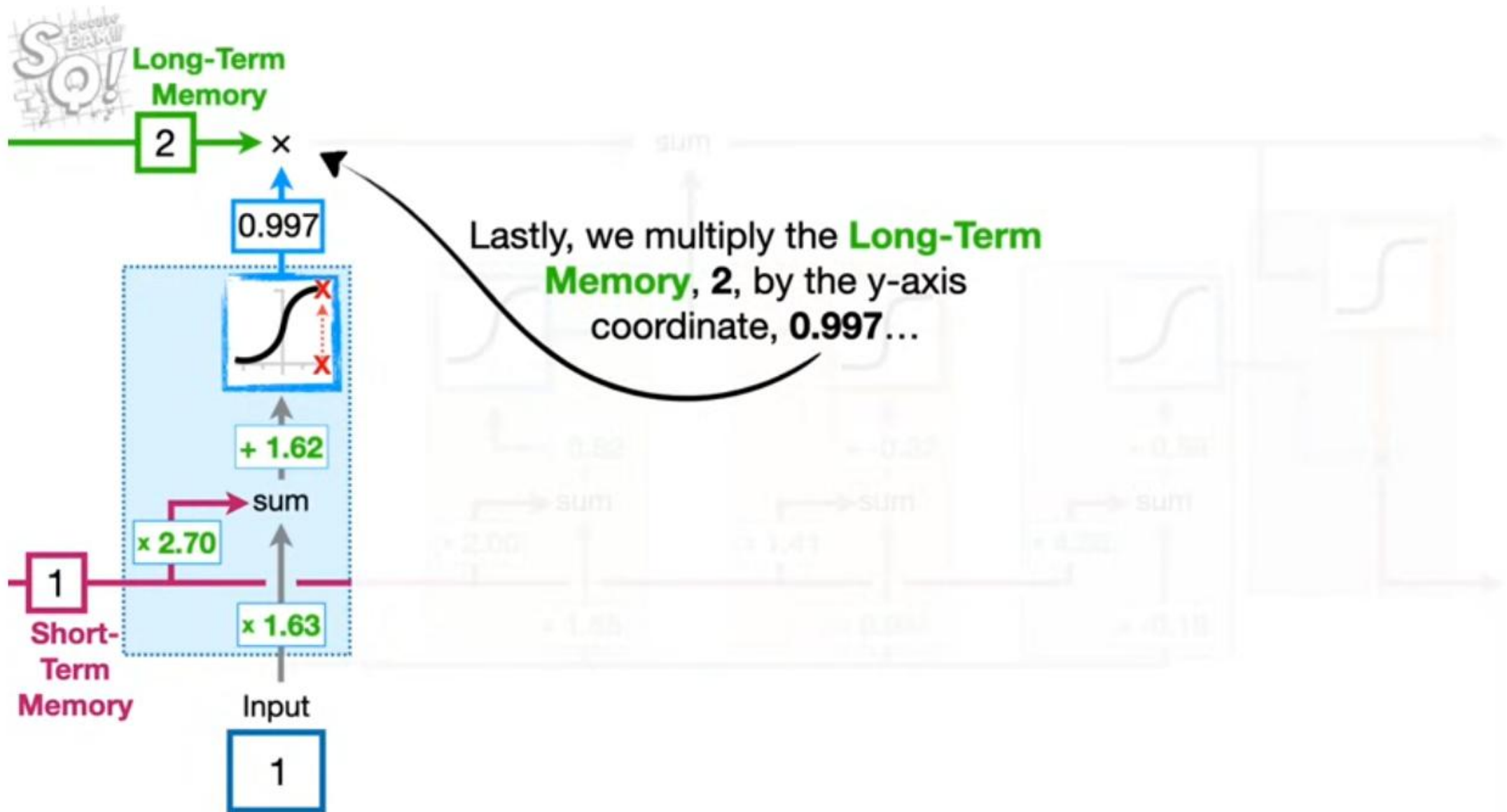
$$(1 \times 2.70) + (1 \times 1.63) + 1.62 = 5.95$$

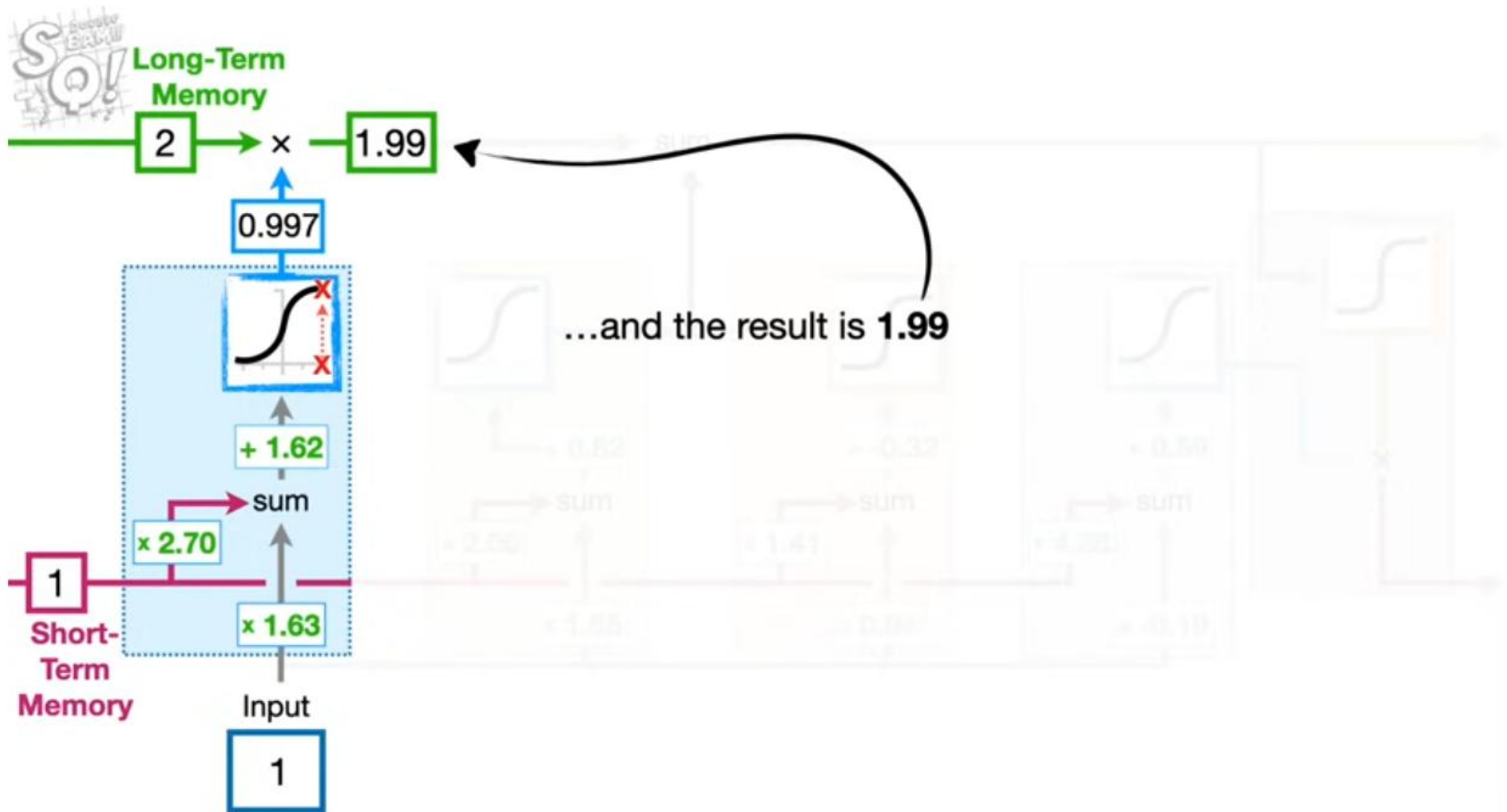
$$f(x) = \frac{e^x}{e^x + 1}$$

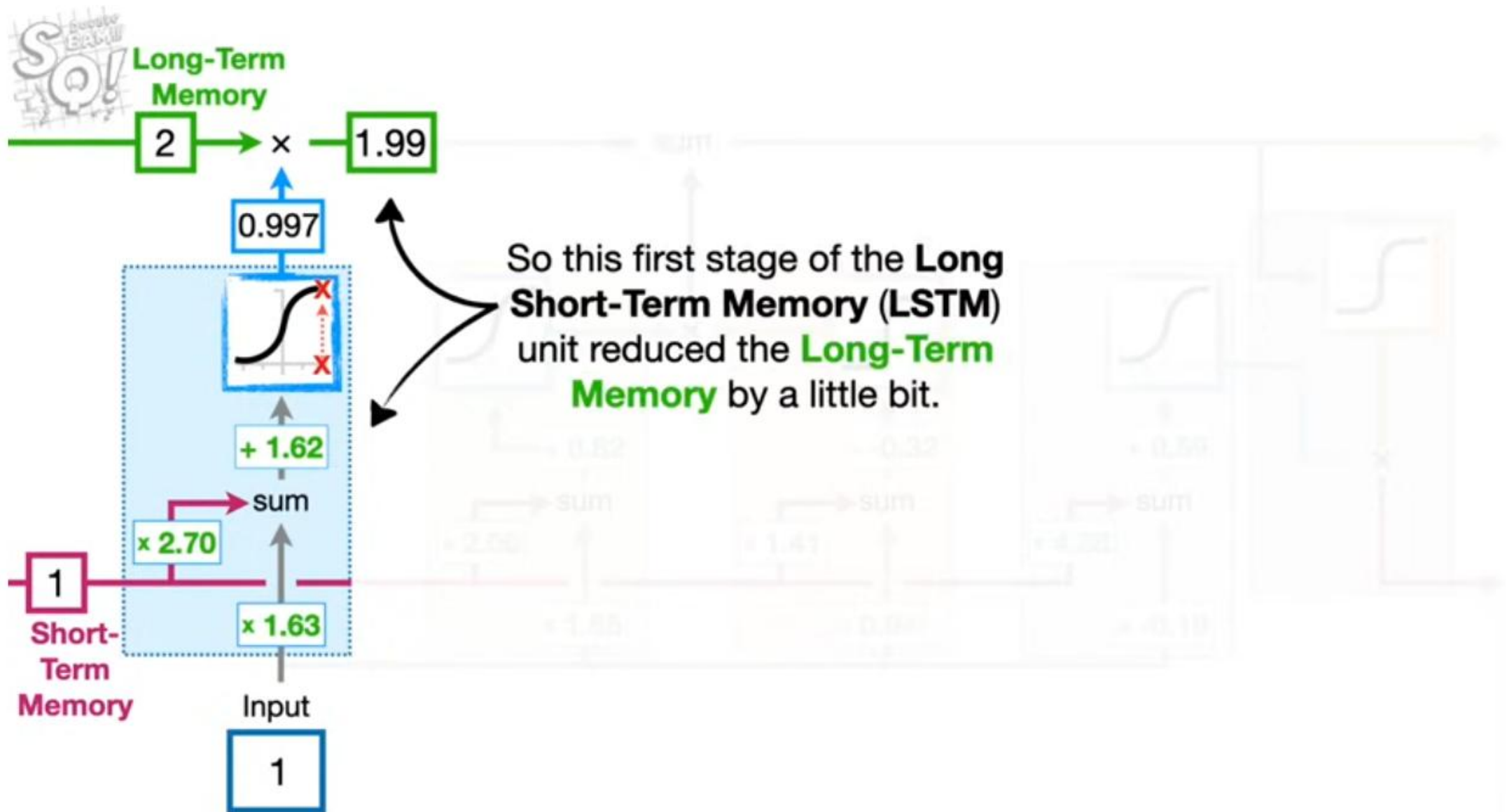
$$f(5.95) = \frac{e^{5.95}}{e^{5.95} + 1}$$

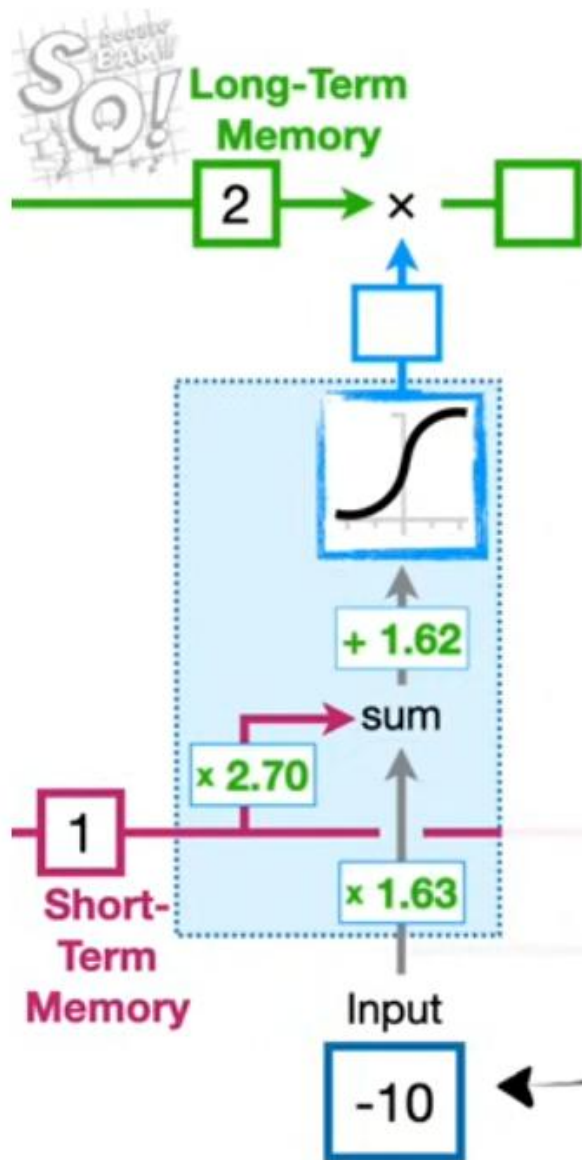
$$= 0.997$$



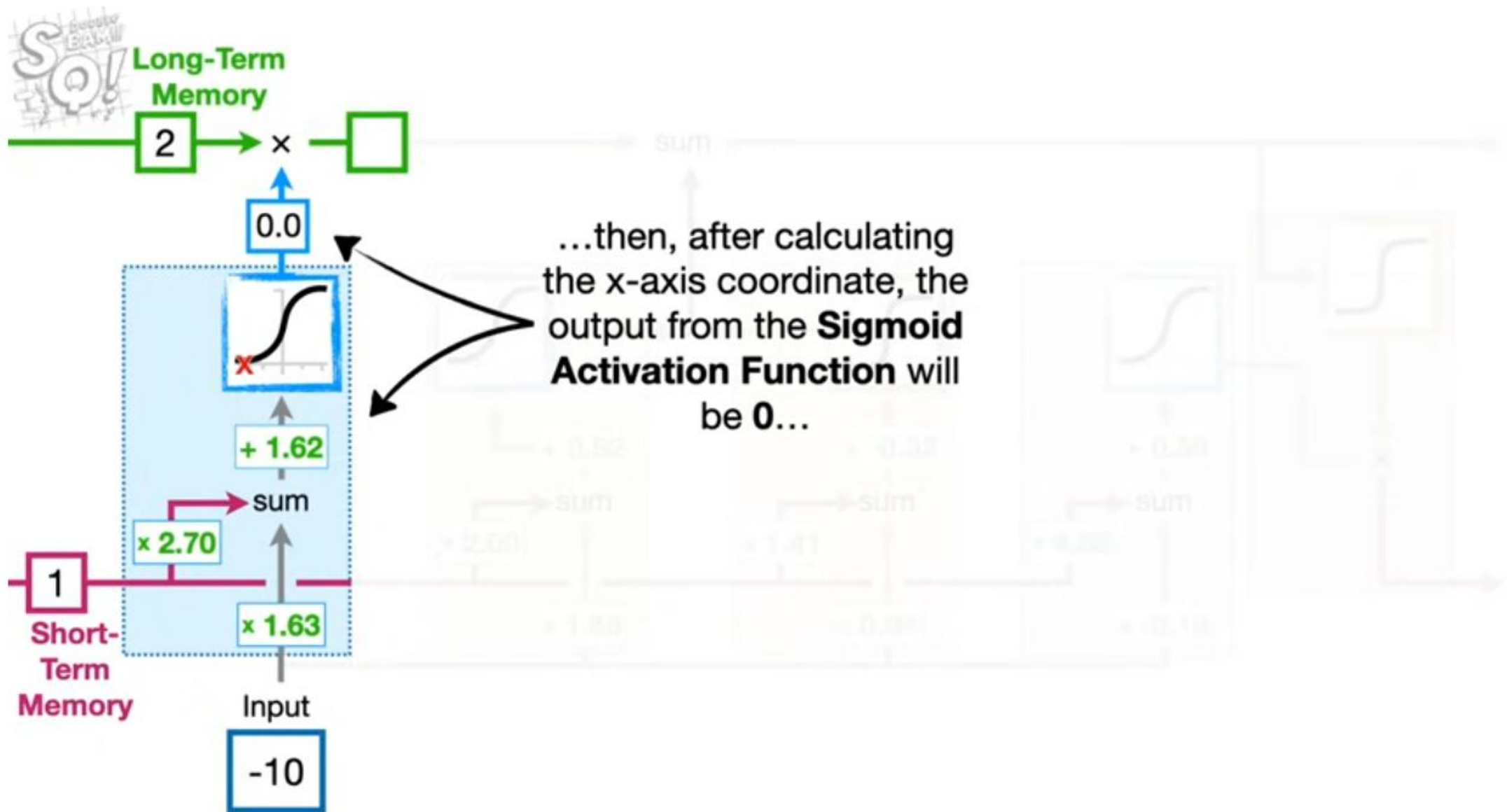


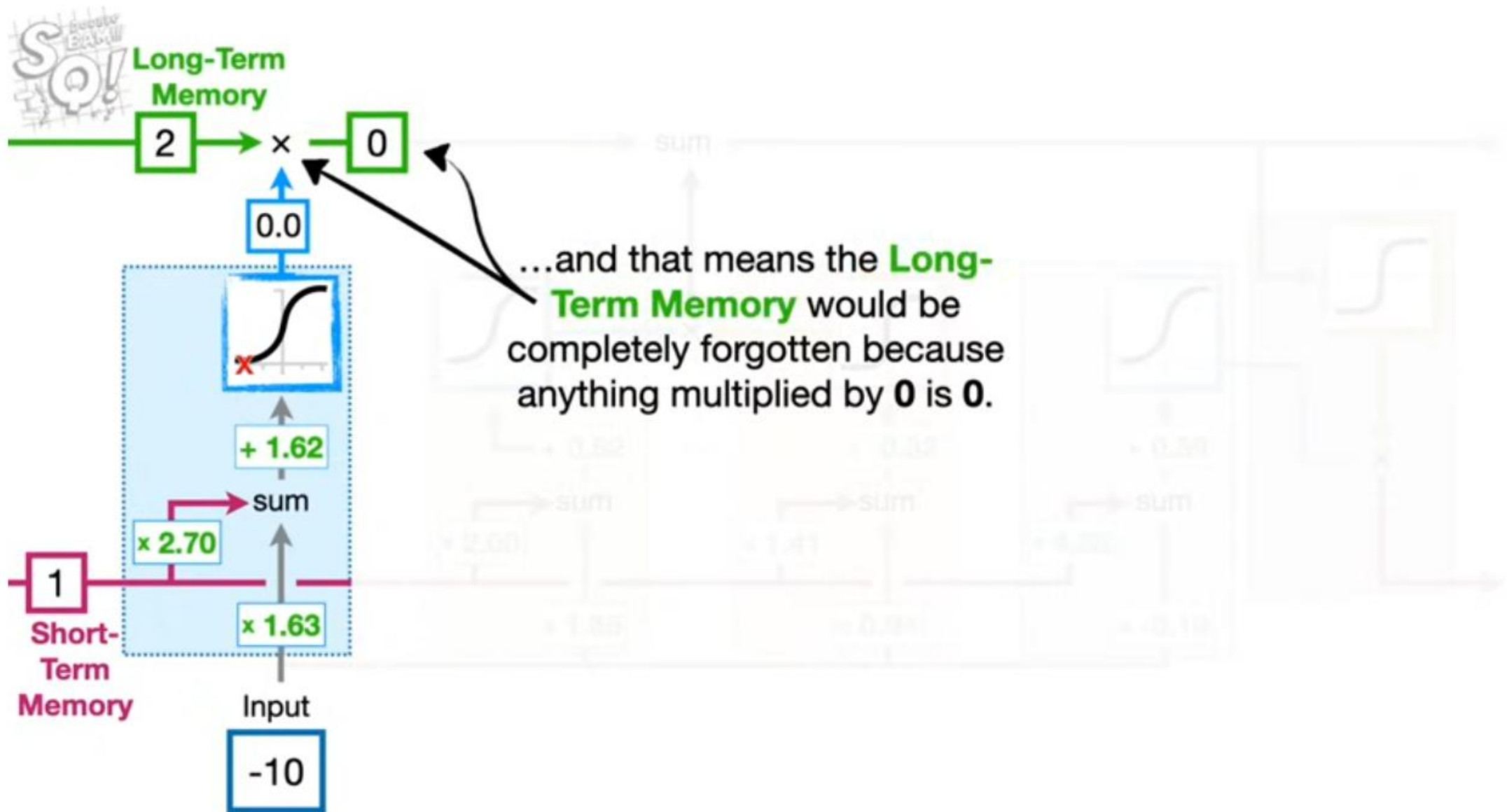


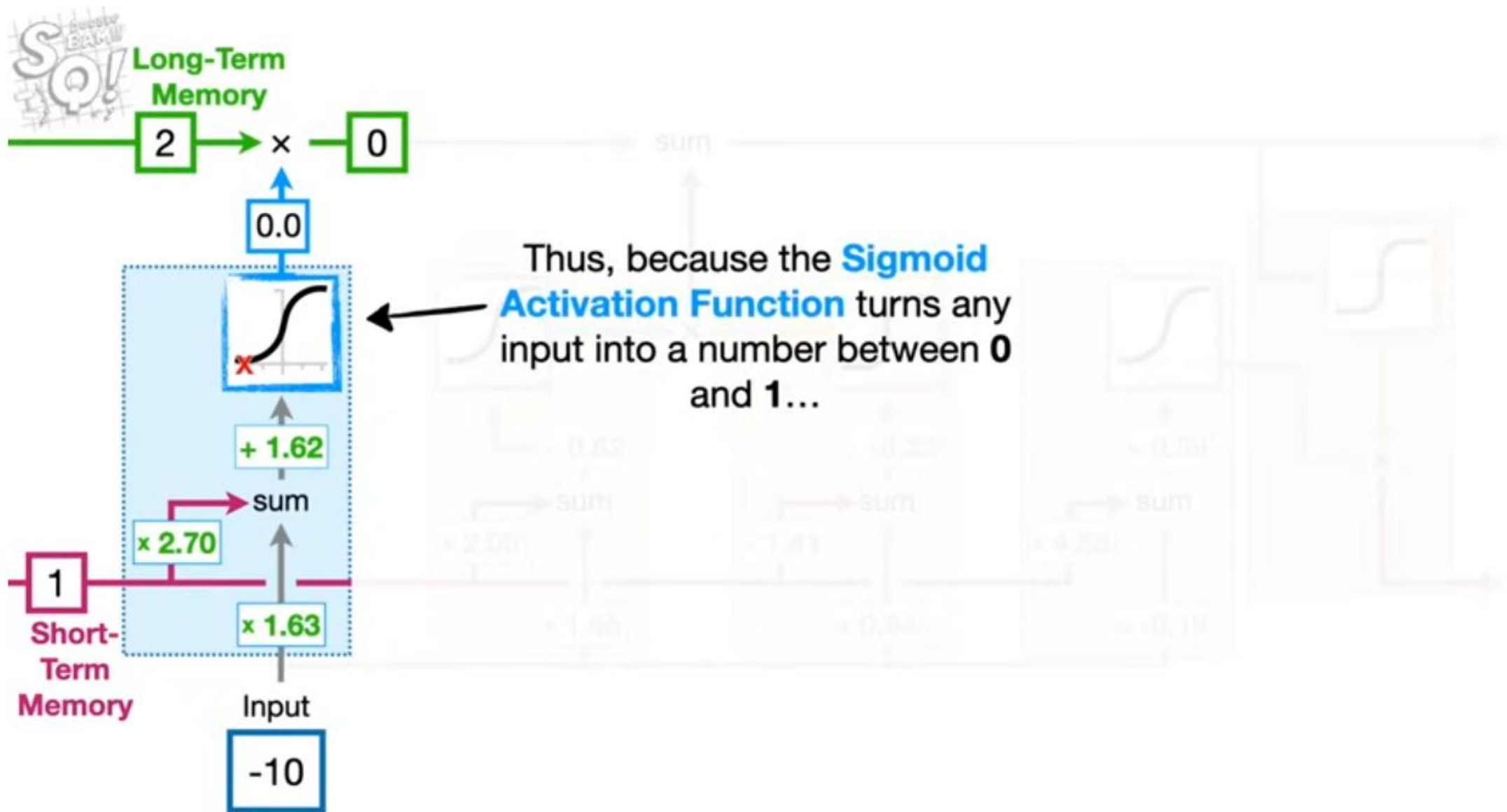


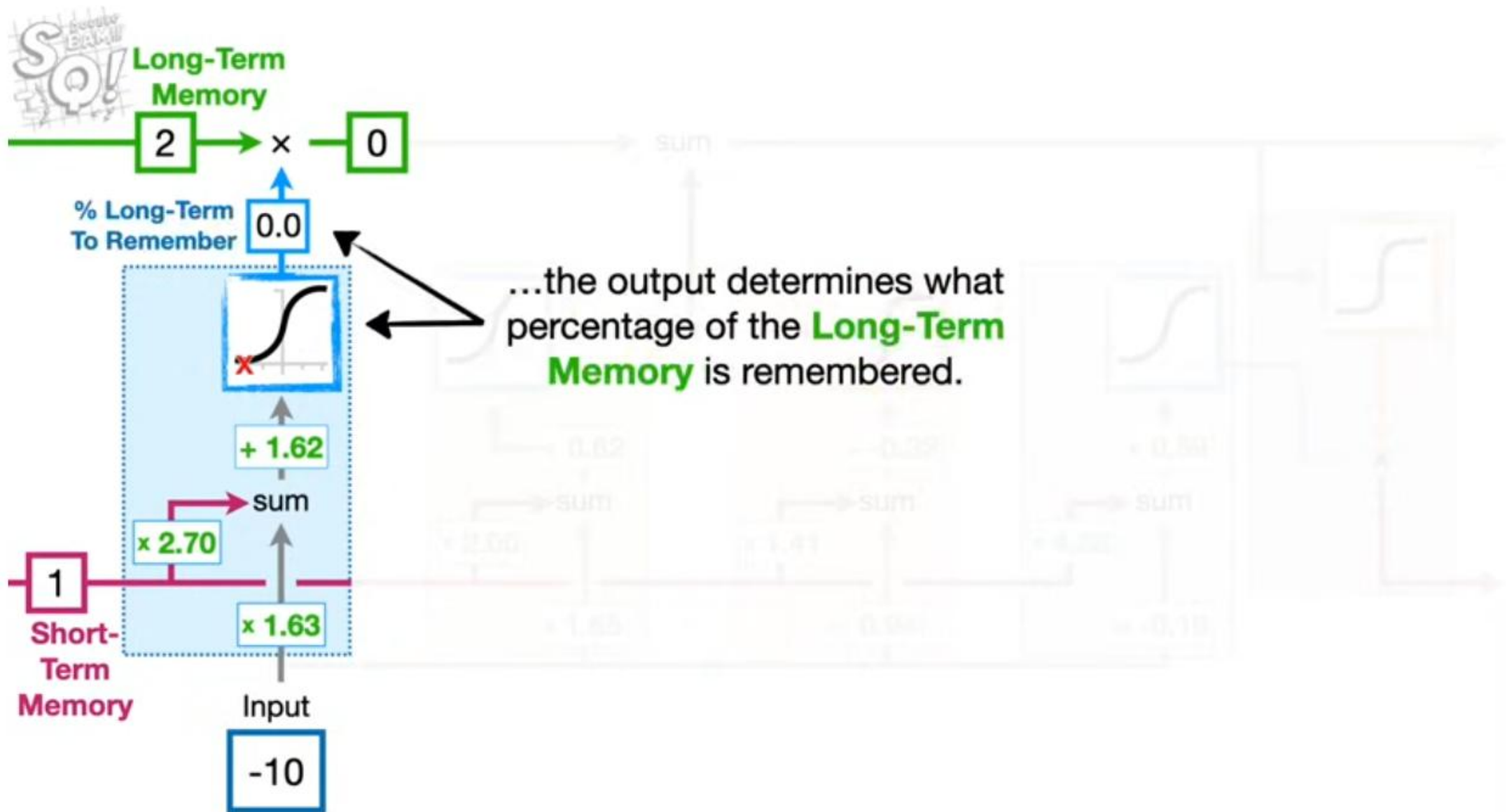


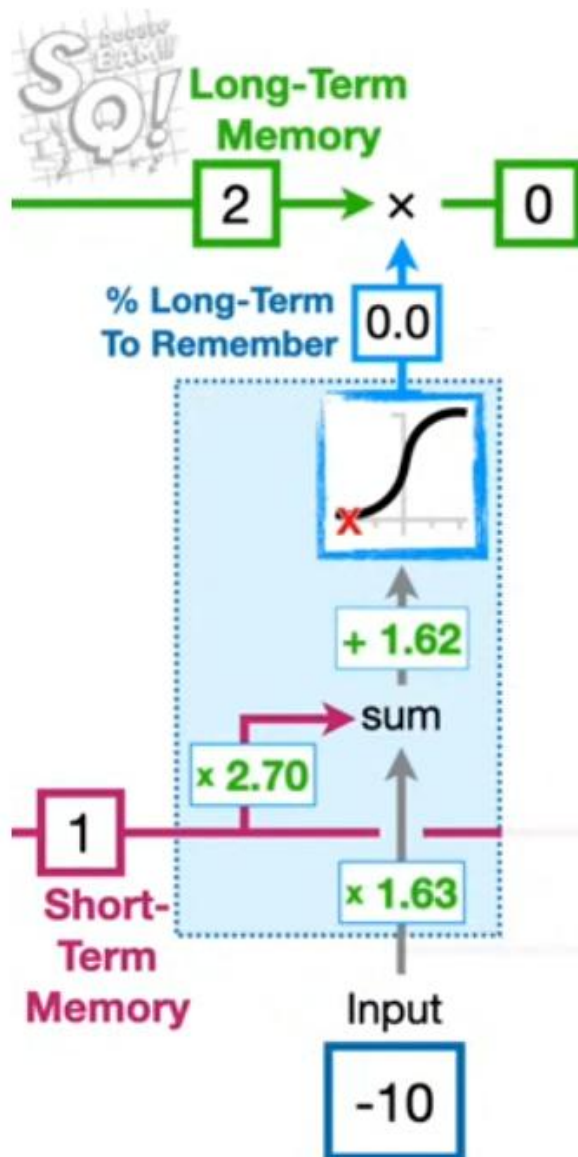
In contrast, if the **Input** to the **LSTM** was a relatively large negative number, like **-10**...





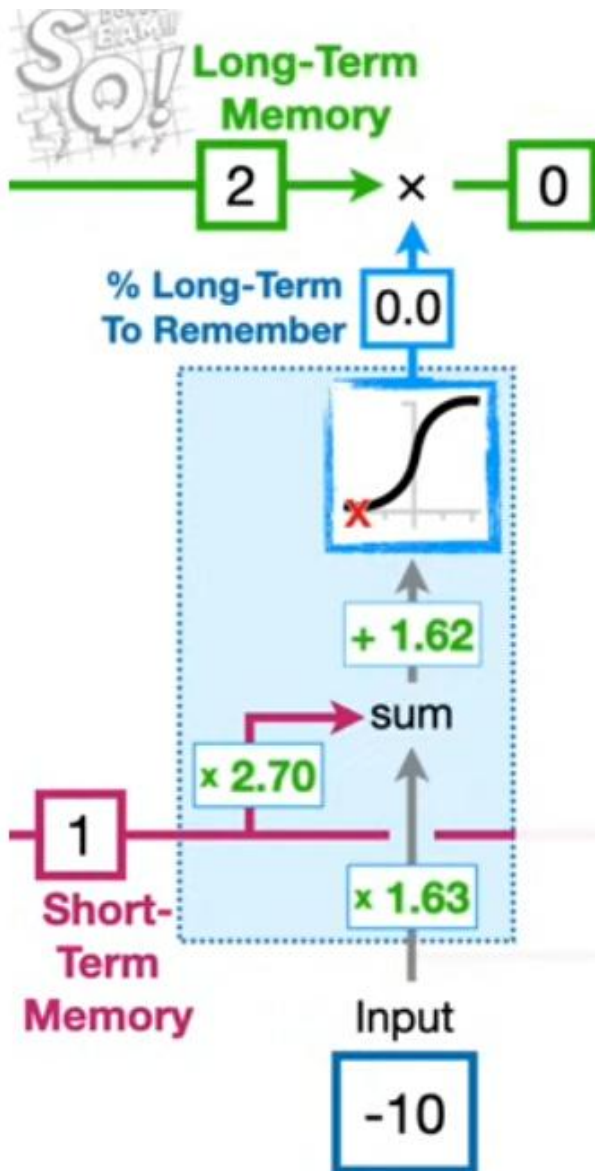




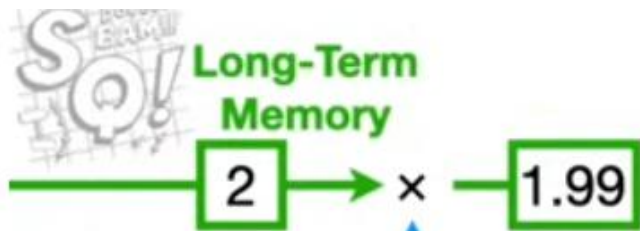


To summarize, the first stage in a **Long Short-Term Memory** unit determines what percentage of the **Long-Term Memory** is remembered.

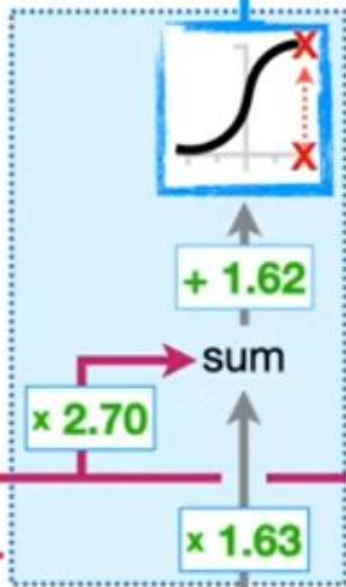
...it is usually called
the **Forget Gate**.



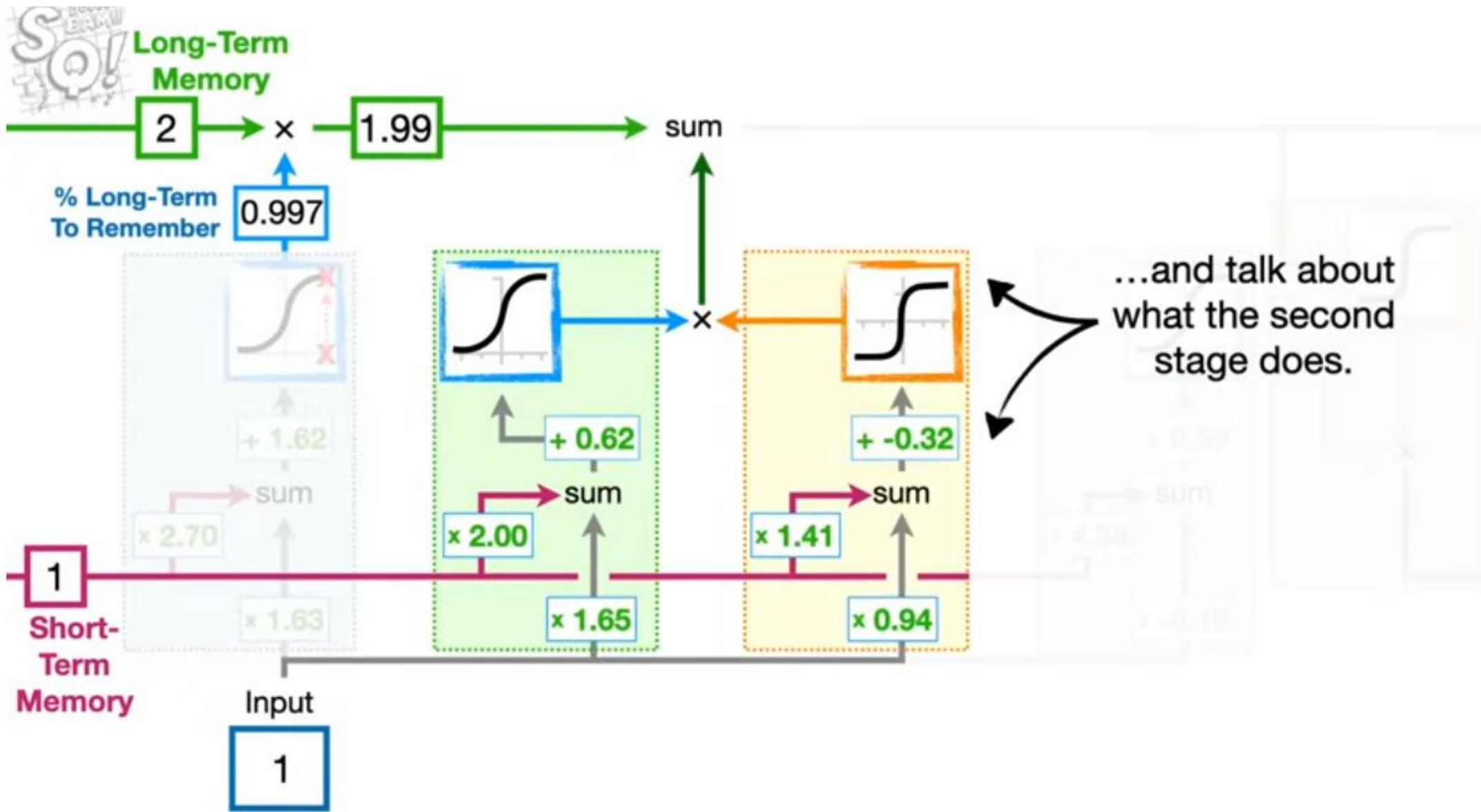
Now that we know what the first part of the **LSTM** unit does, it determines what percentage of the **Long-Term Memory** will be remembered...



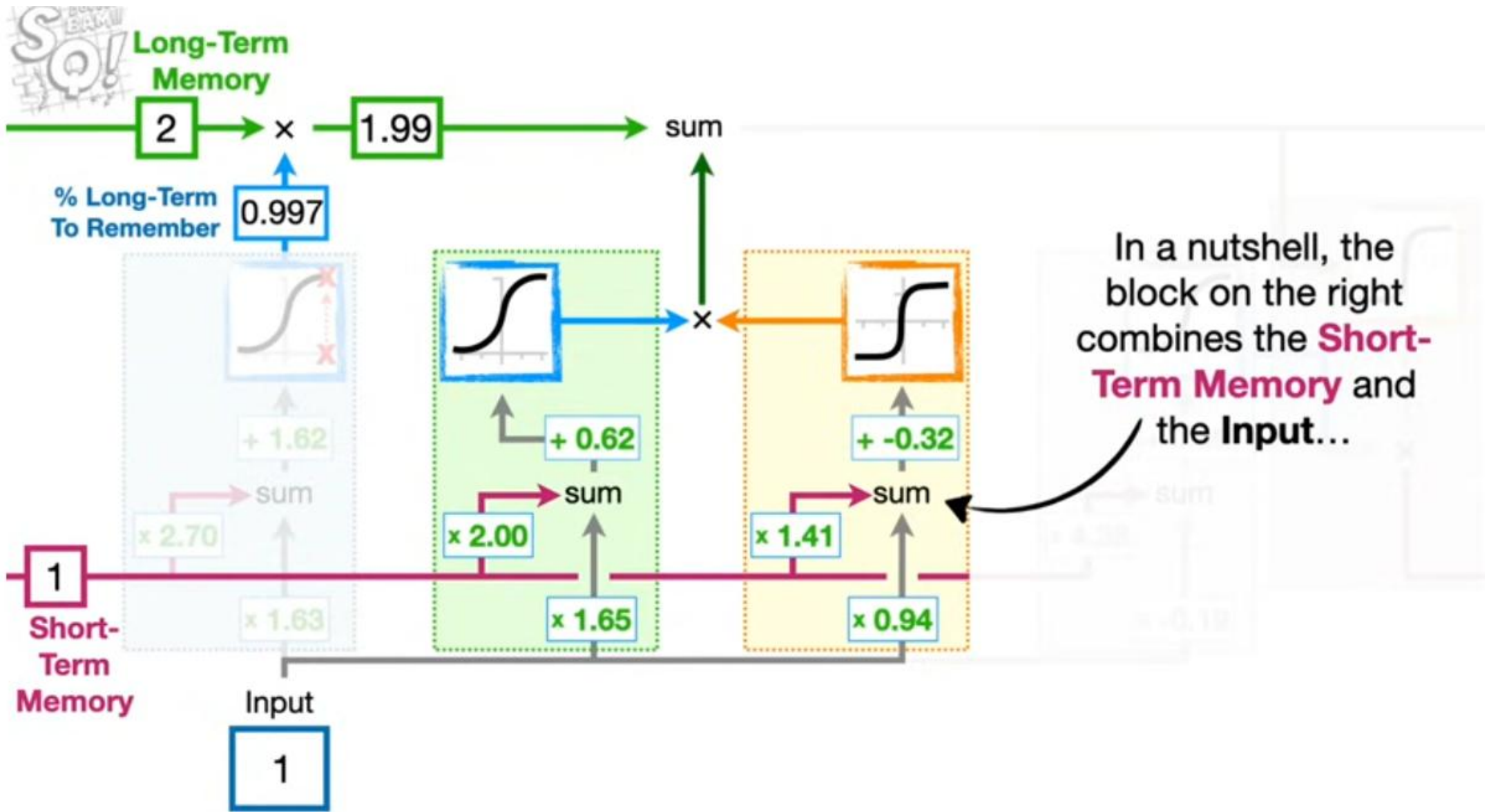
% Long-Term To Remember 0.997

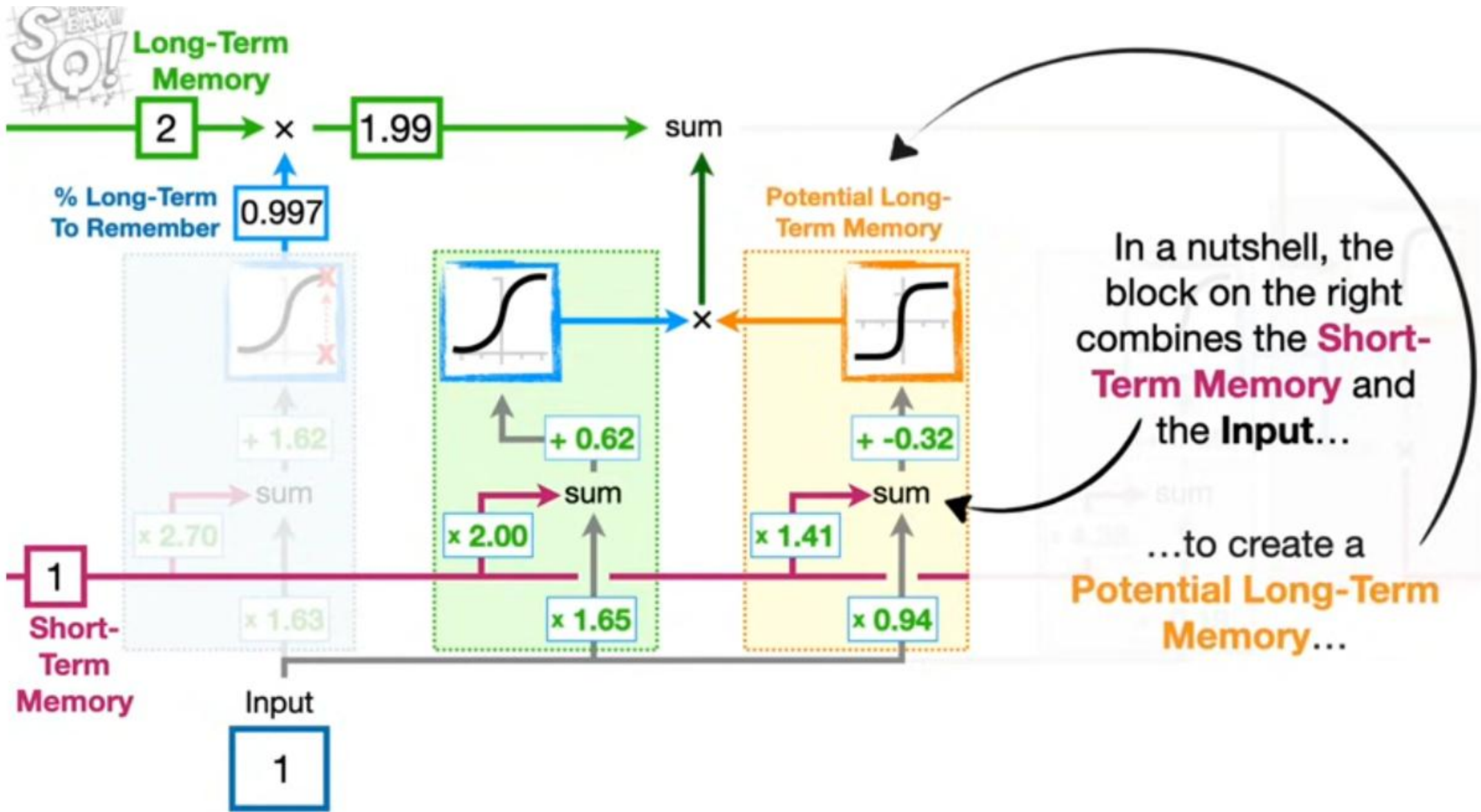


...let's go back to when the **Input** was 1...

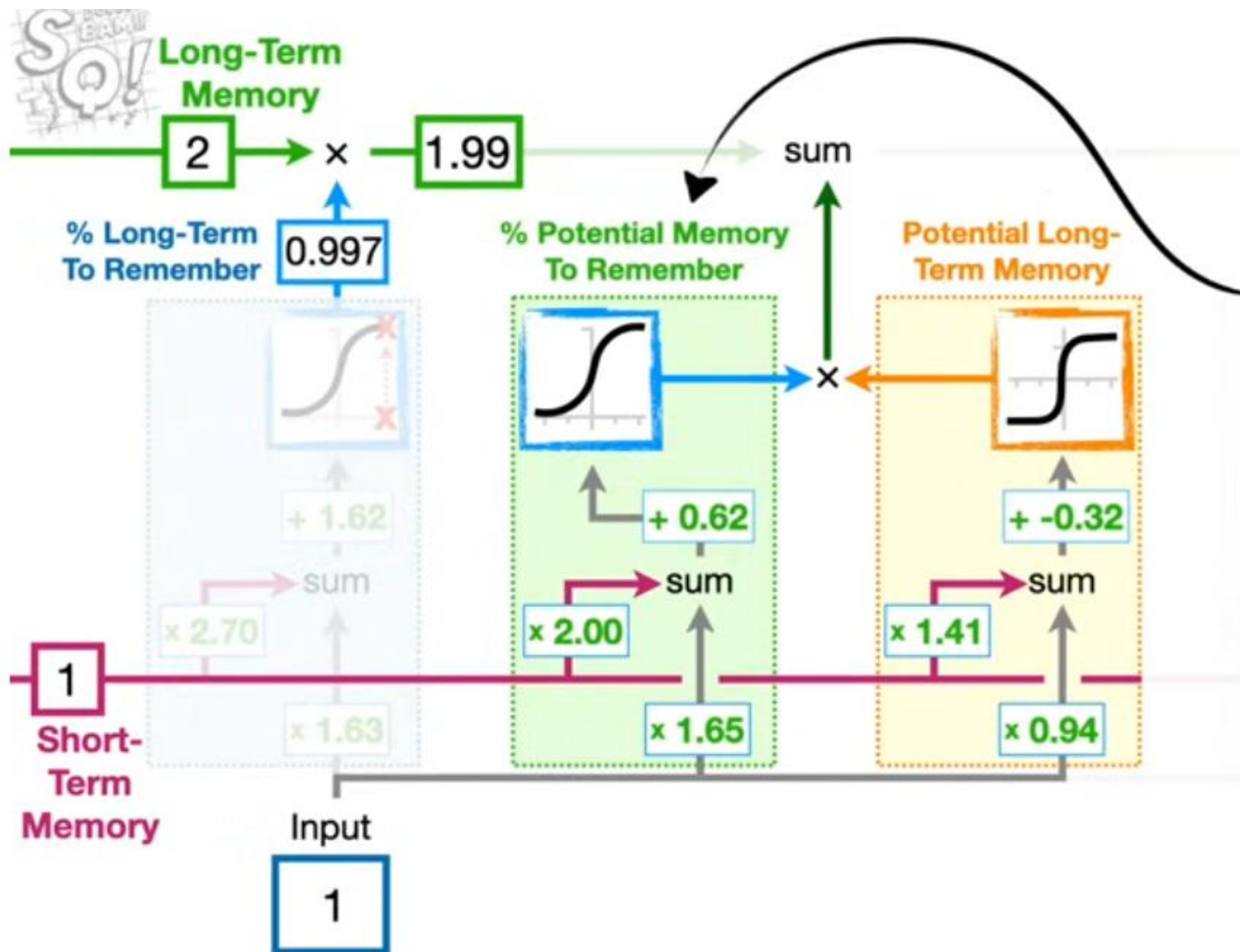


...and talk about what the second stage does.

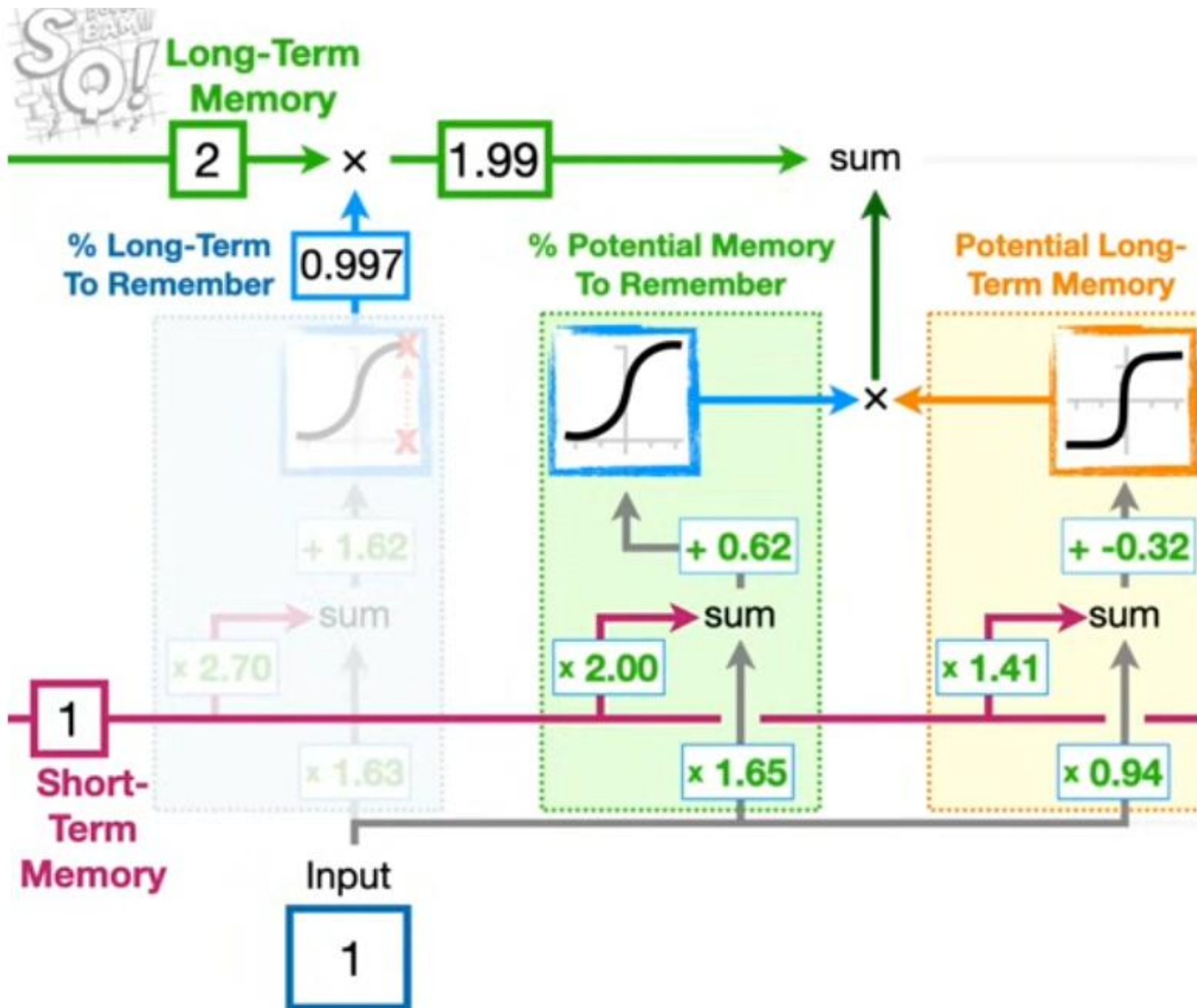




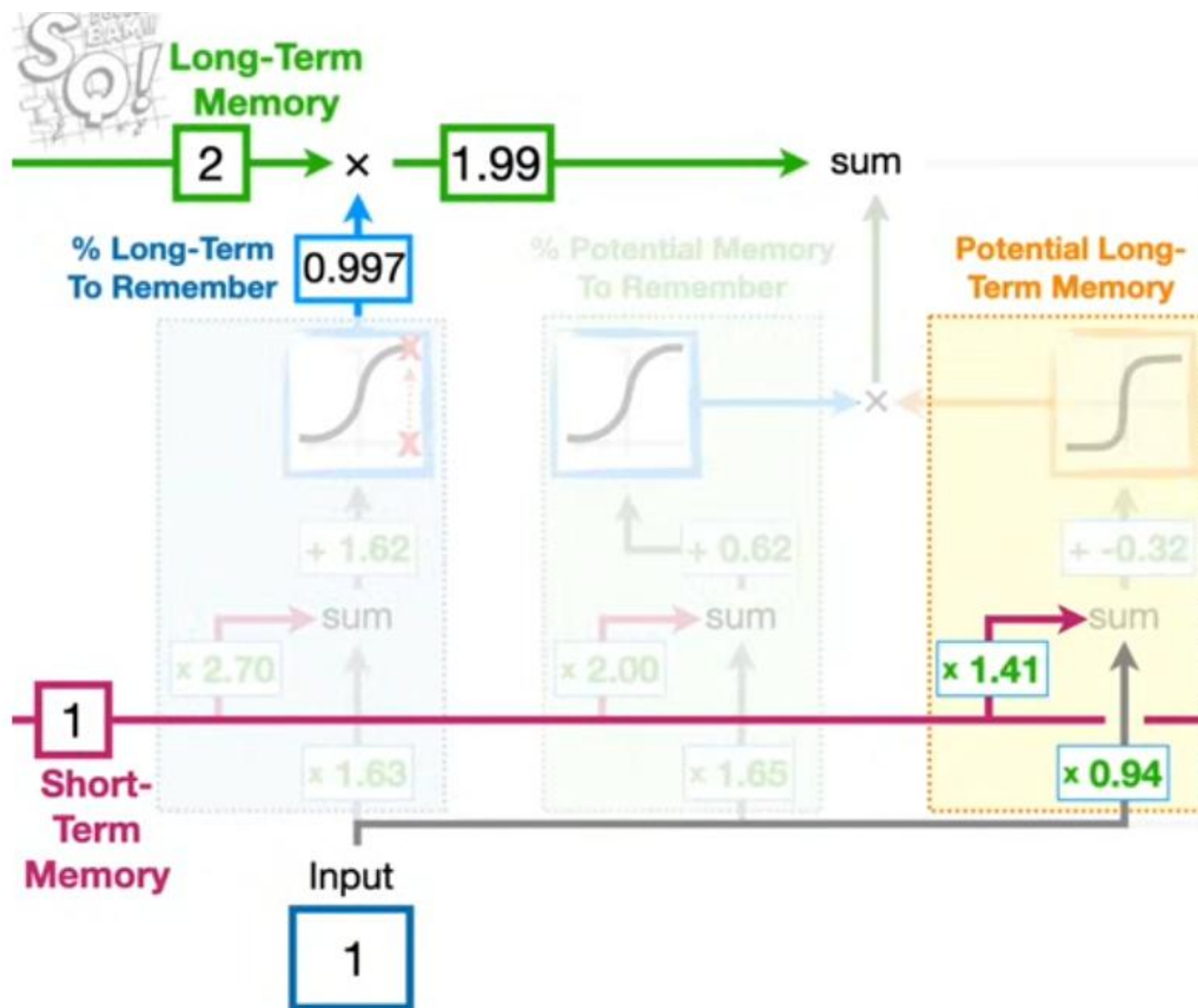
In a nutshell, the block on the right combines the **Short-Term Memory** and the **Input**...
...to create a **Potential Long-Term Memory**...



...and the block on the left determines what percentage of that **Potential Memory** to add to the **Long-Term Memory**.

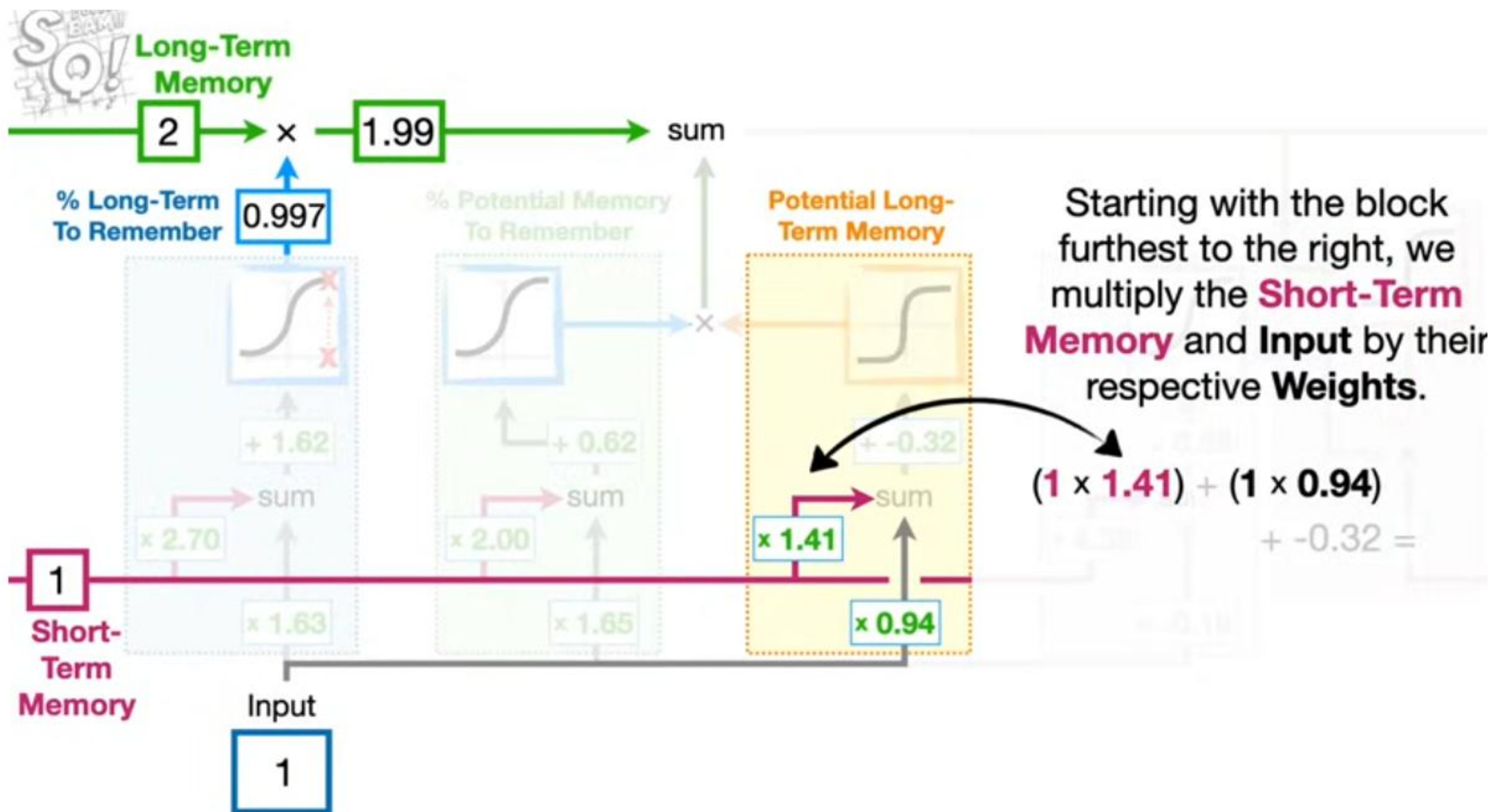


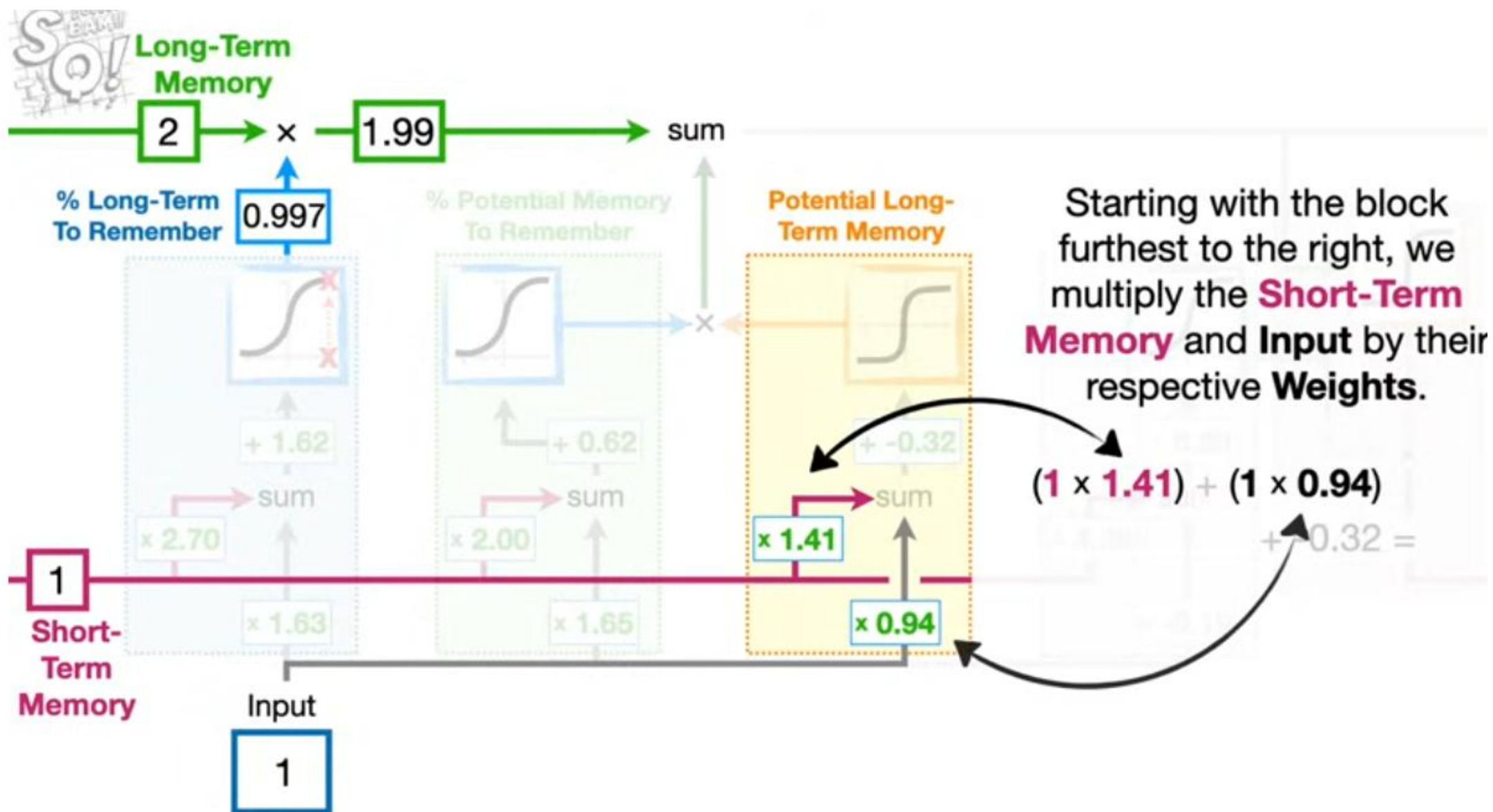
So let's plug the numbers in and do the math to see how a **Potential Memory** is created and how much of it is added to the **Long-Term Memory**.

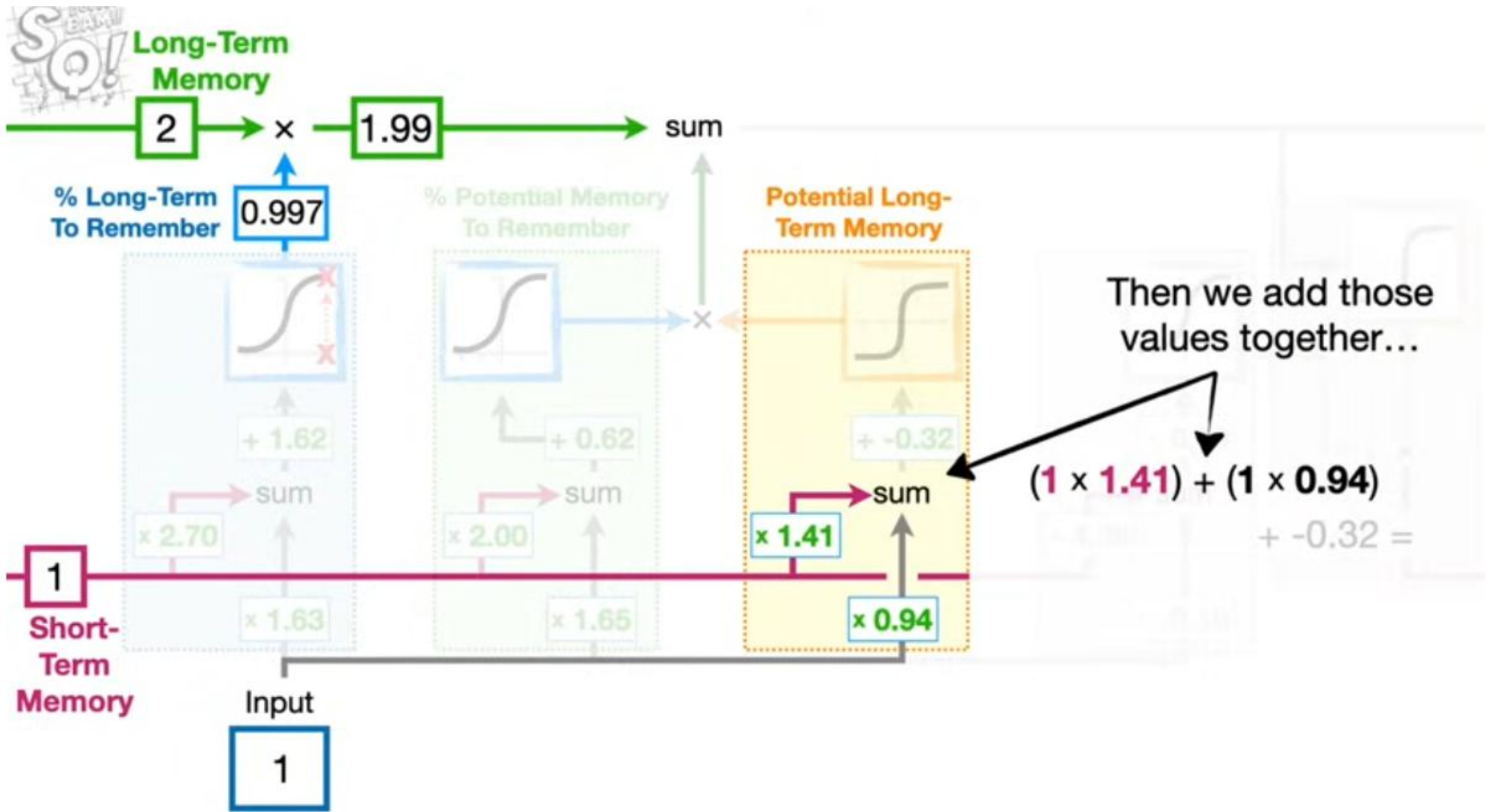


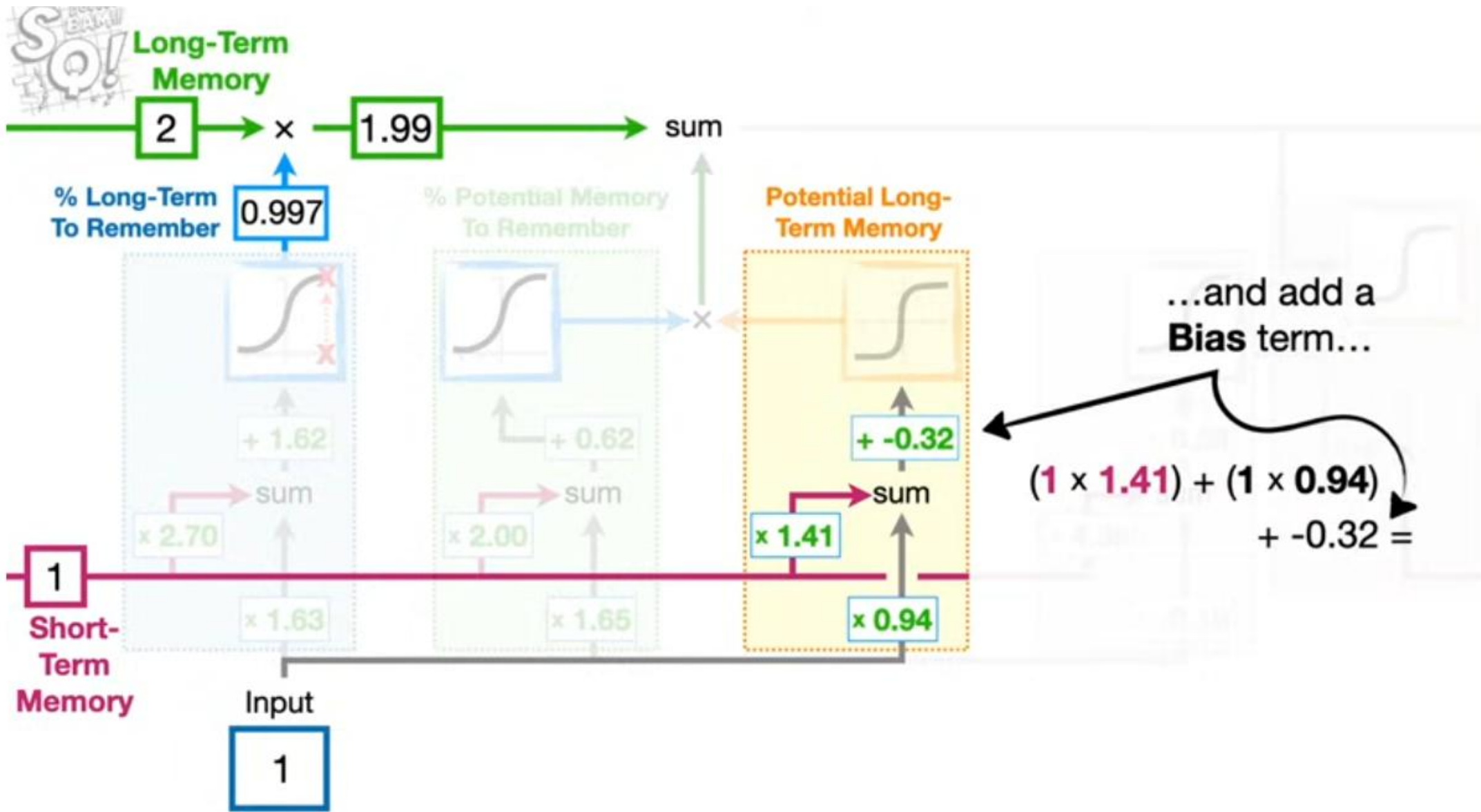
Starting with the block furthest to the right, we multiply the **Short-Term Memory** and **Input** by their respective **Weights**.

$$(1 \times 1.41) + (1 \times 0.94) + -0.32 =$$



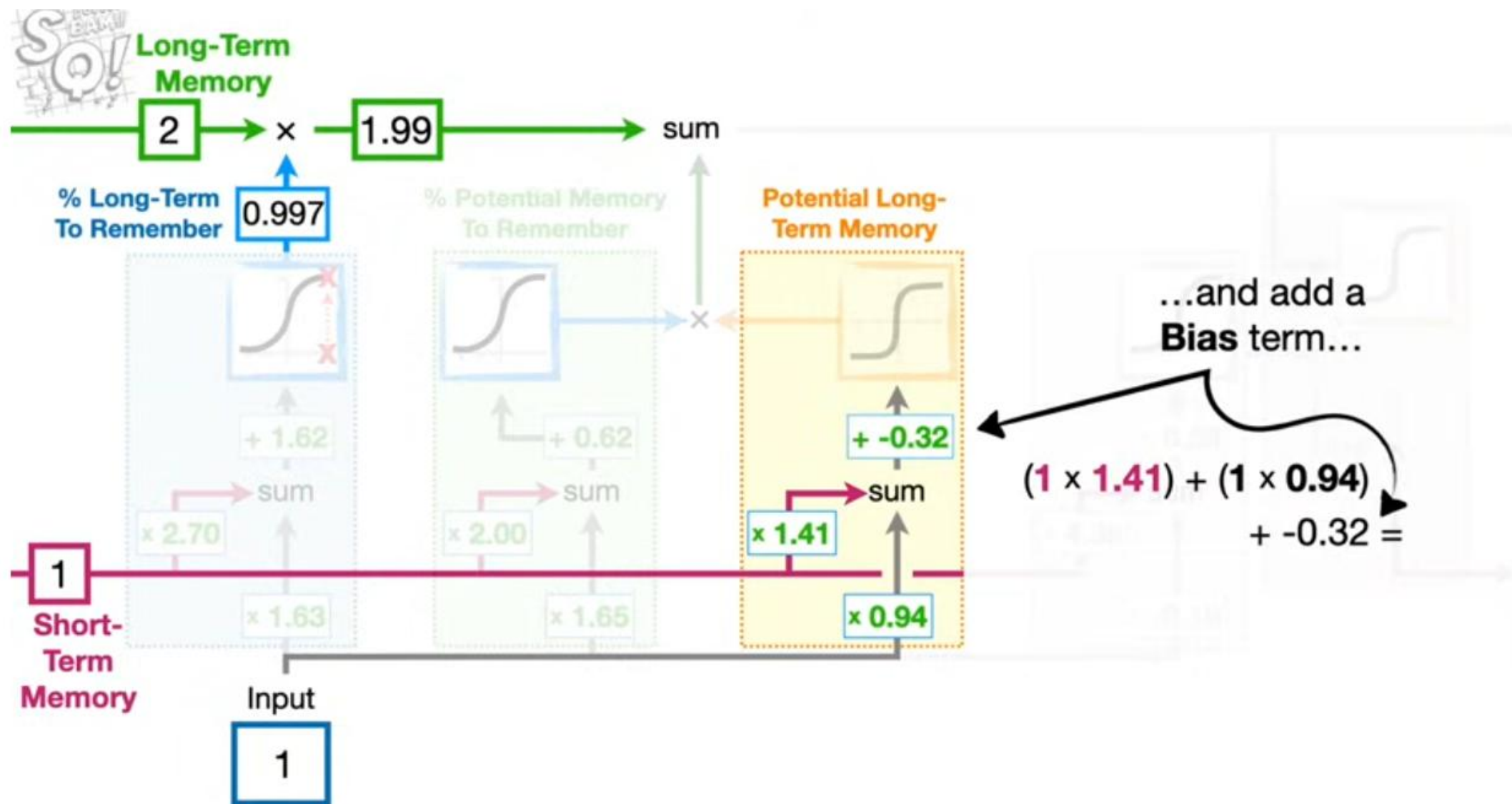


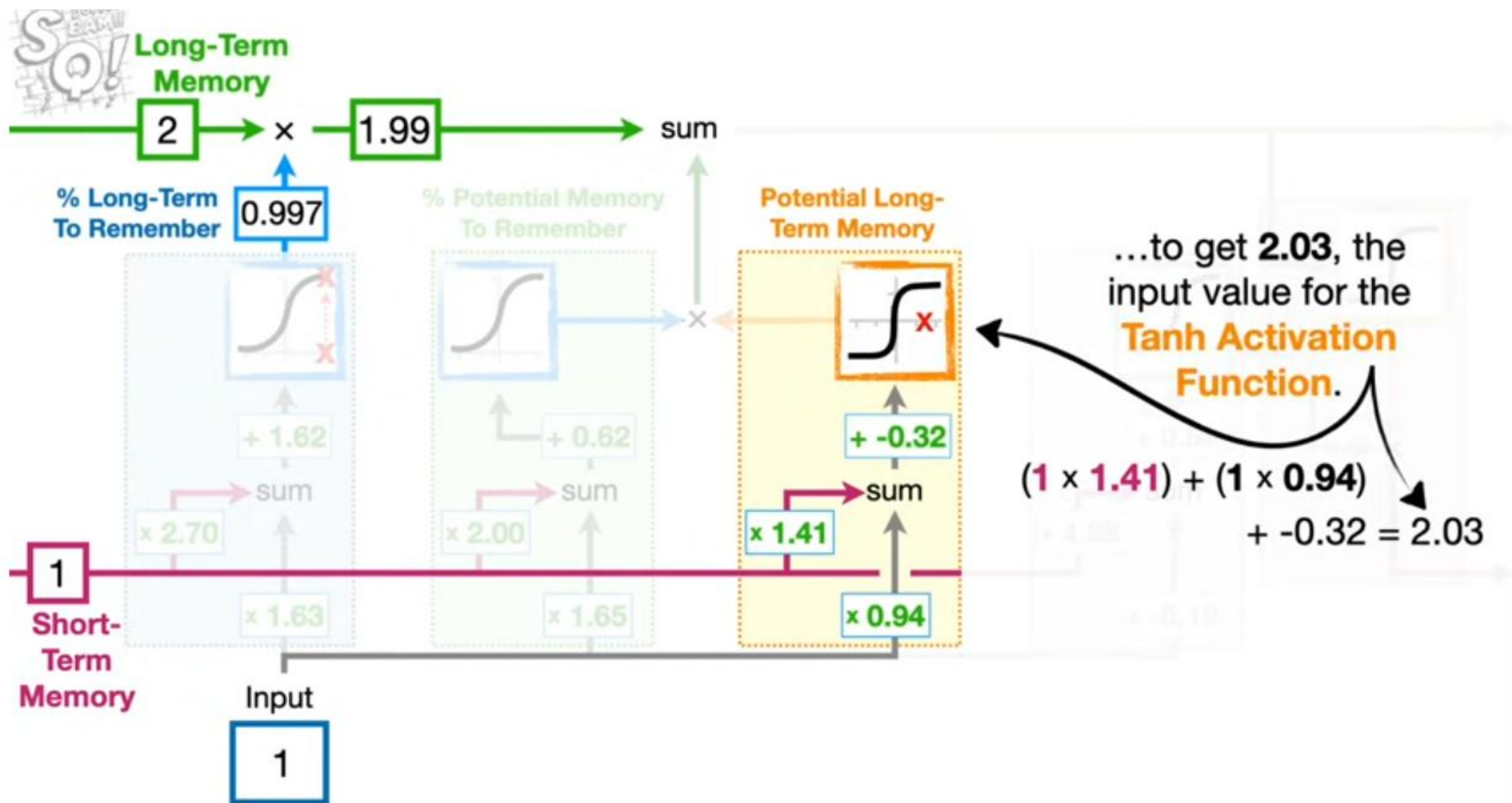


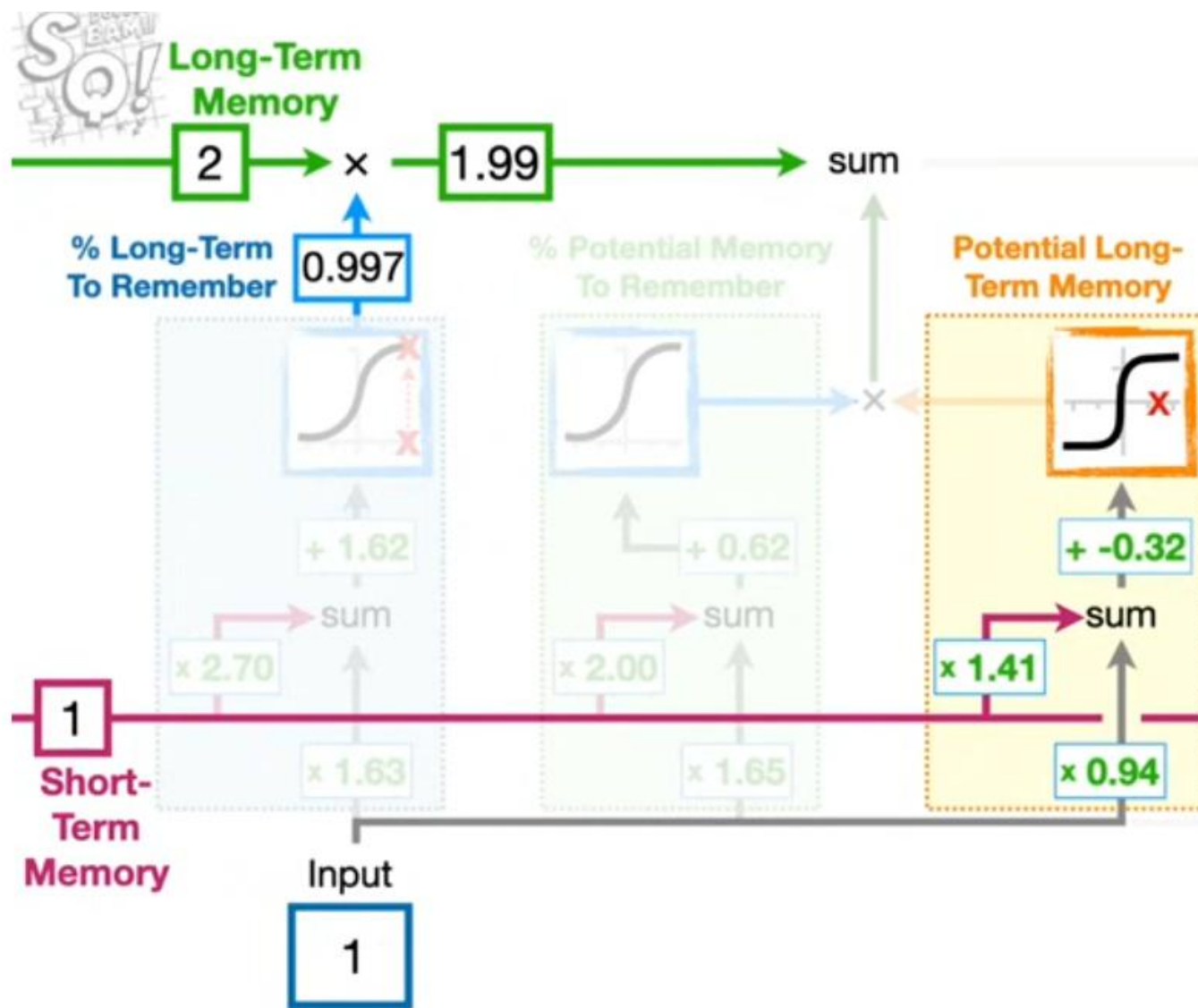


...and add a **Bias** term...

$$(1 \times 1.41) + (1 \times 0.94) + -0.32 =$$

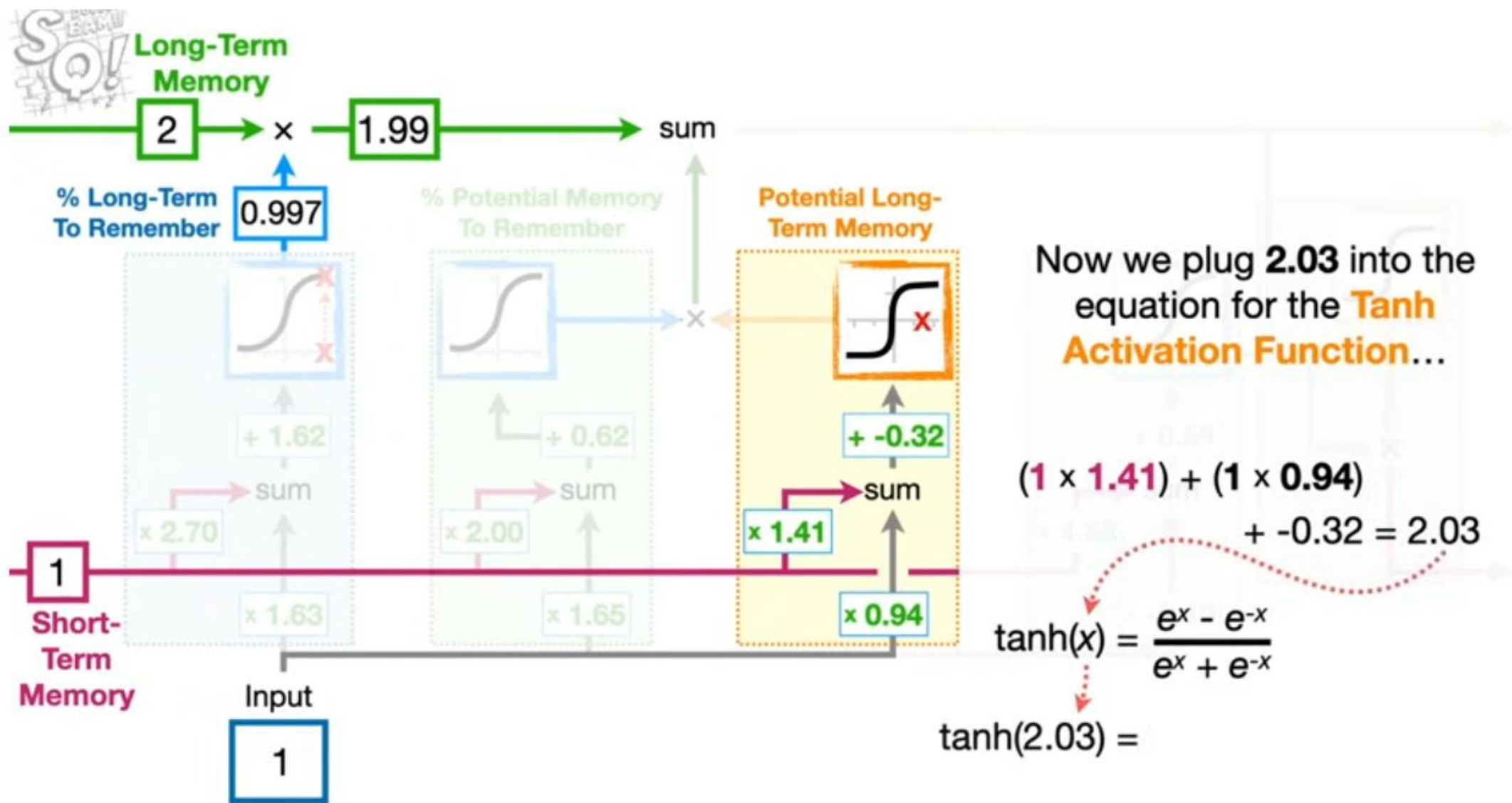


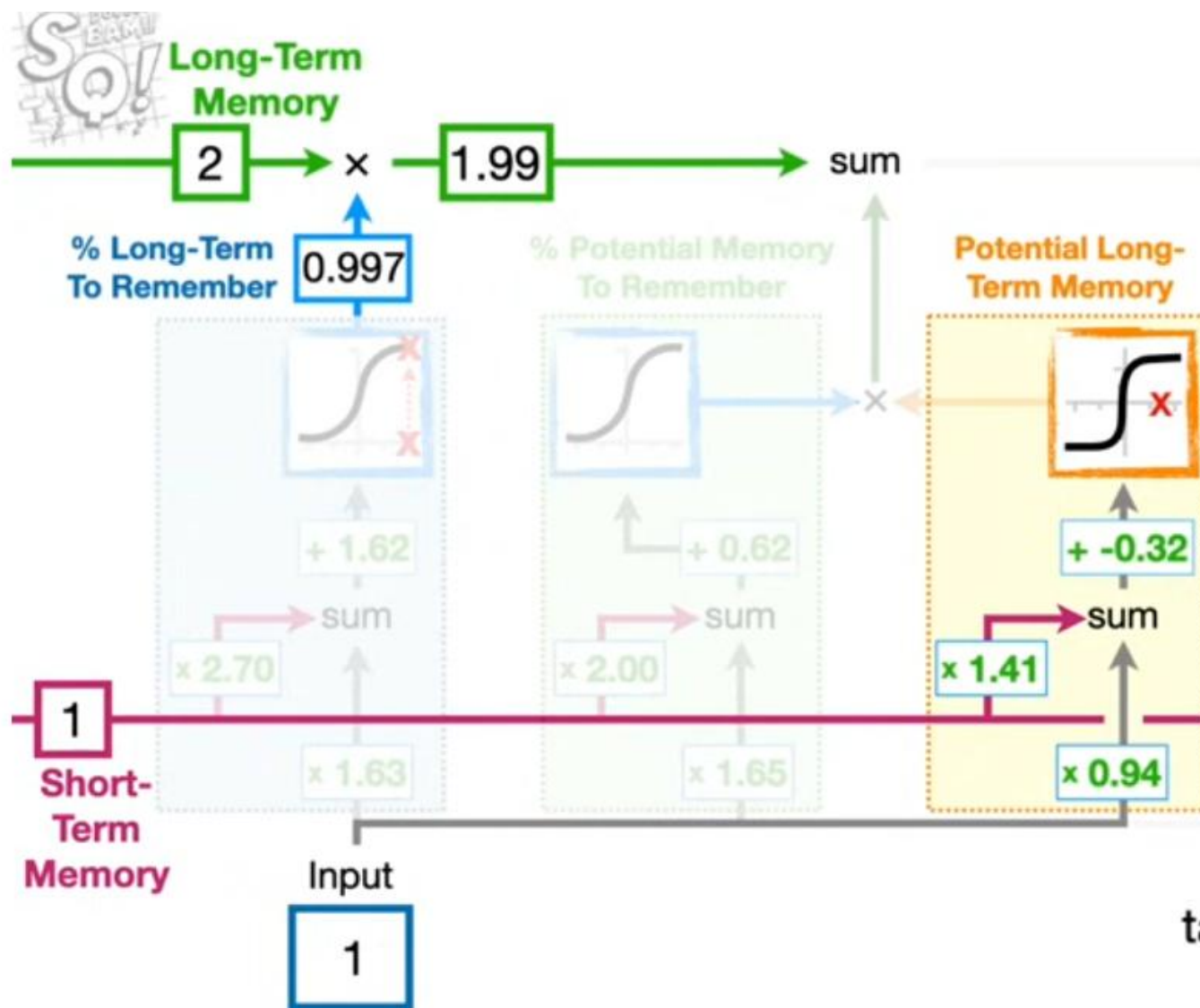




Now we plug **2.03** into the equation for the **Tanh Activation Function**...

$$(1 \times 1.41) + (1 \times 0.94) + -0.32 = 2.03$$



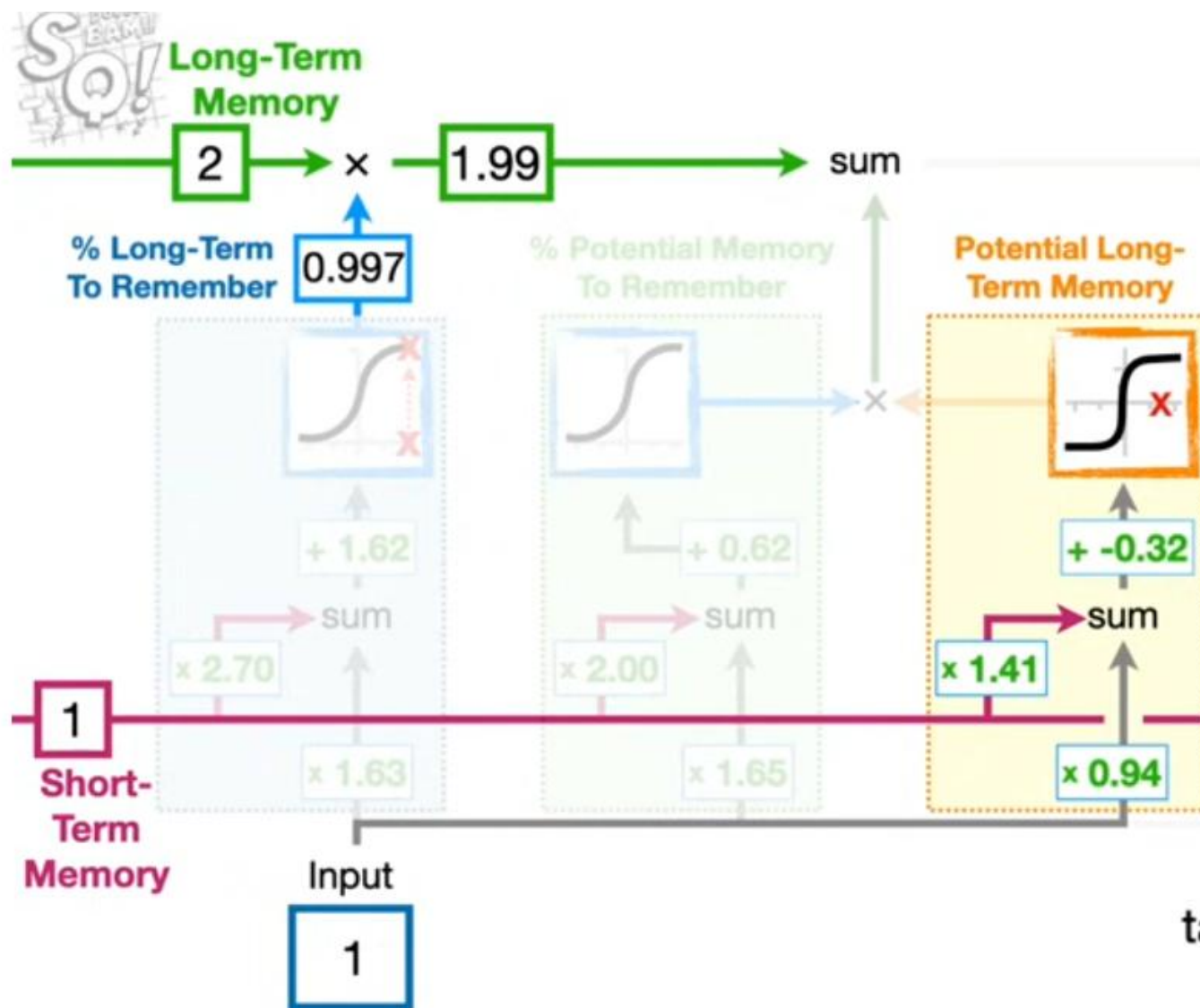


Now we plug **2.03** into the equation for the **Tanh Activation Function**...

$$(1 \times 1.41) + (1 \times 0.94) + -0.32 = 2.03$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh(2.03) = \frac{e^{2.03} - e^{-2.03}}{e^{2.03} + e^{-2.03}} = 0.97$$

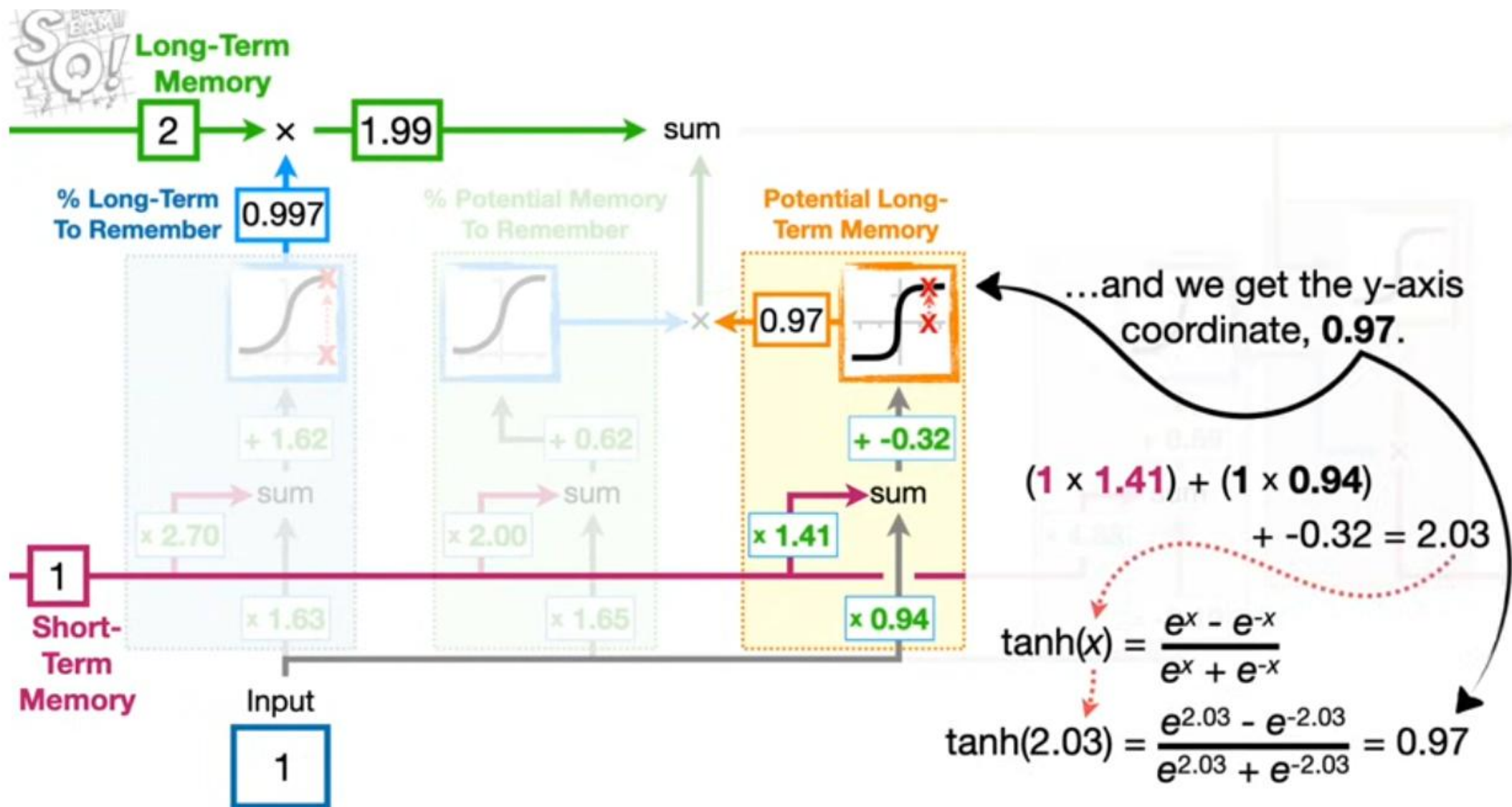


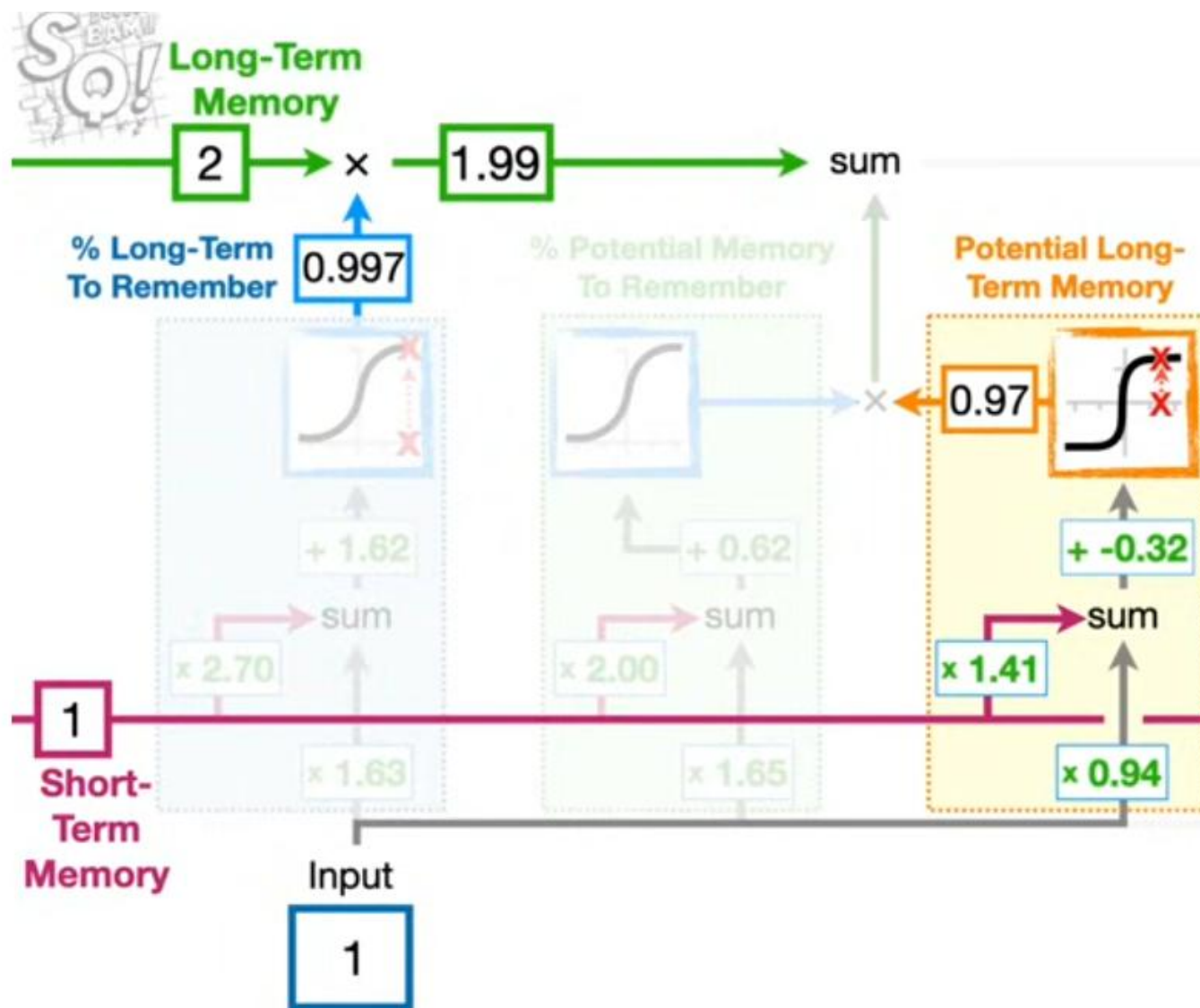
Now we plug **2.03** into the equation for the **Tanh Activation Function**...

$$(1 \times 1.41) + (1 \times 0.94) + -0.32 = 2.03$$

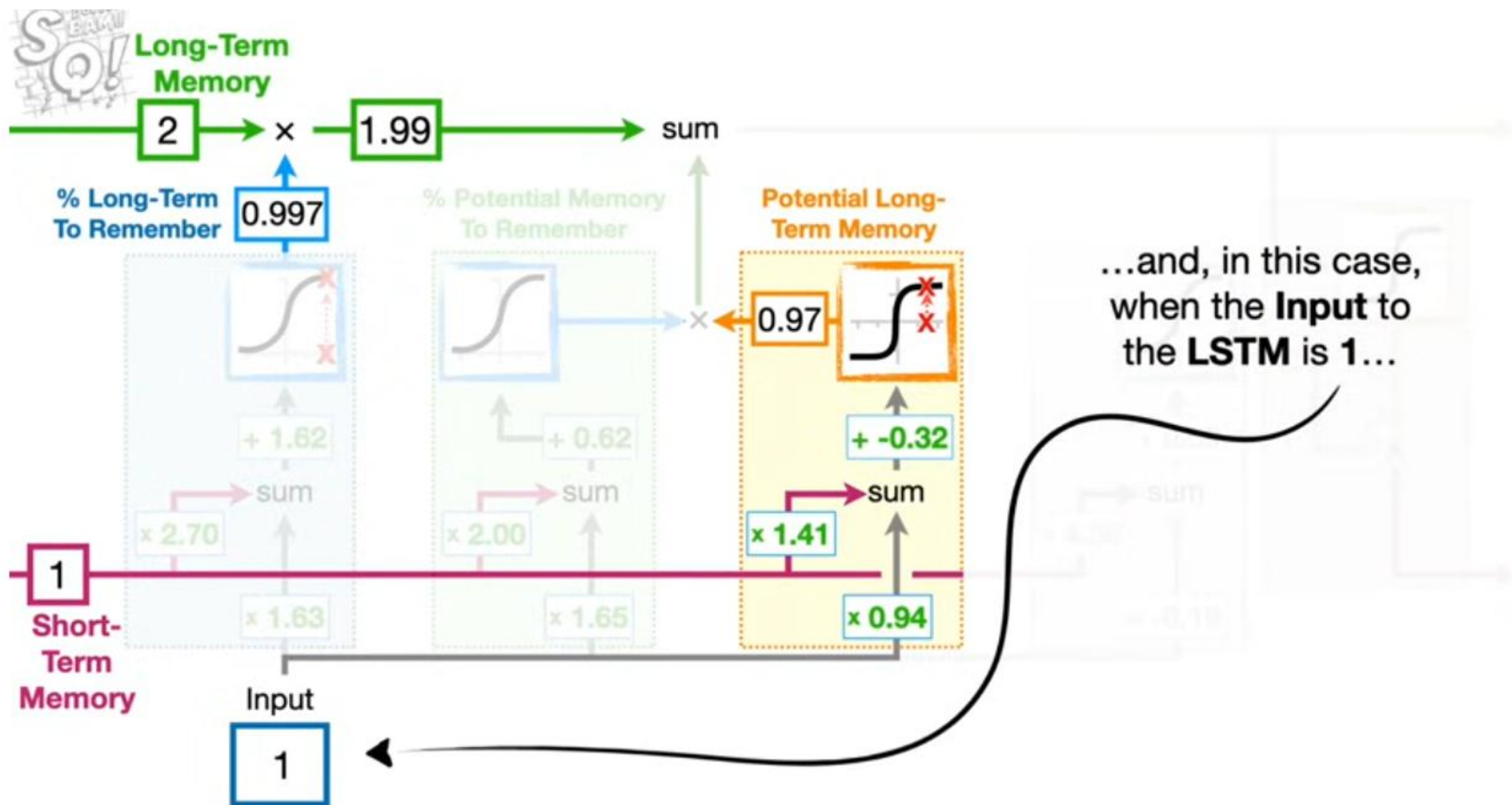
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh(2.03) = \frac{e^{2.03} - e^{-2.03}}{e^{2.03} + e^{-2.03}} = 0.97$$

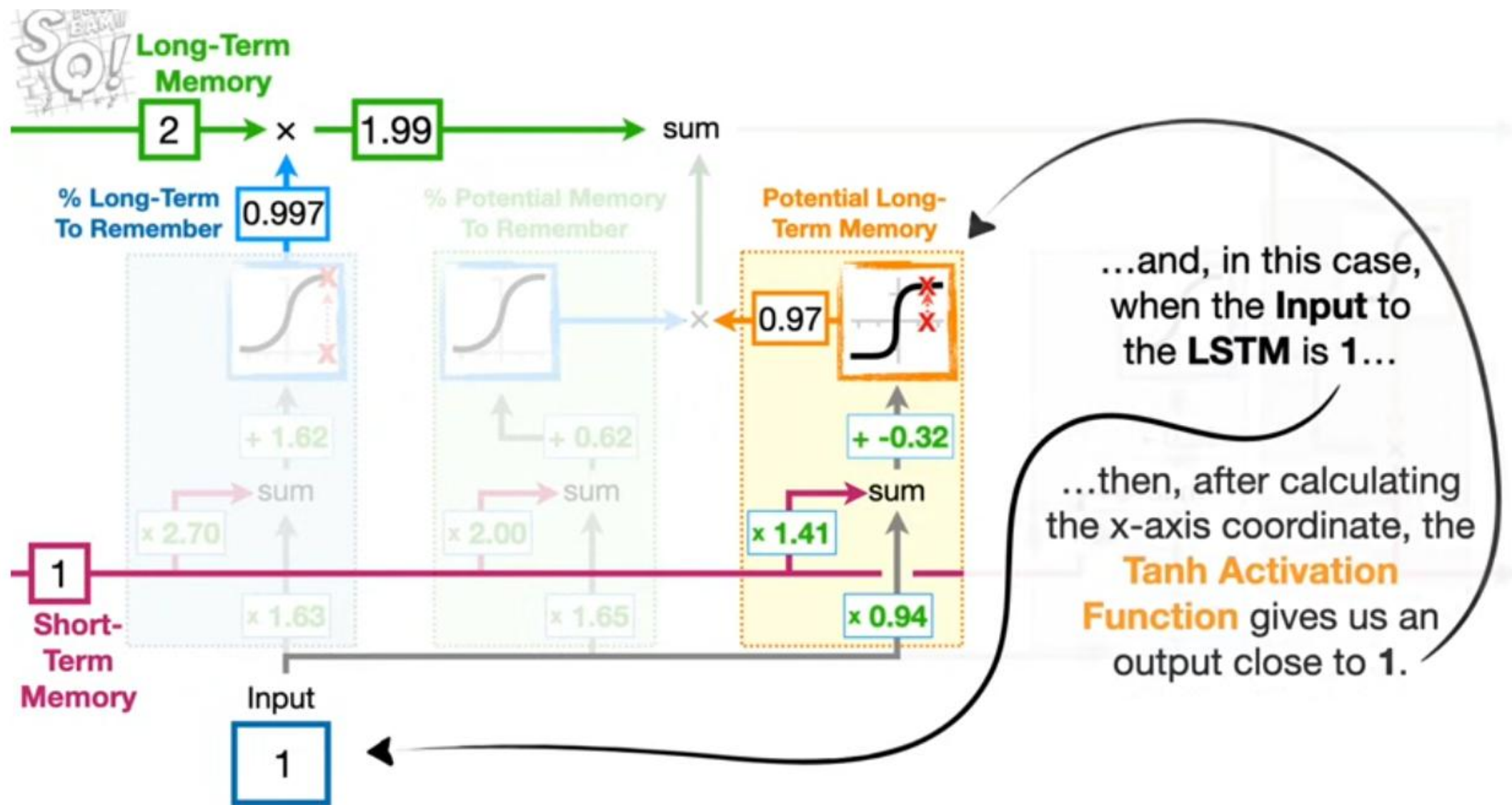


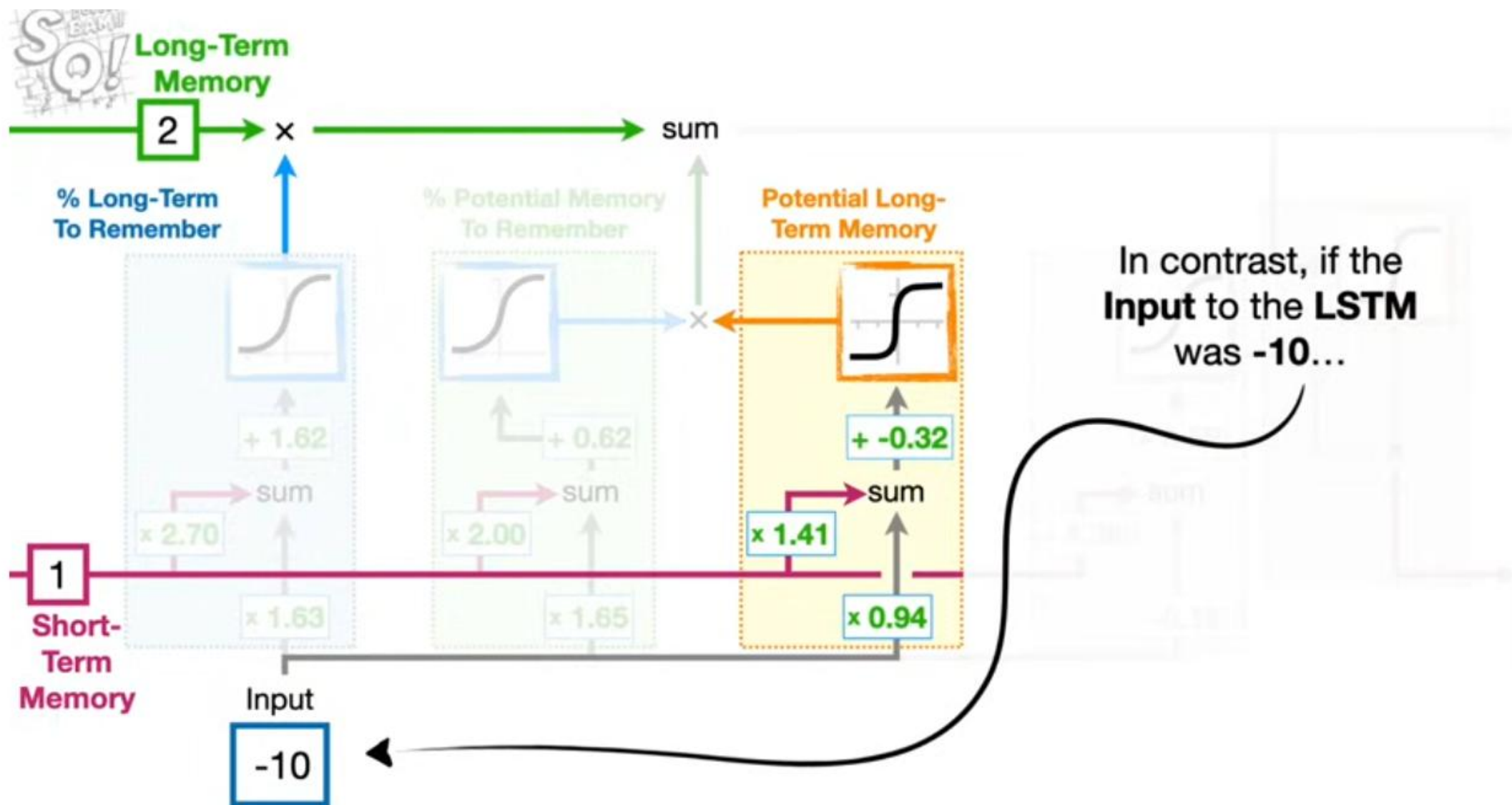


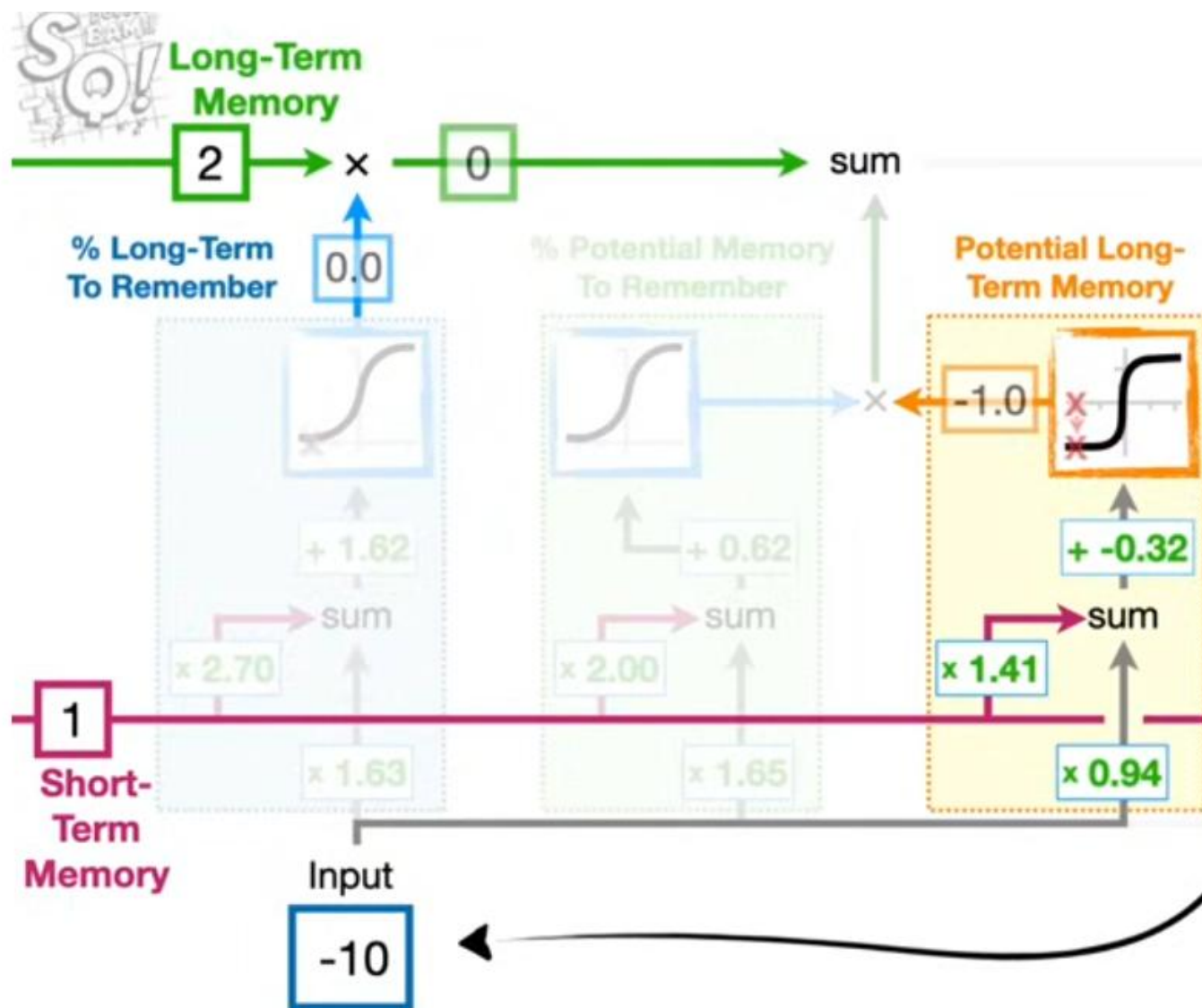
Remember, the **Tanh Activation Function** turns any input into a number between **-1** and **1**...



...and, in this case,
when the **Input** to
the **LSTM** is 1...

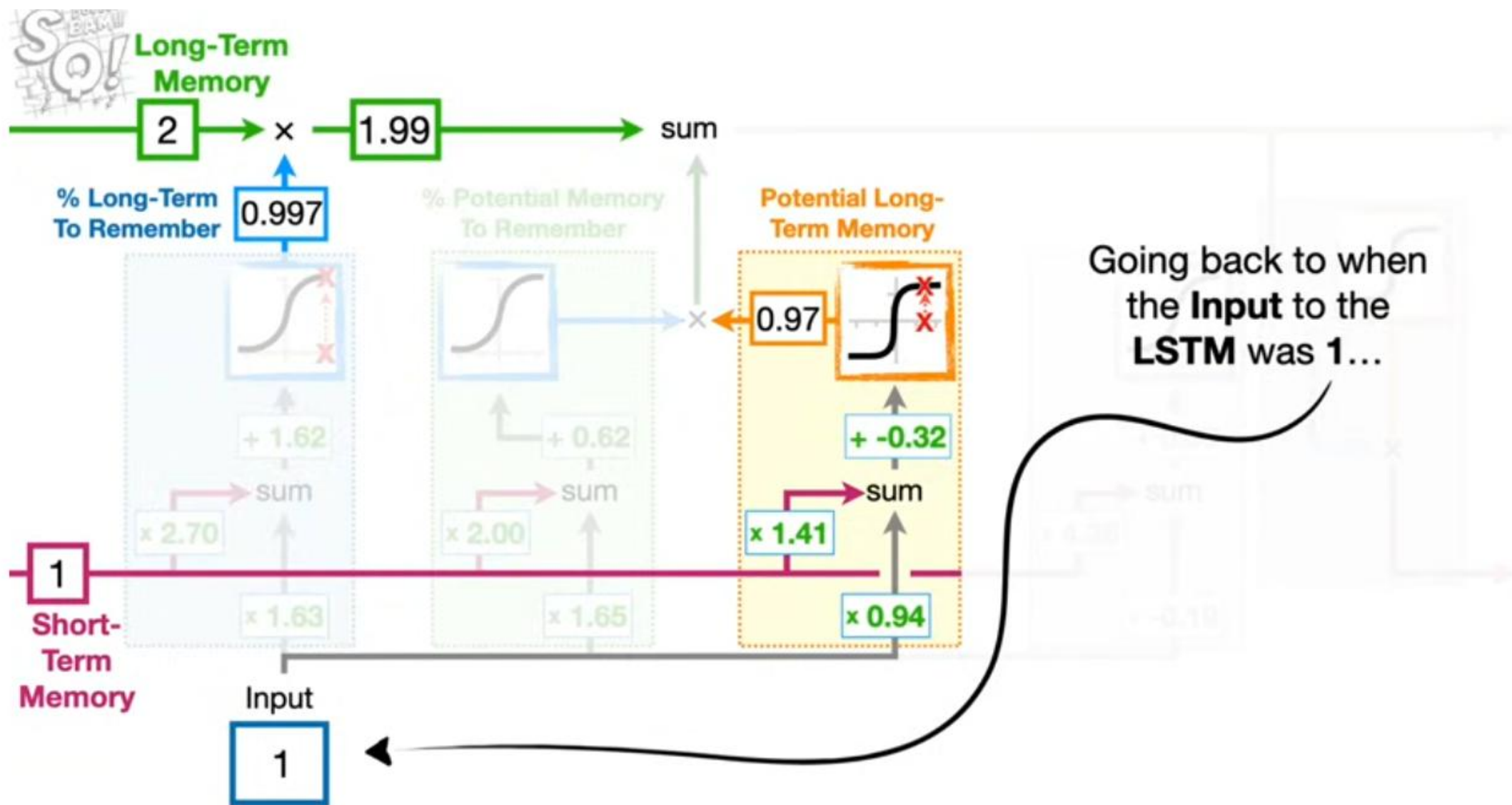


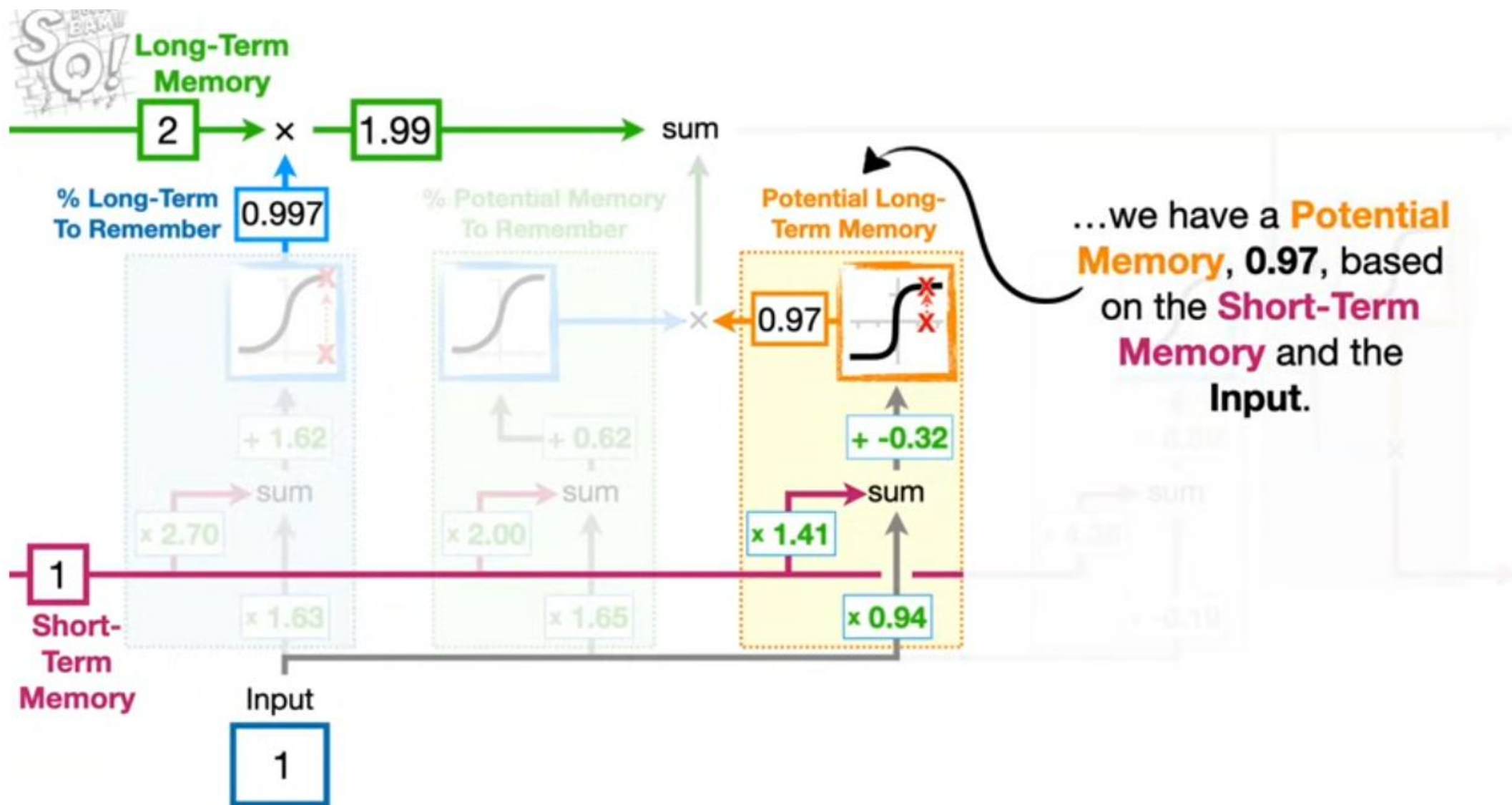


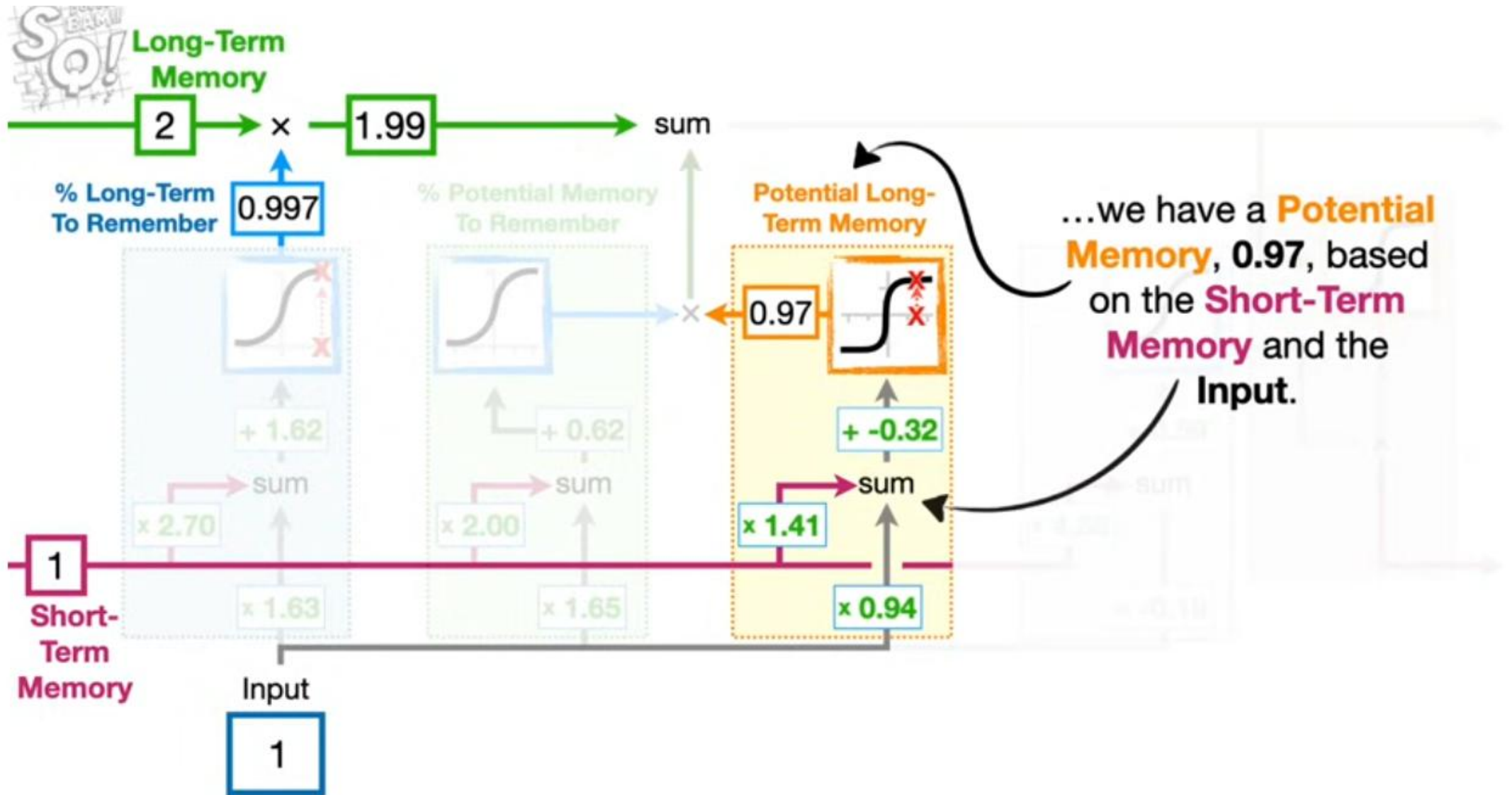


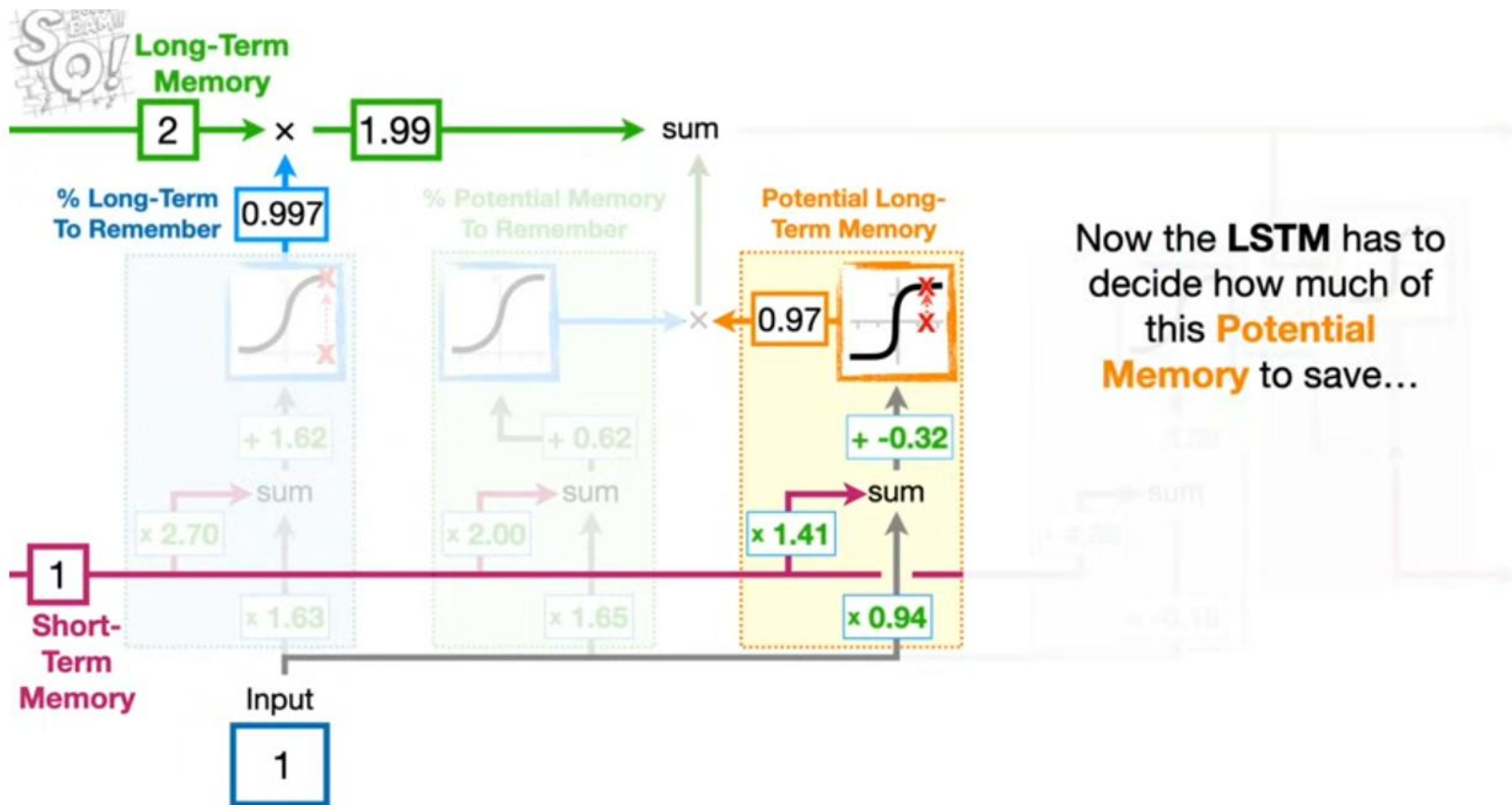
In contrast, if the **Input** to the **LSTM** was **-10...**

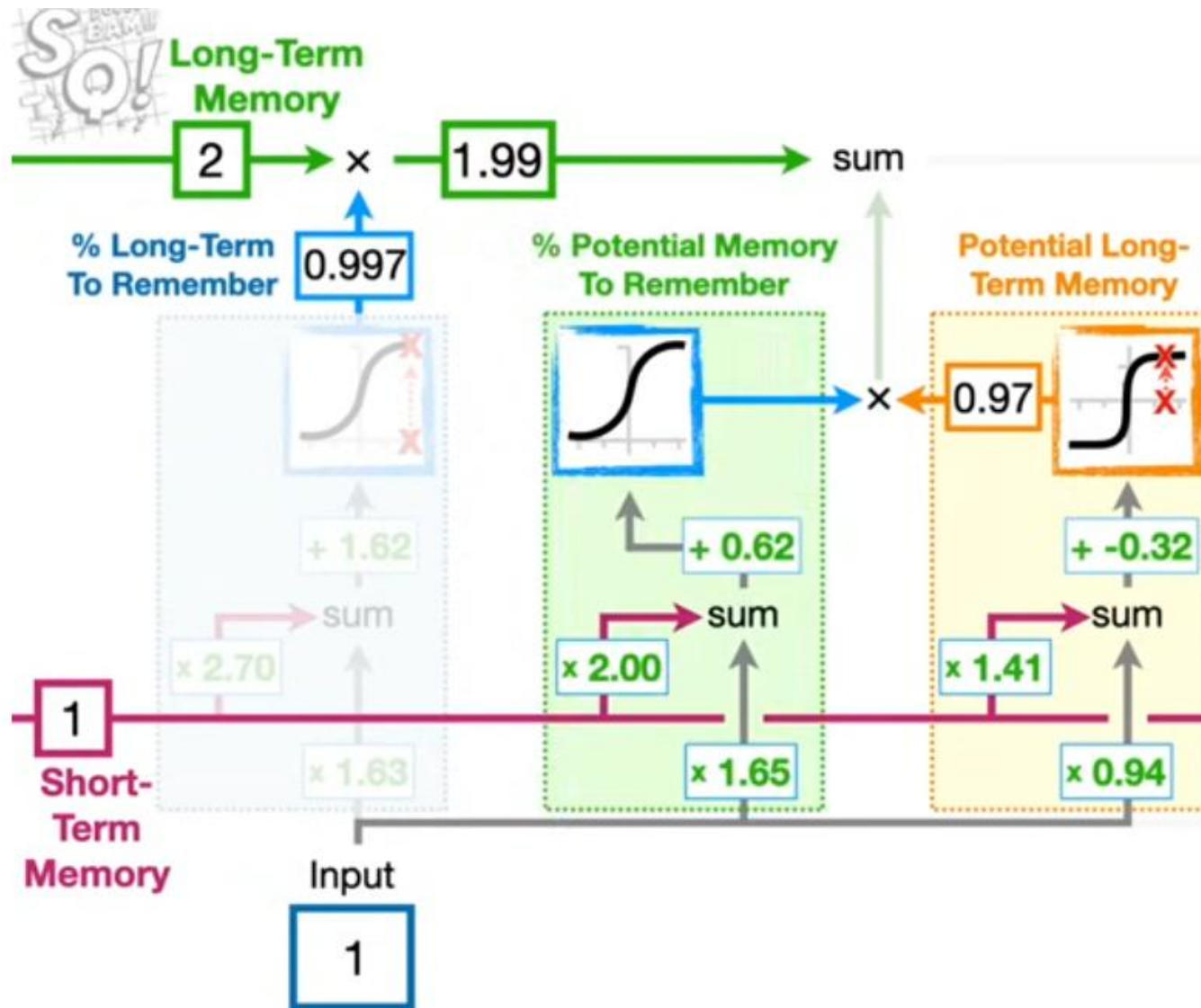
...then, after calculating the x-axis coordinate, the output from the **Tanh Activation Function** function would be **-1**.



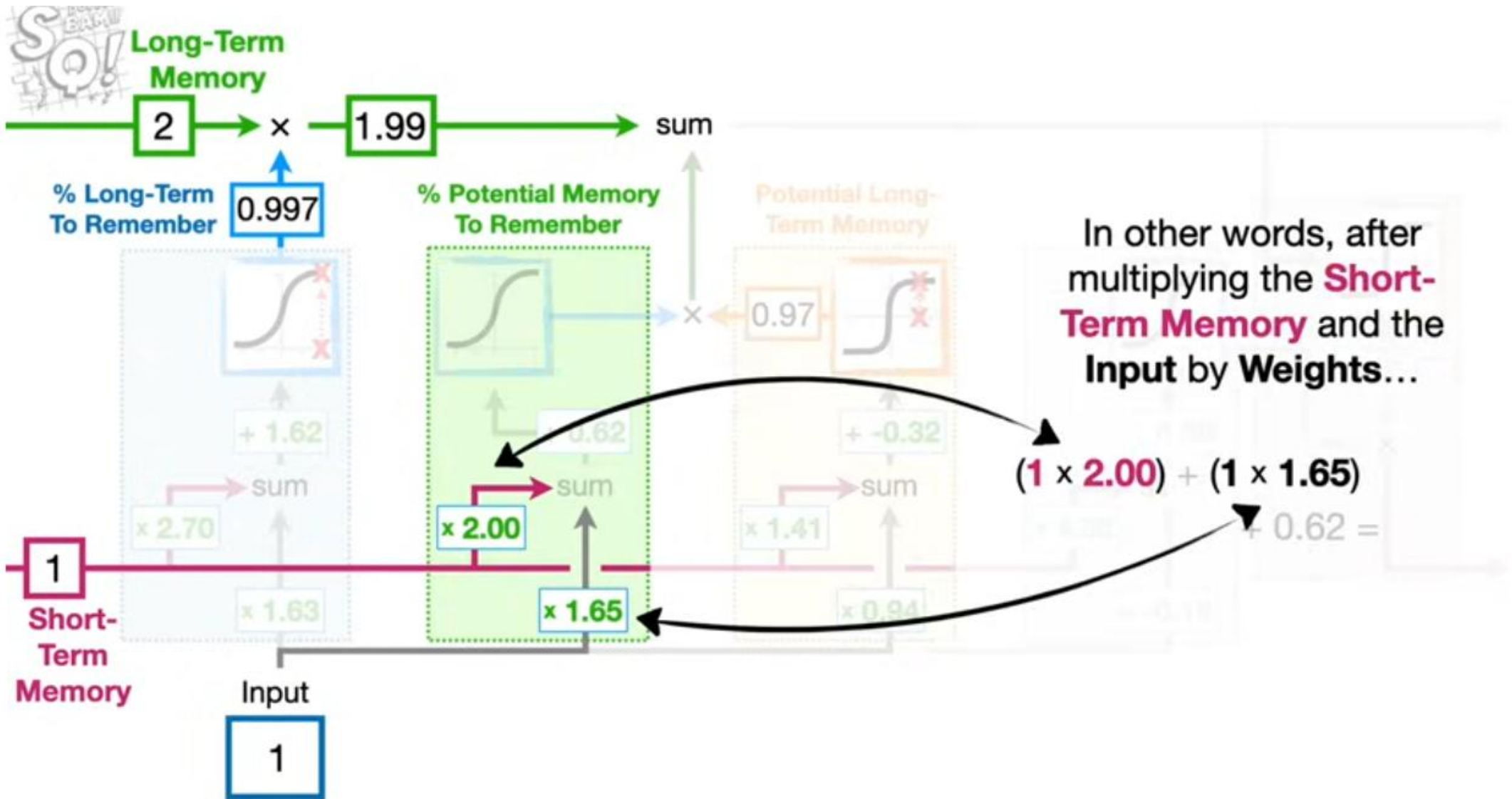


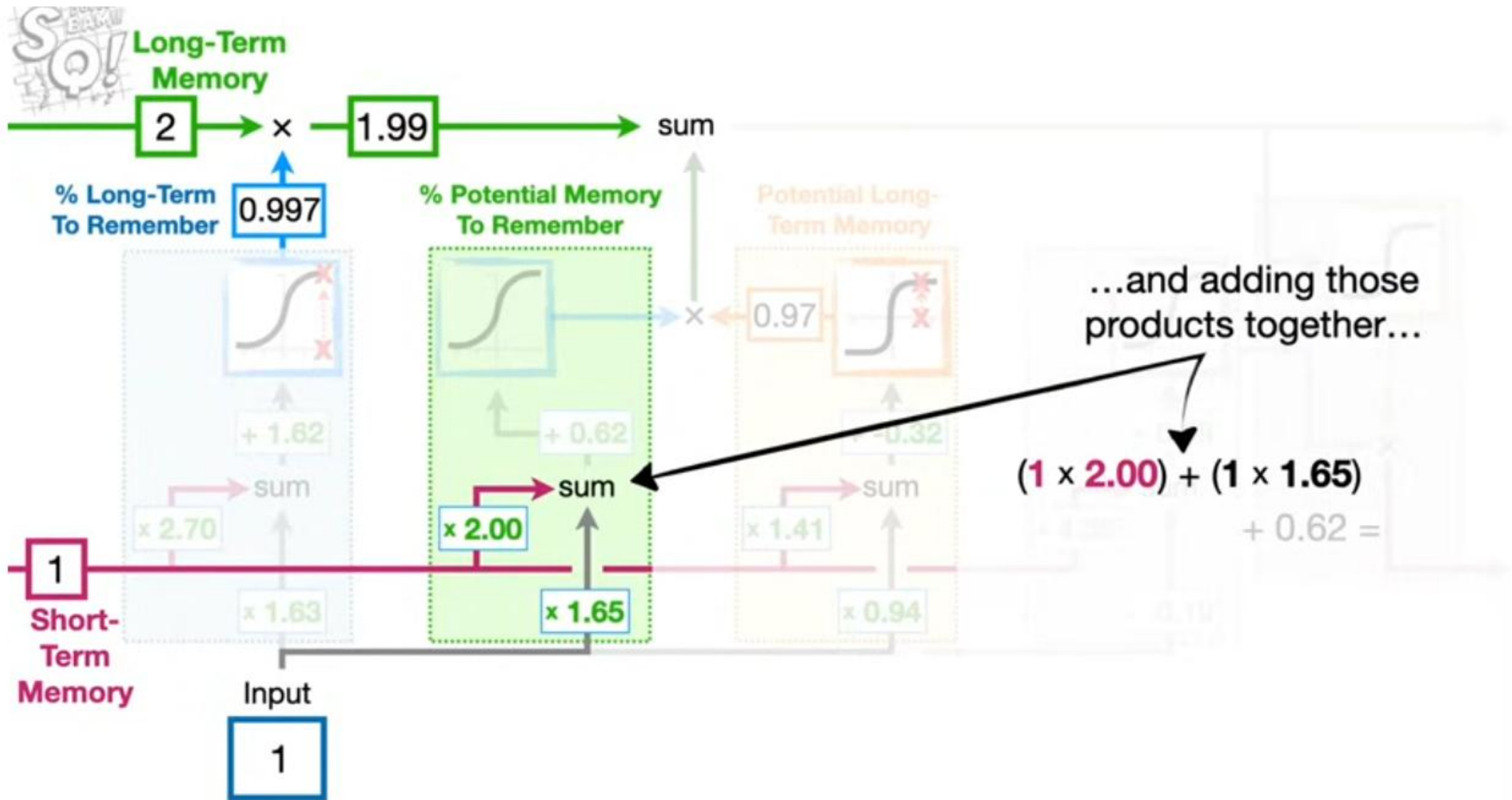


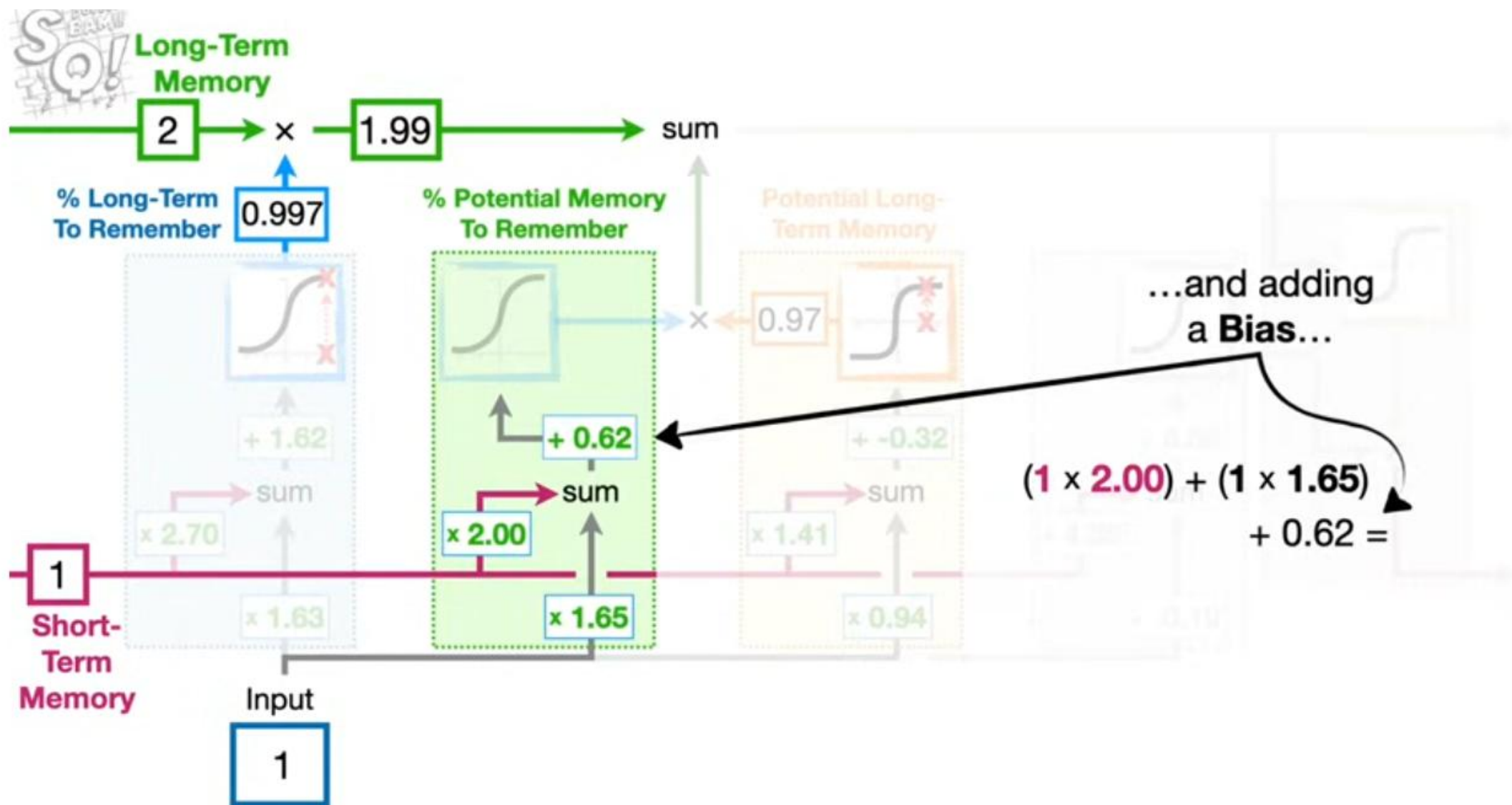


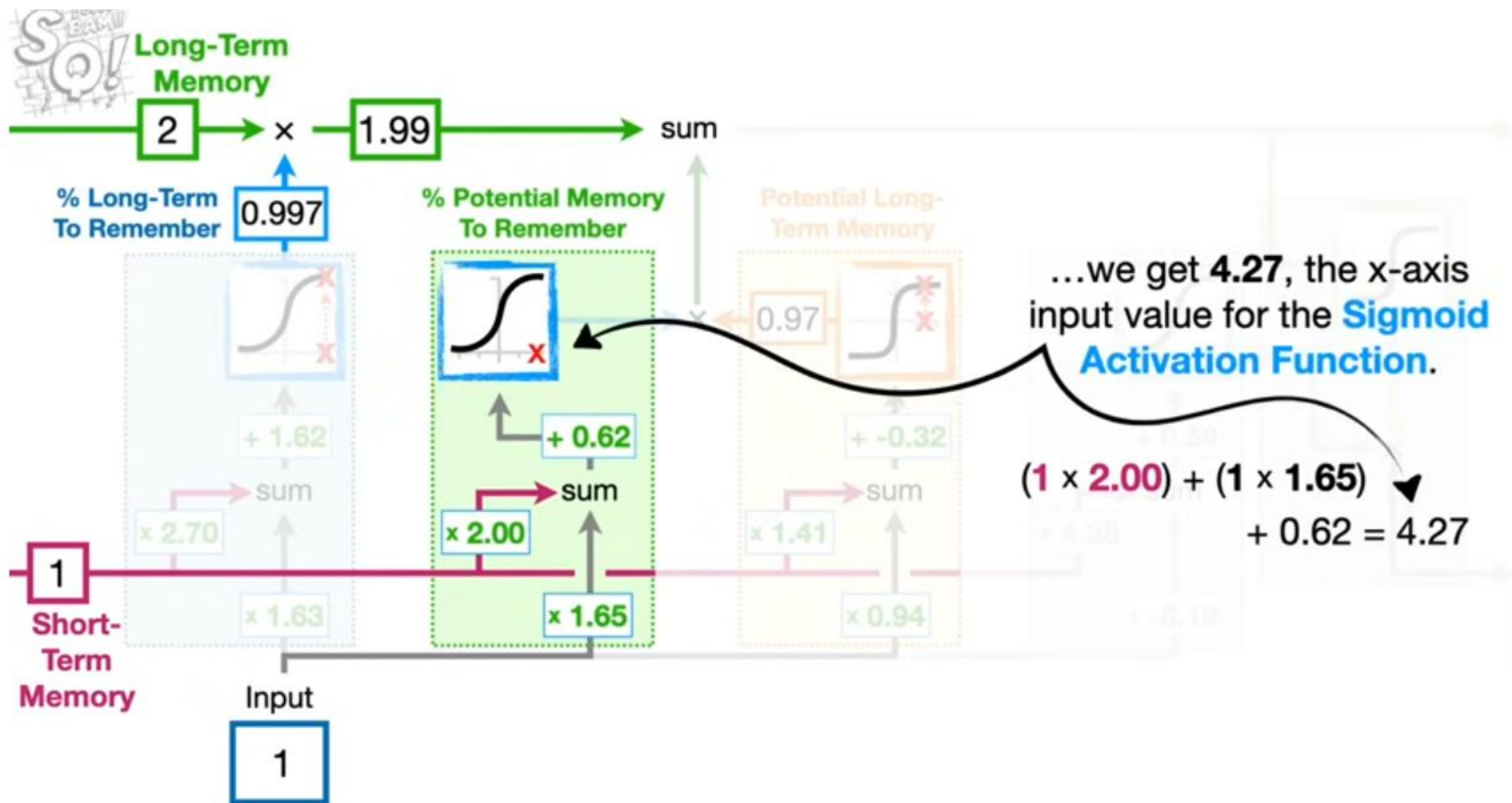


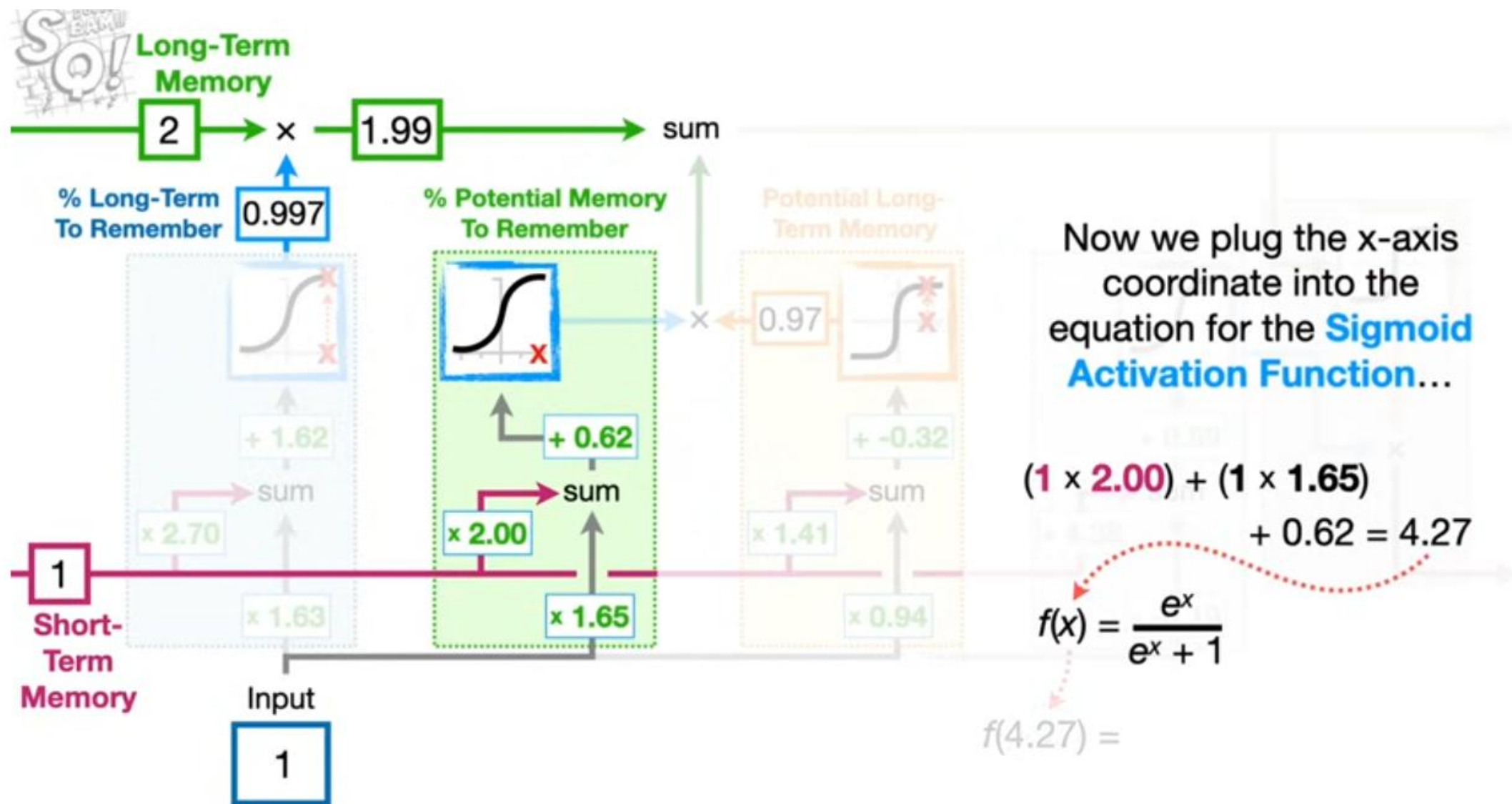
...and this is done using the exact same method we used earlier, when we determined what percentage of the **Long-Term Memory** to remember.

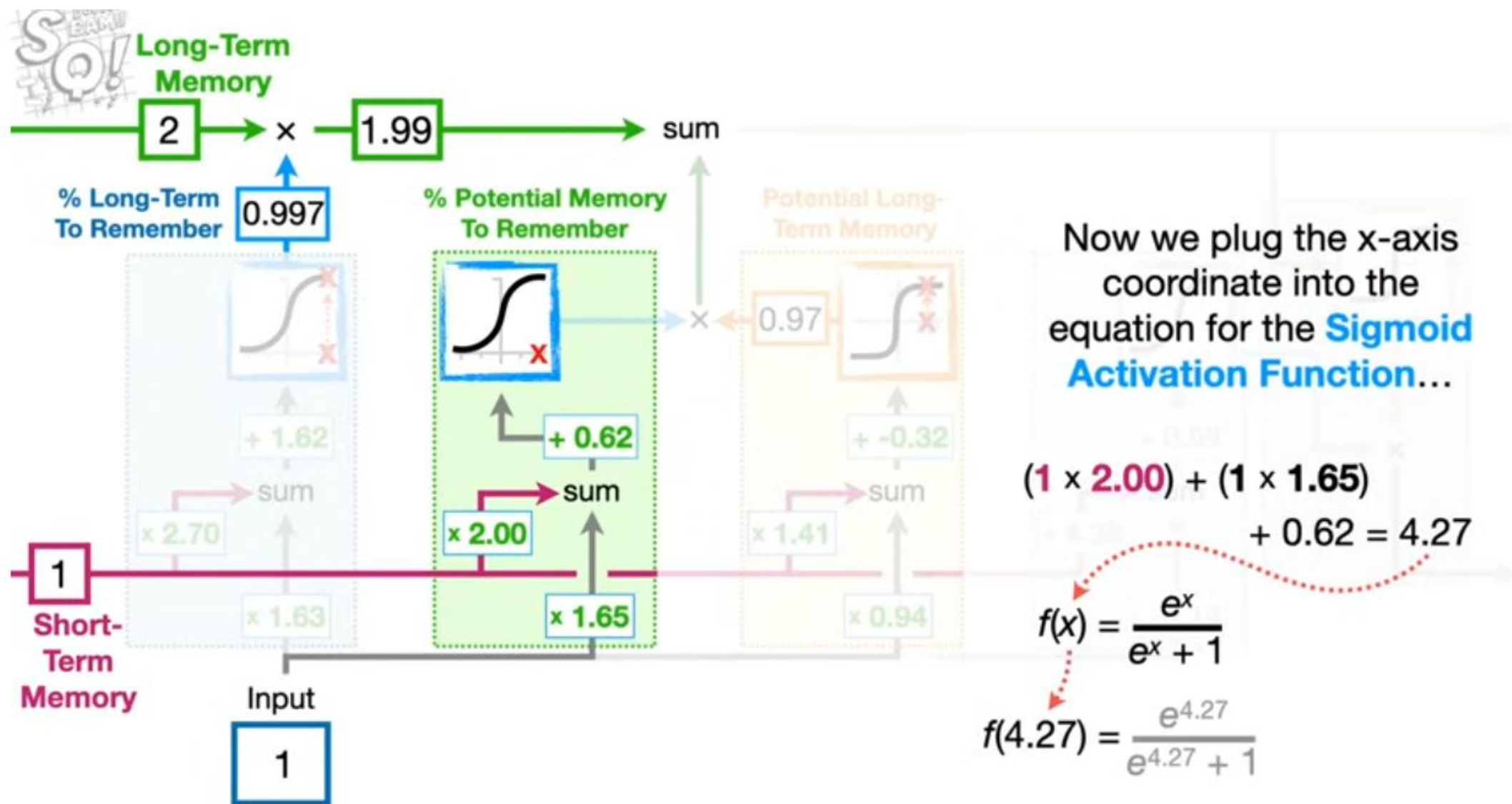


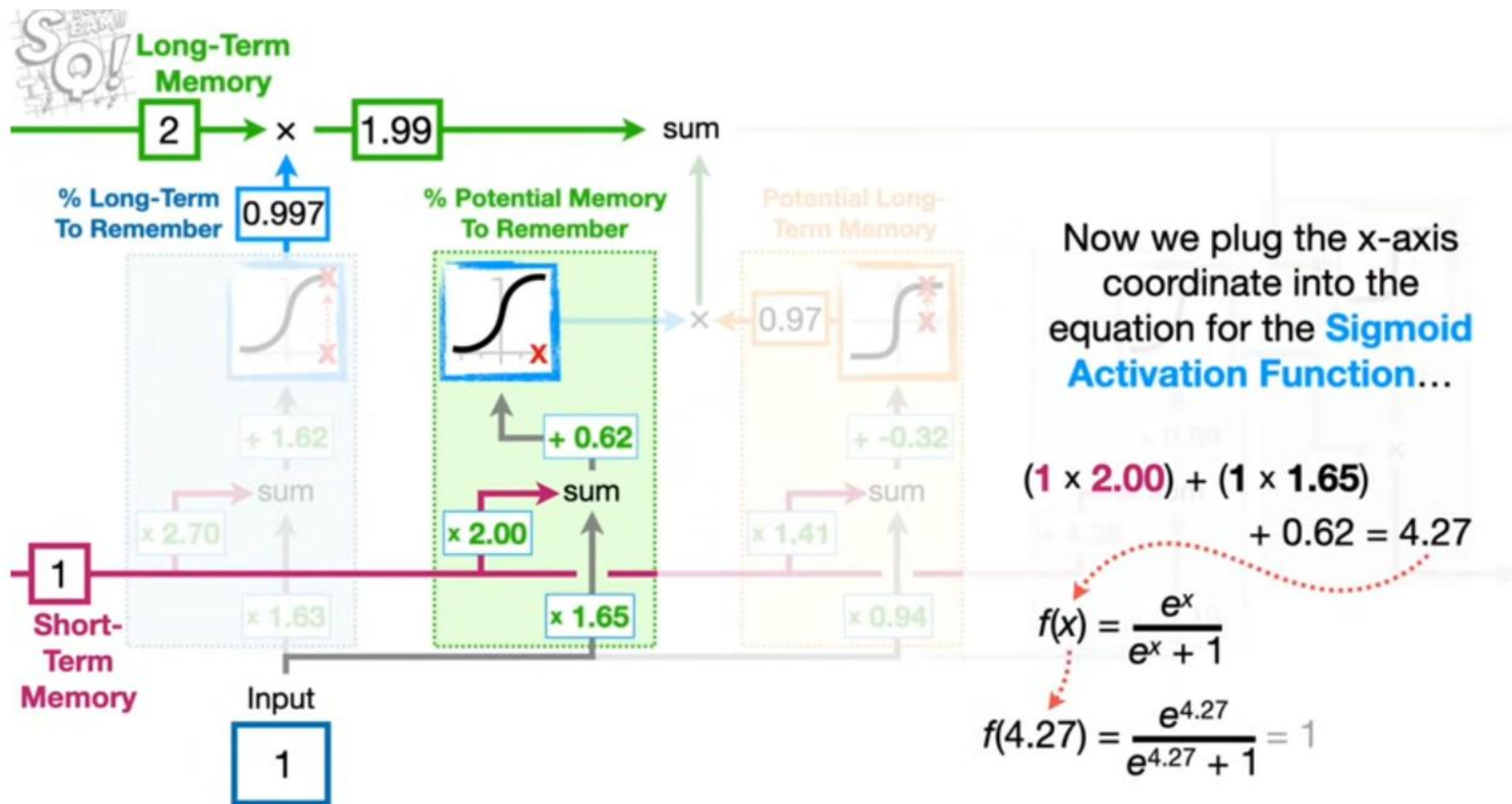










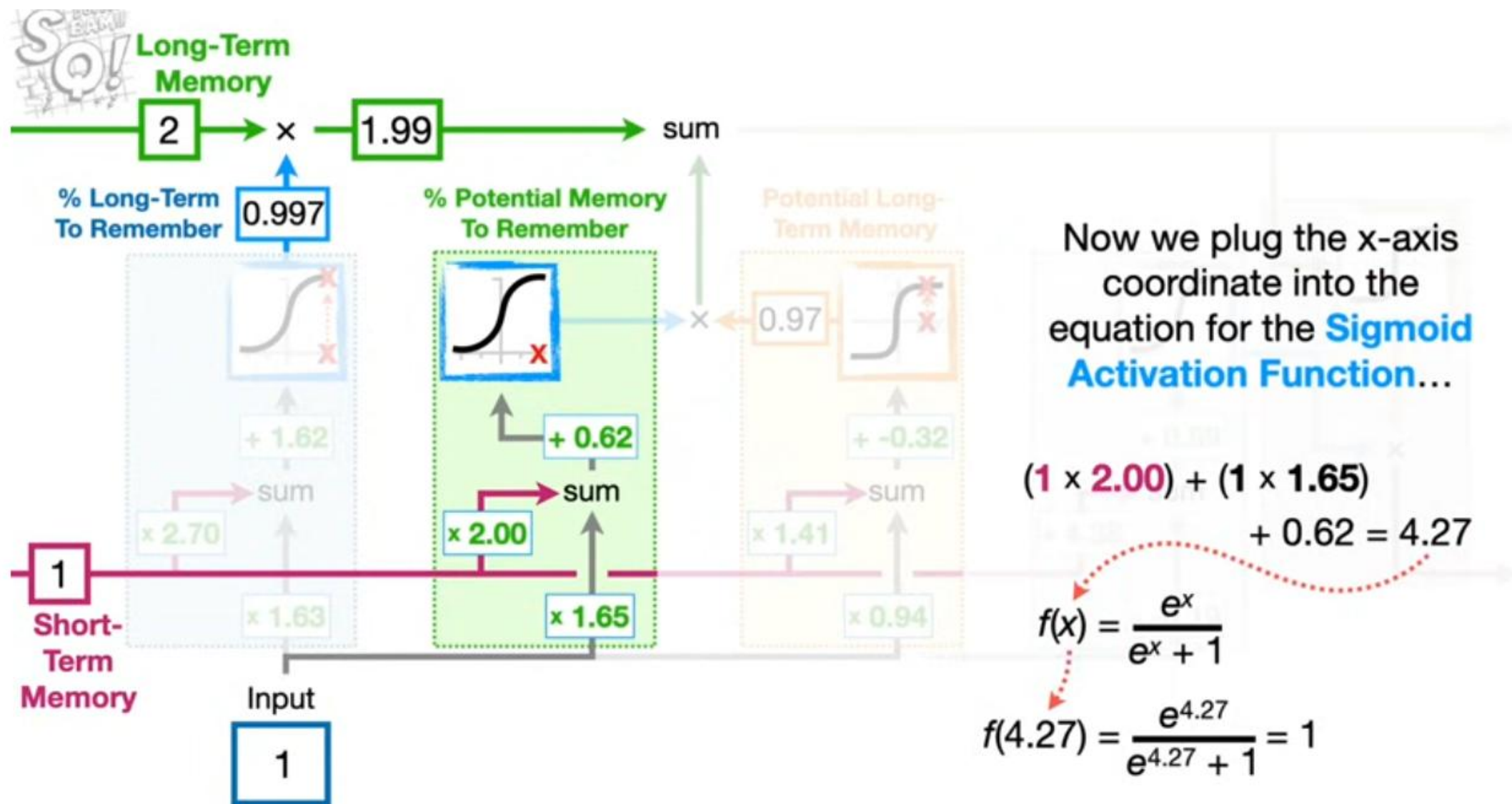


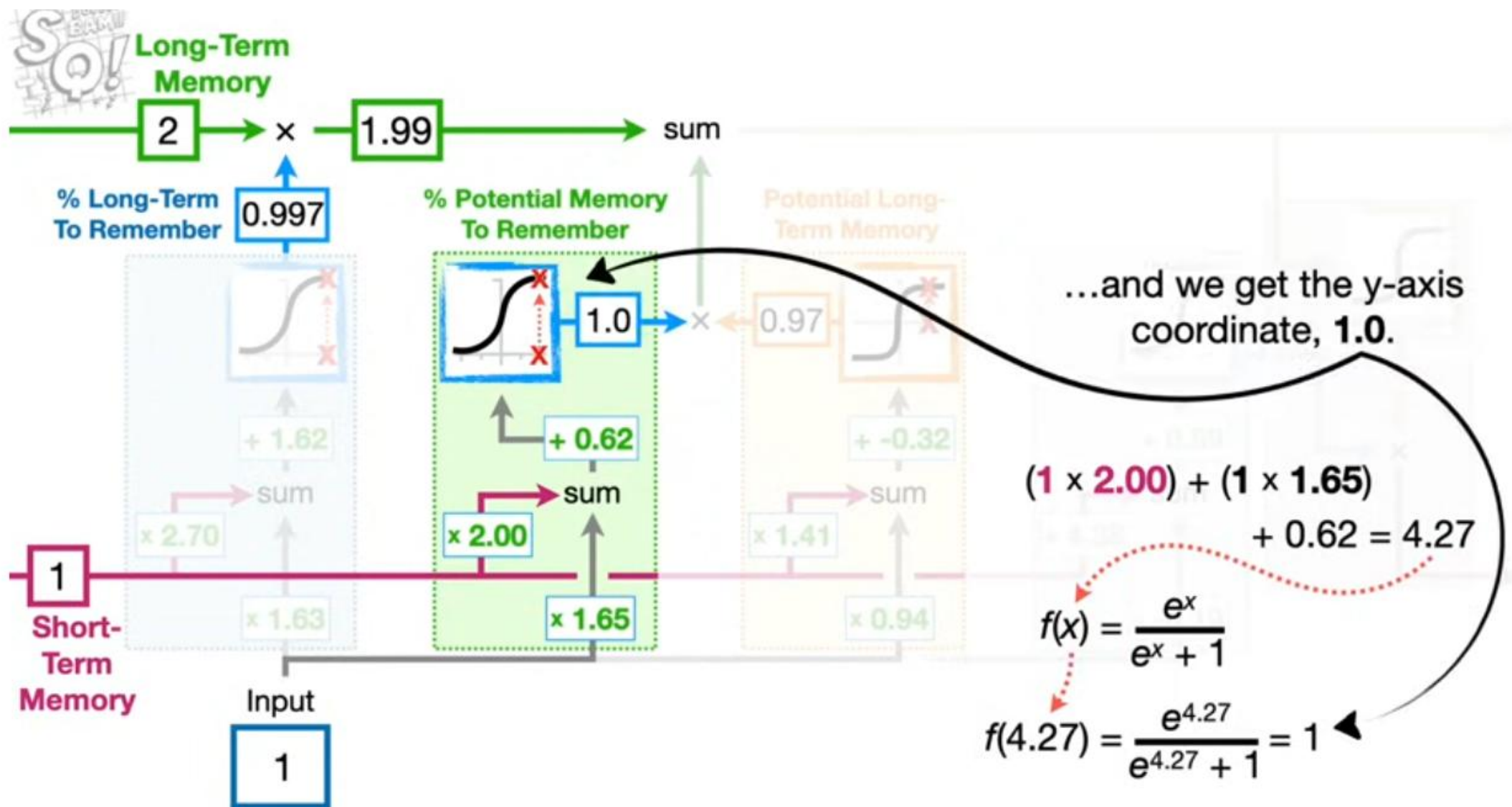
Now we plug the x-axis coordinate into the equation for the **Sigmoid Activation Function**...

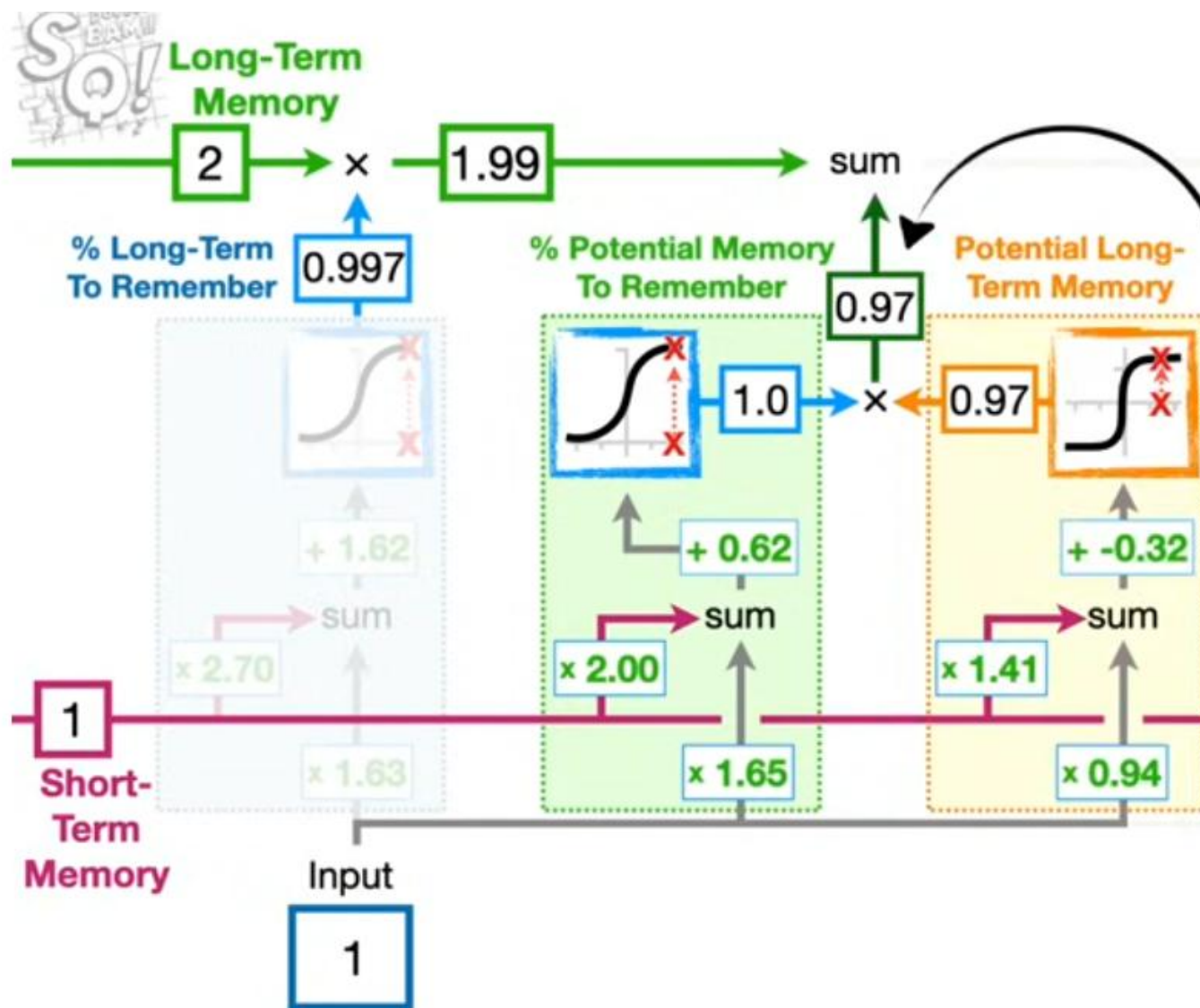
$$(1 \times 2.00) + (1 \times 1.65) + 0.62 = 4.27$$

$$f(x) = \frac{e^x}{e^x + 1}$$

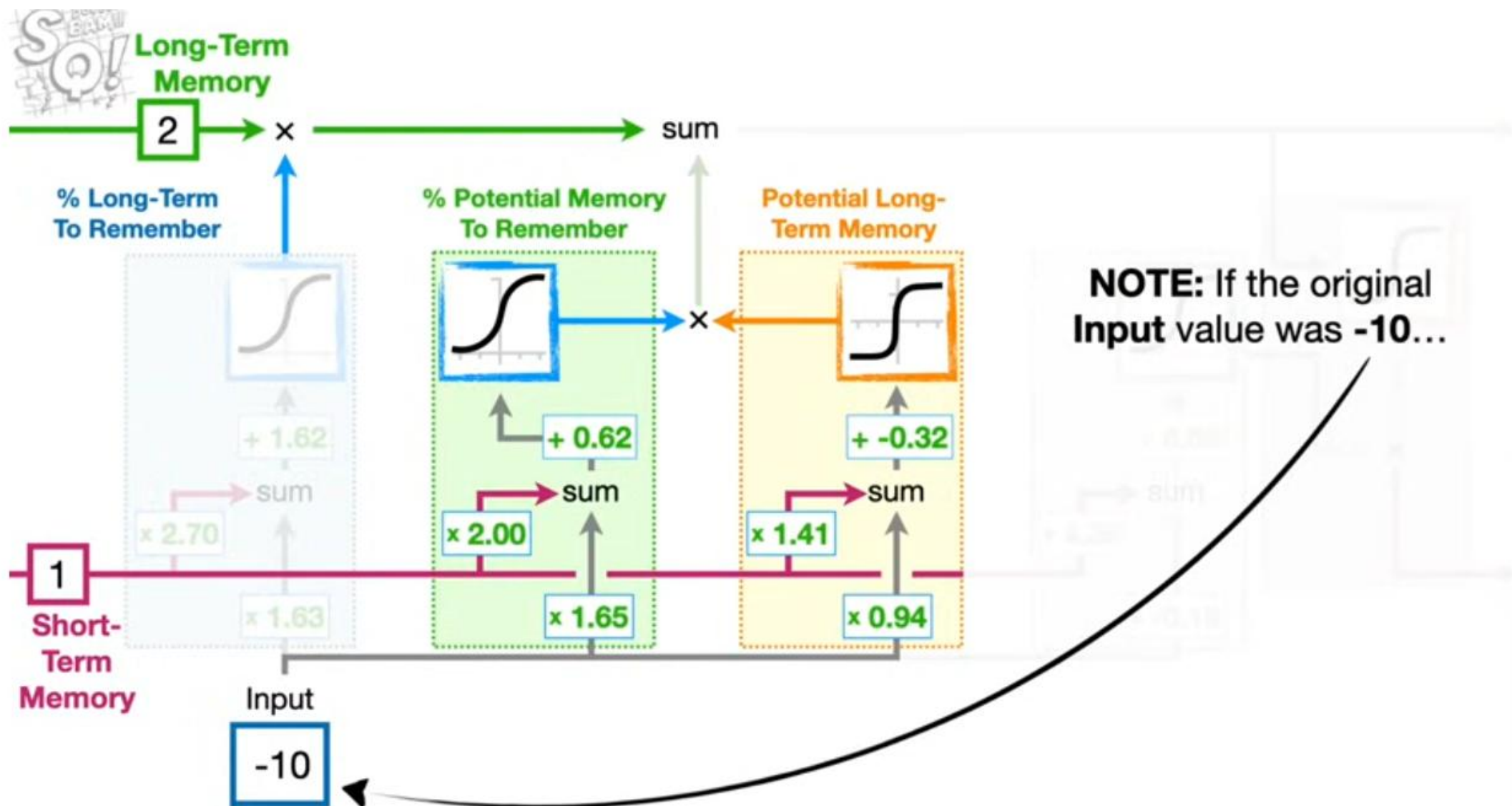
$$f(4.27) = \frac{e^{4.27}}{e^{4.27} + 1} = 1$$

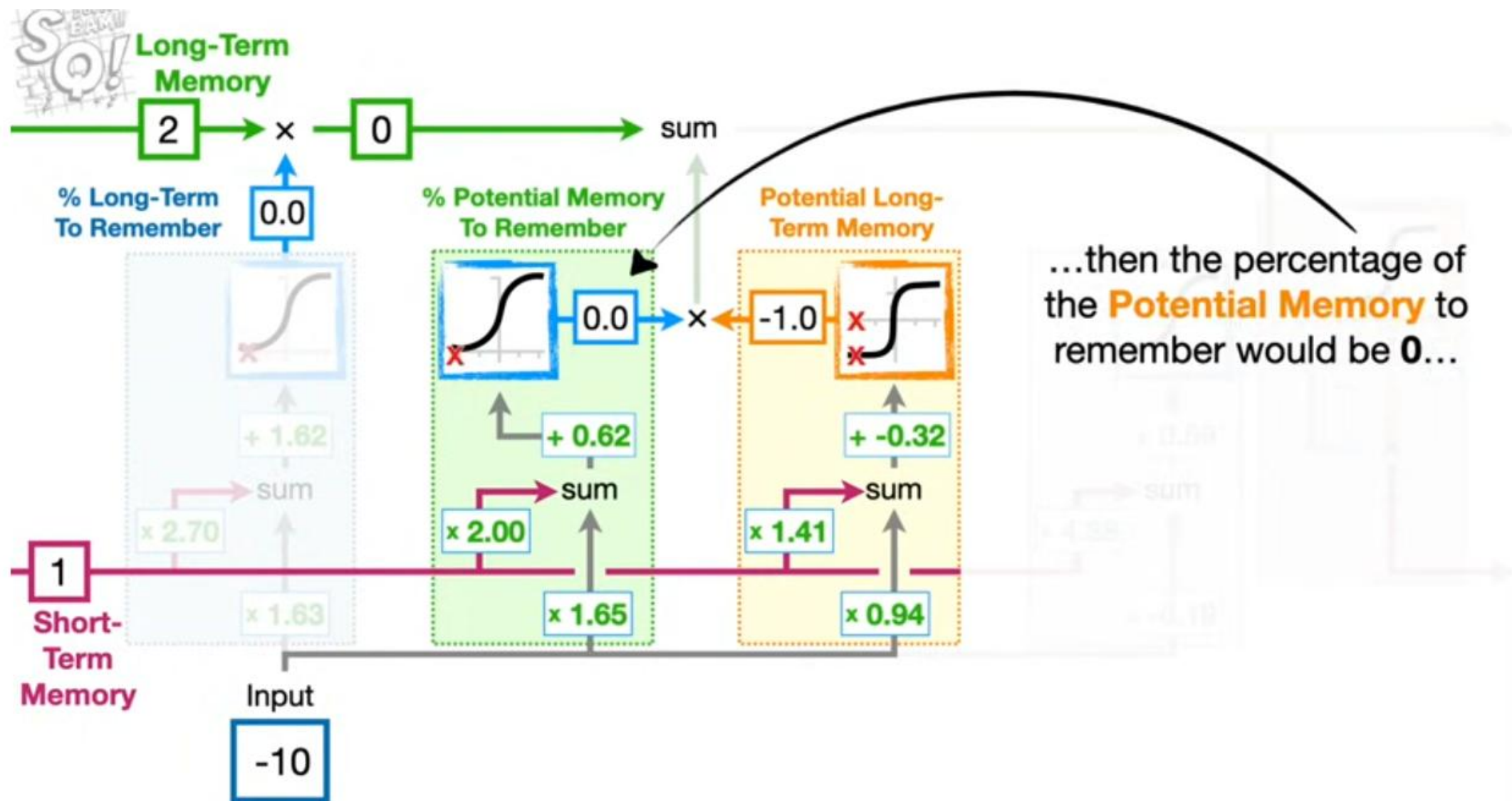


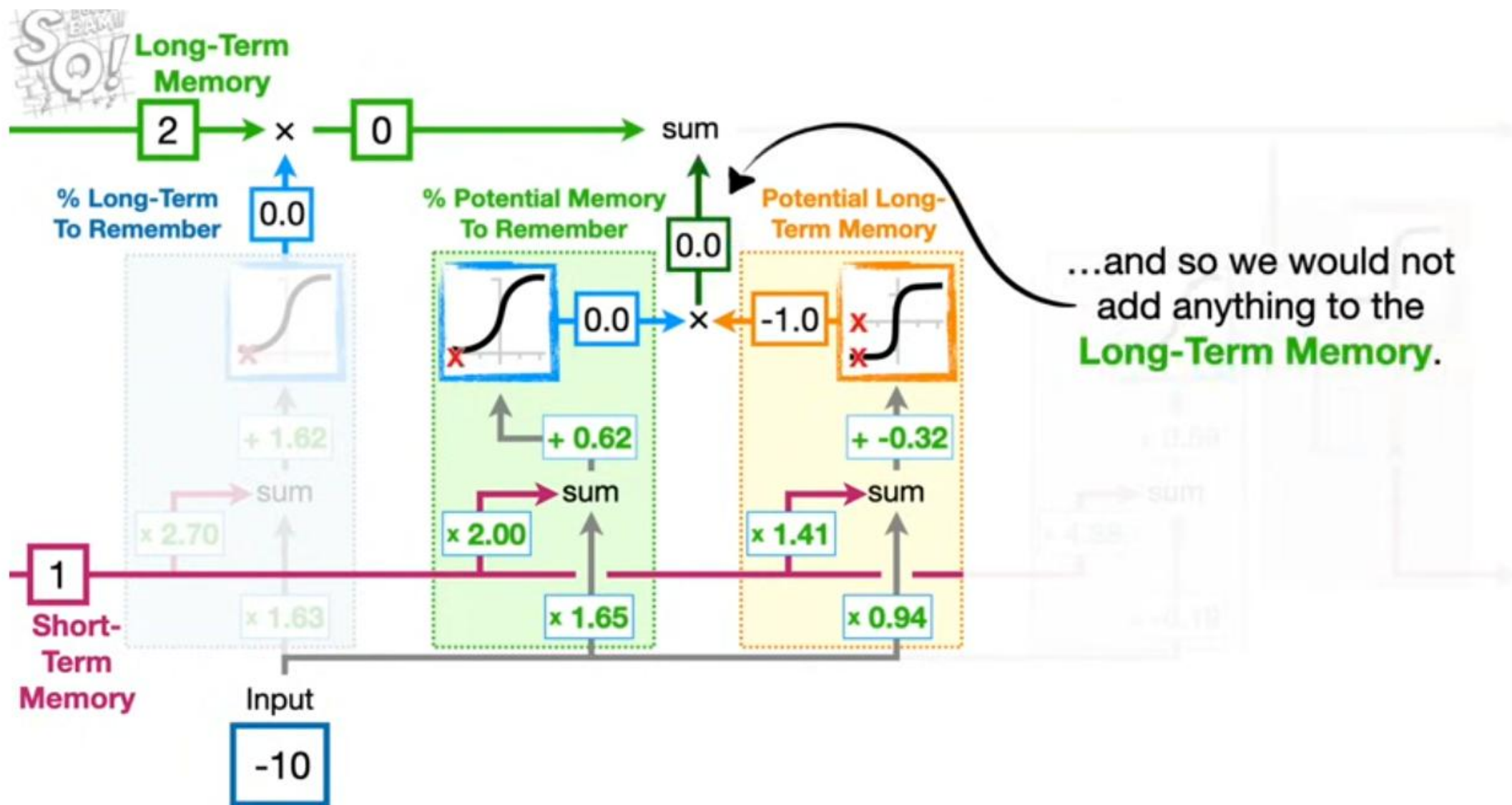


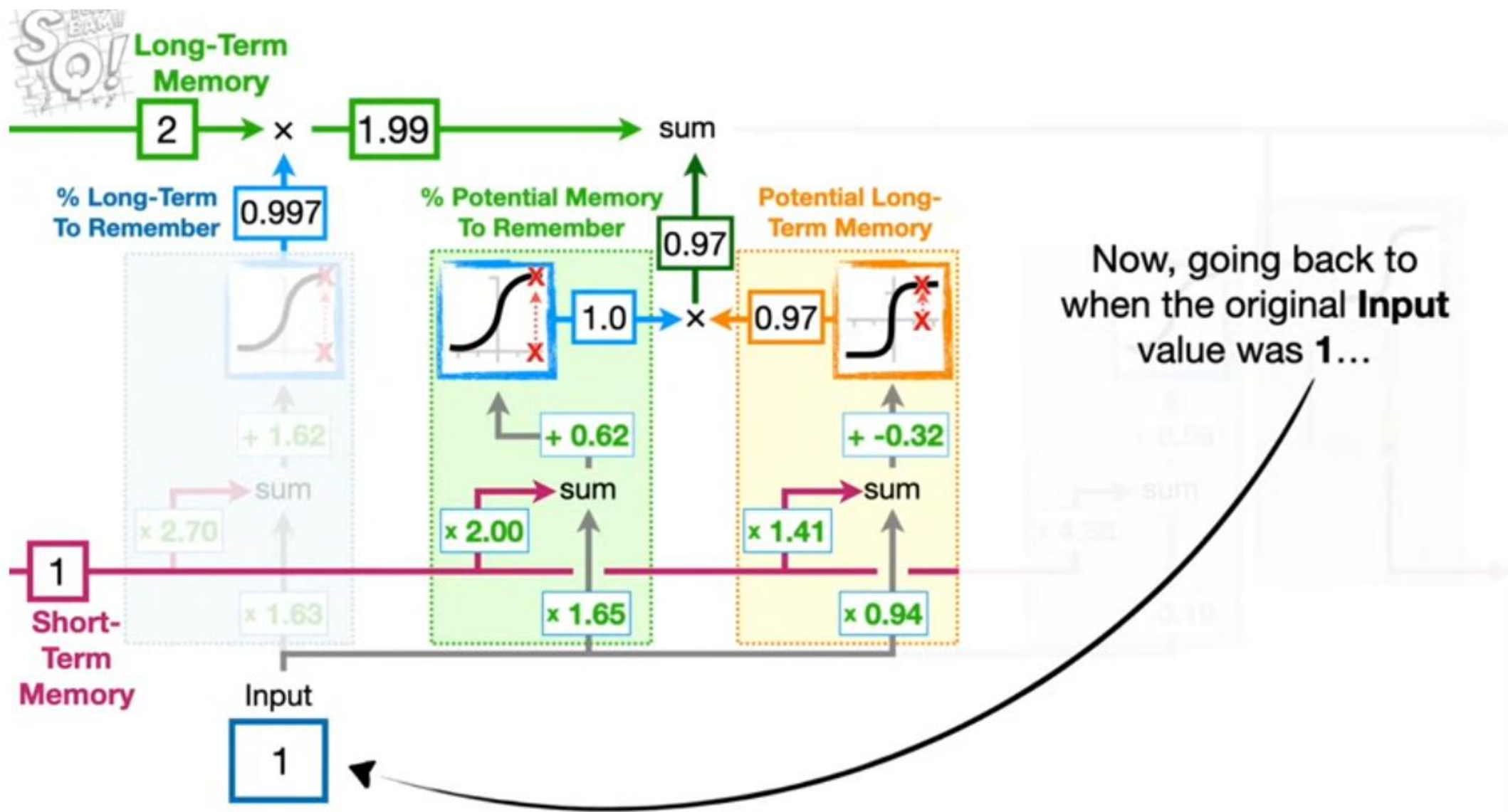


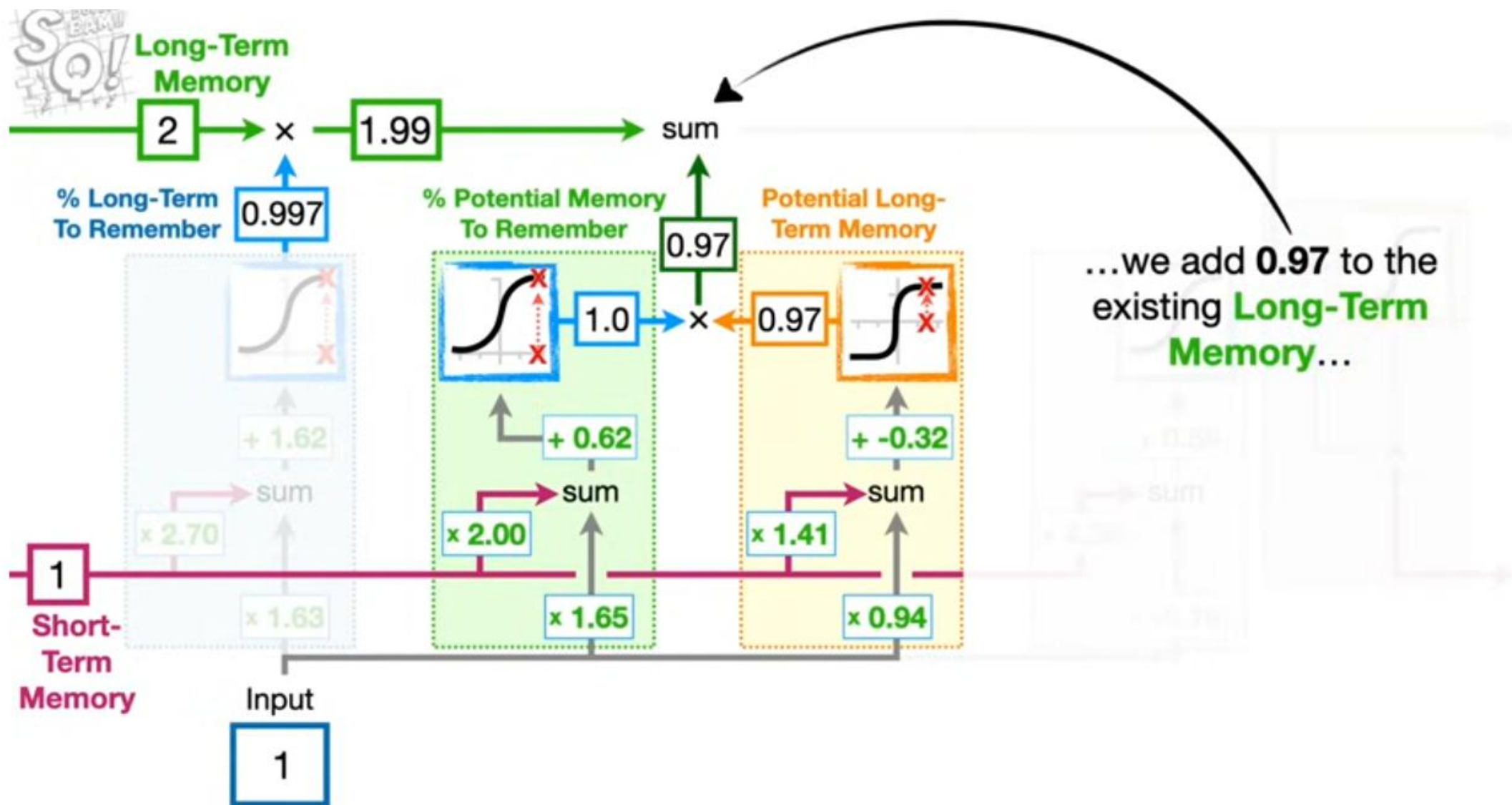
And that means the entire **Potential Long-Term Memory** is retained, because multiplying it by 1 doesn't change it.

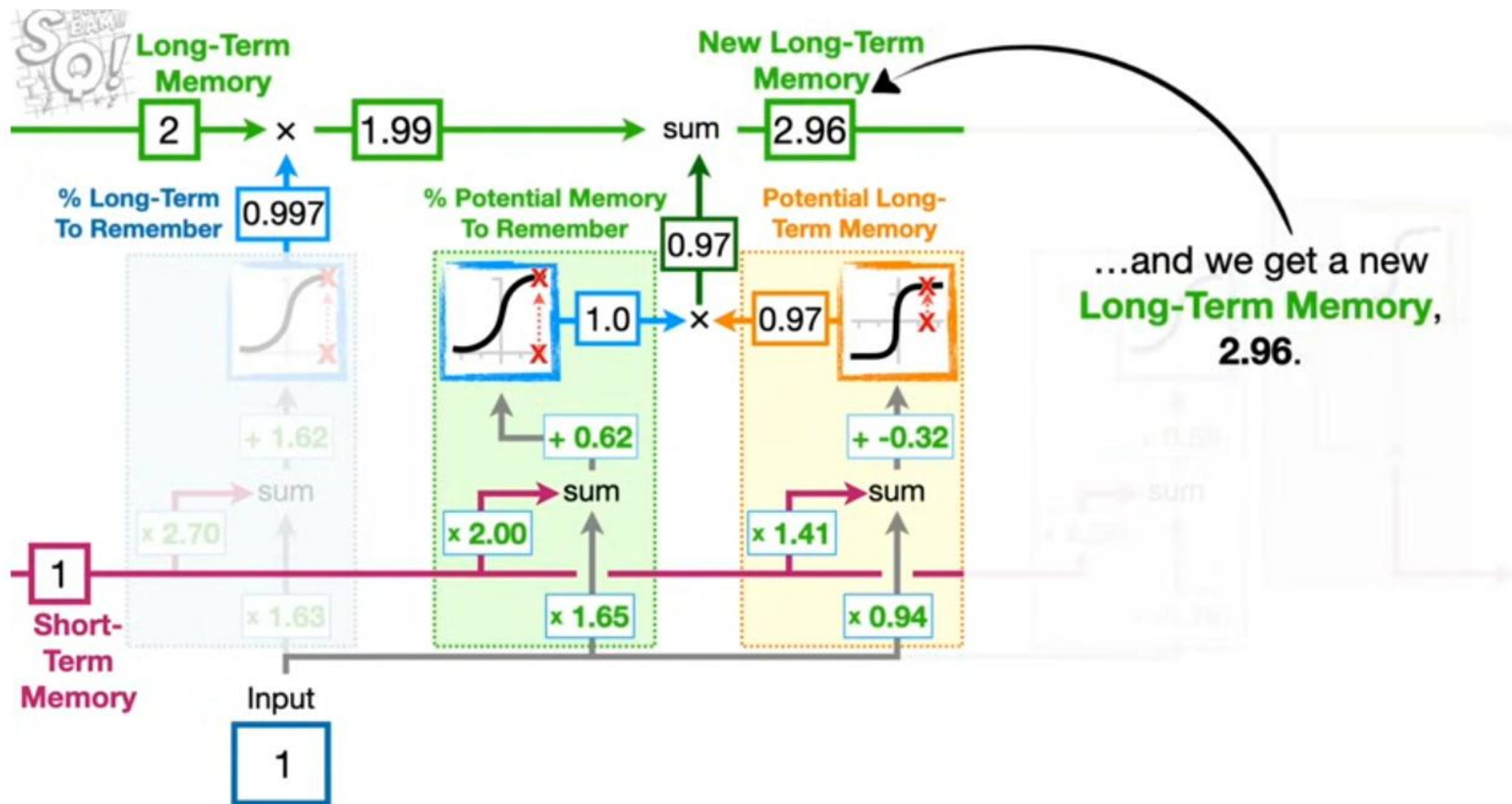




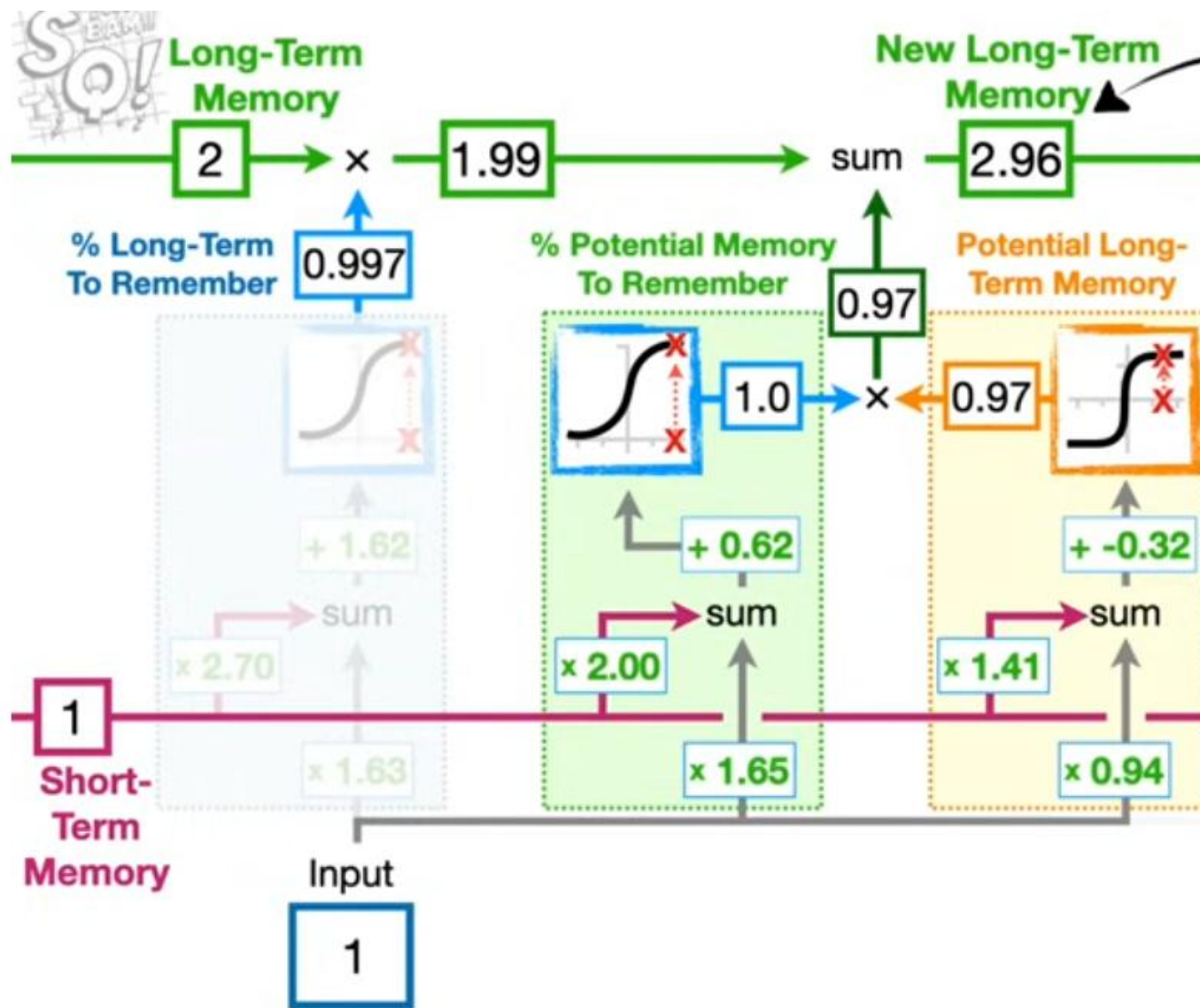




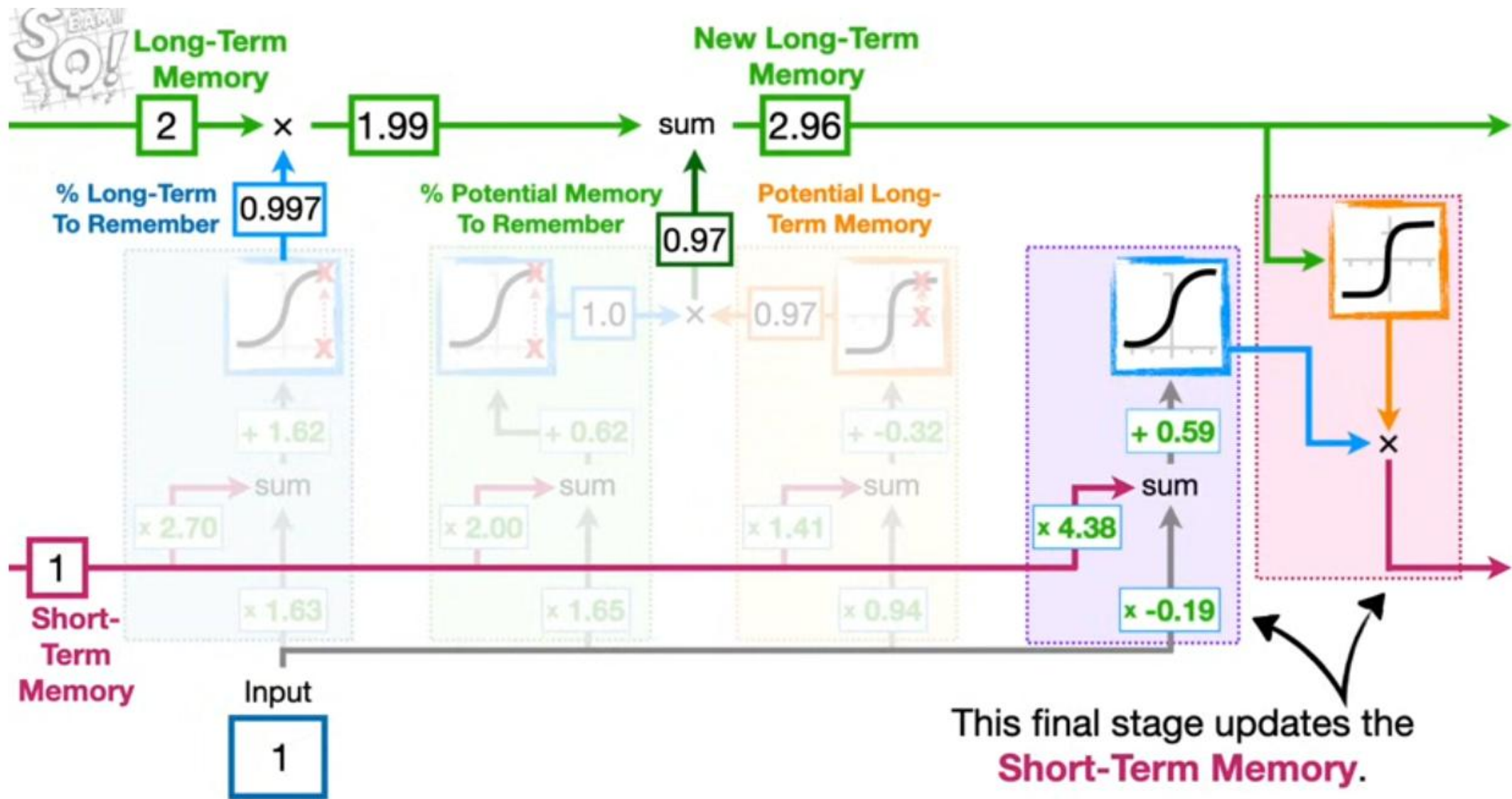


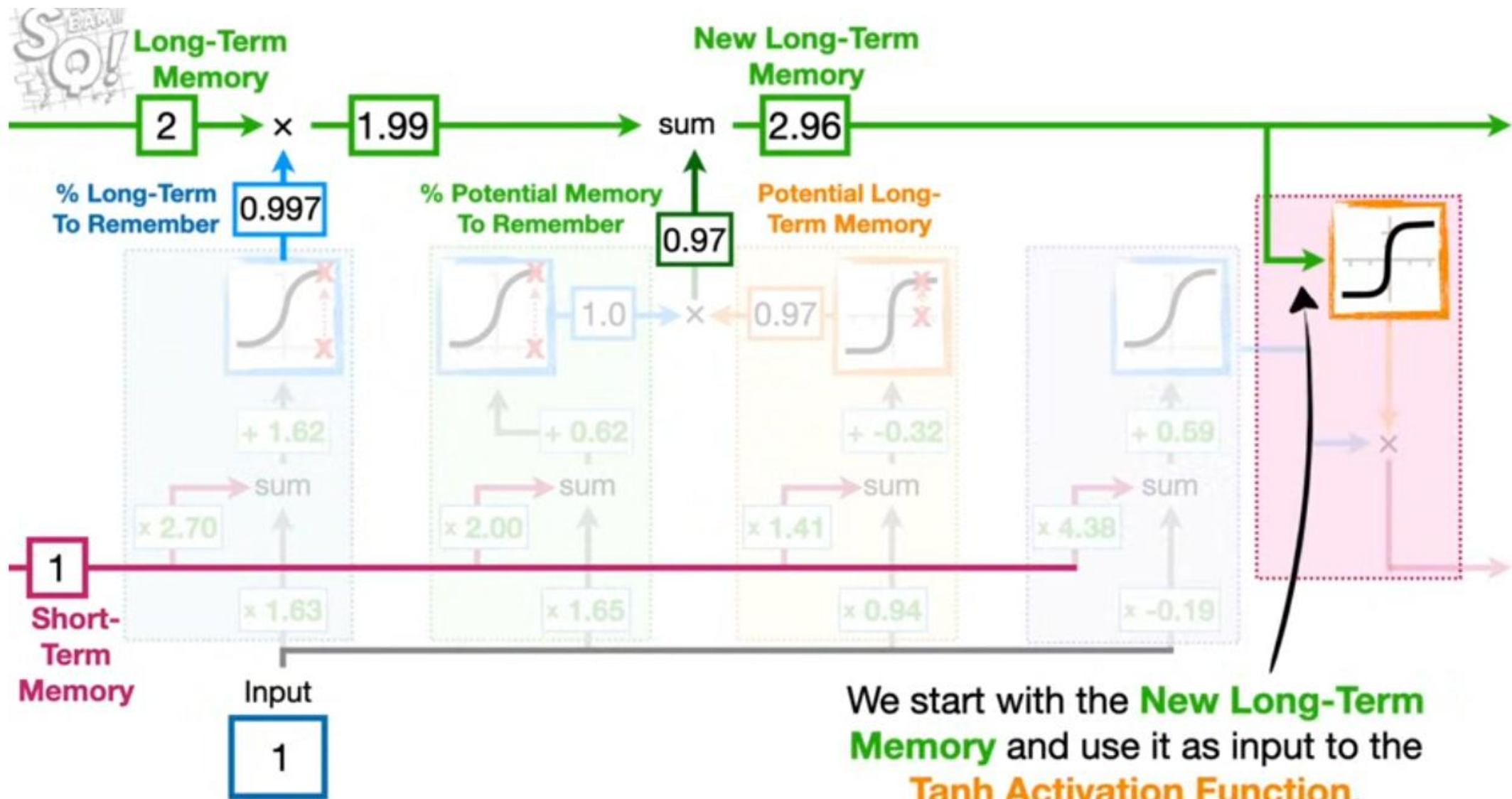


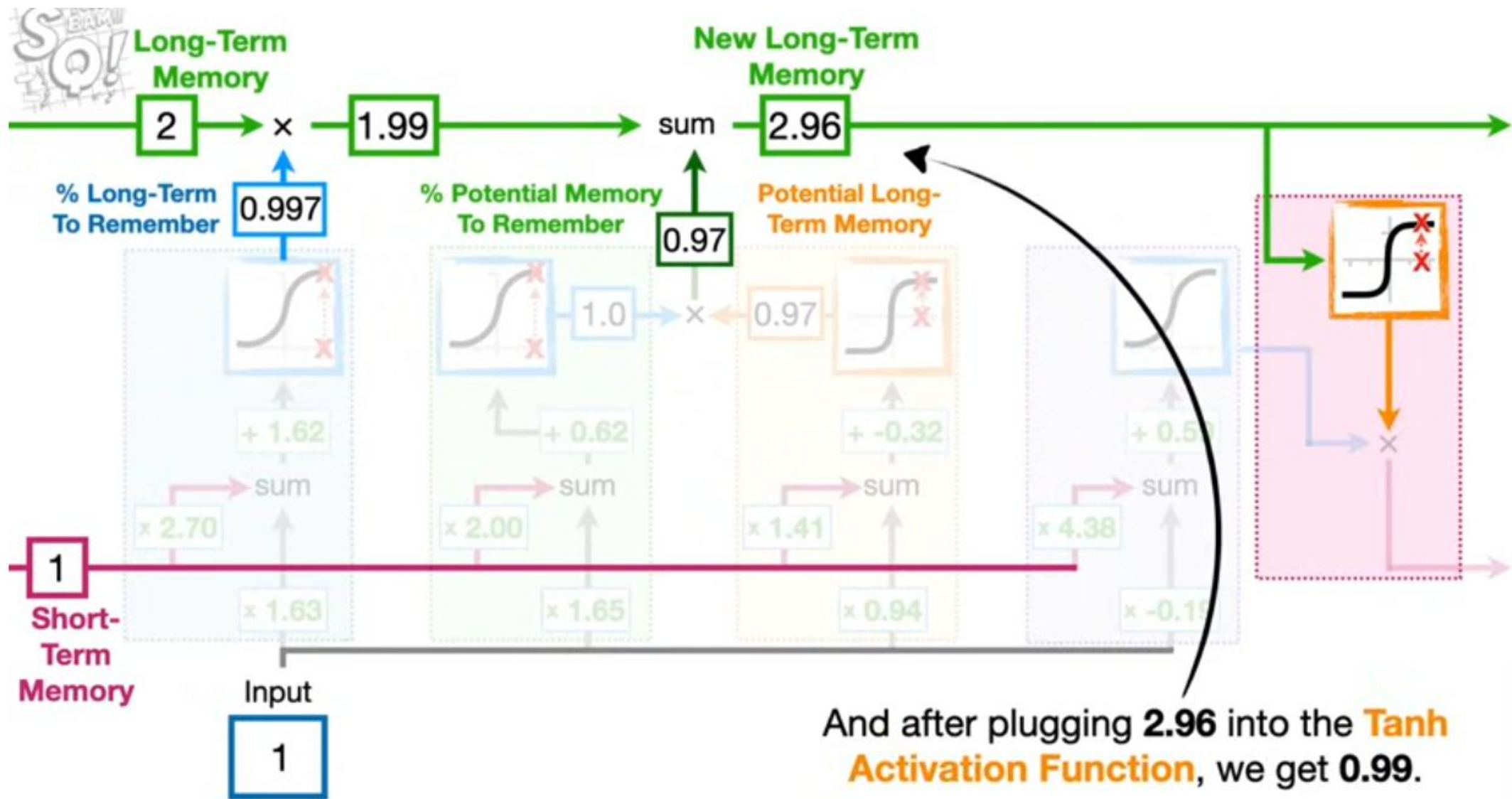
...it is usually called
the Input Gate.

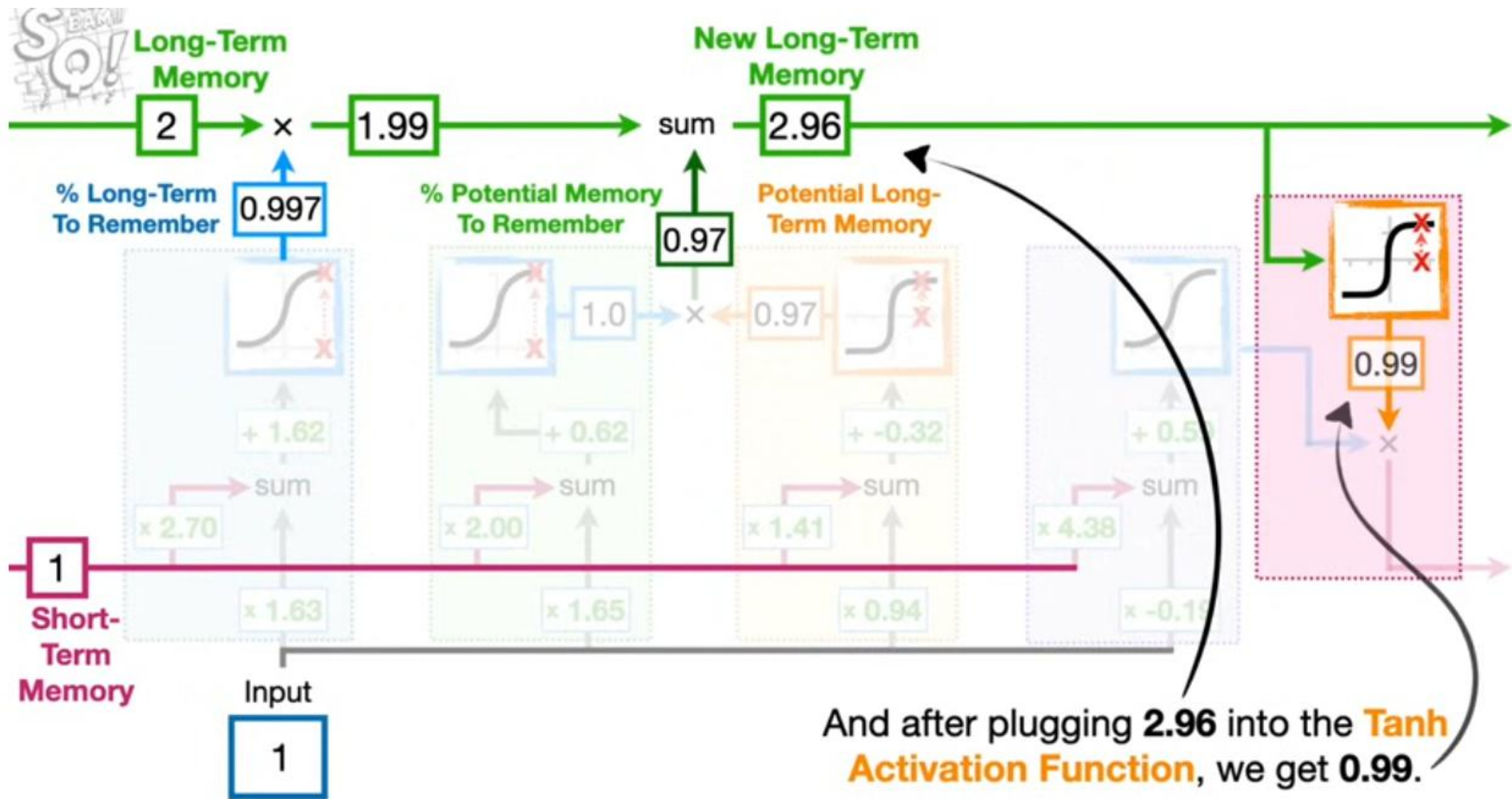


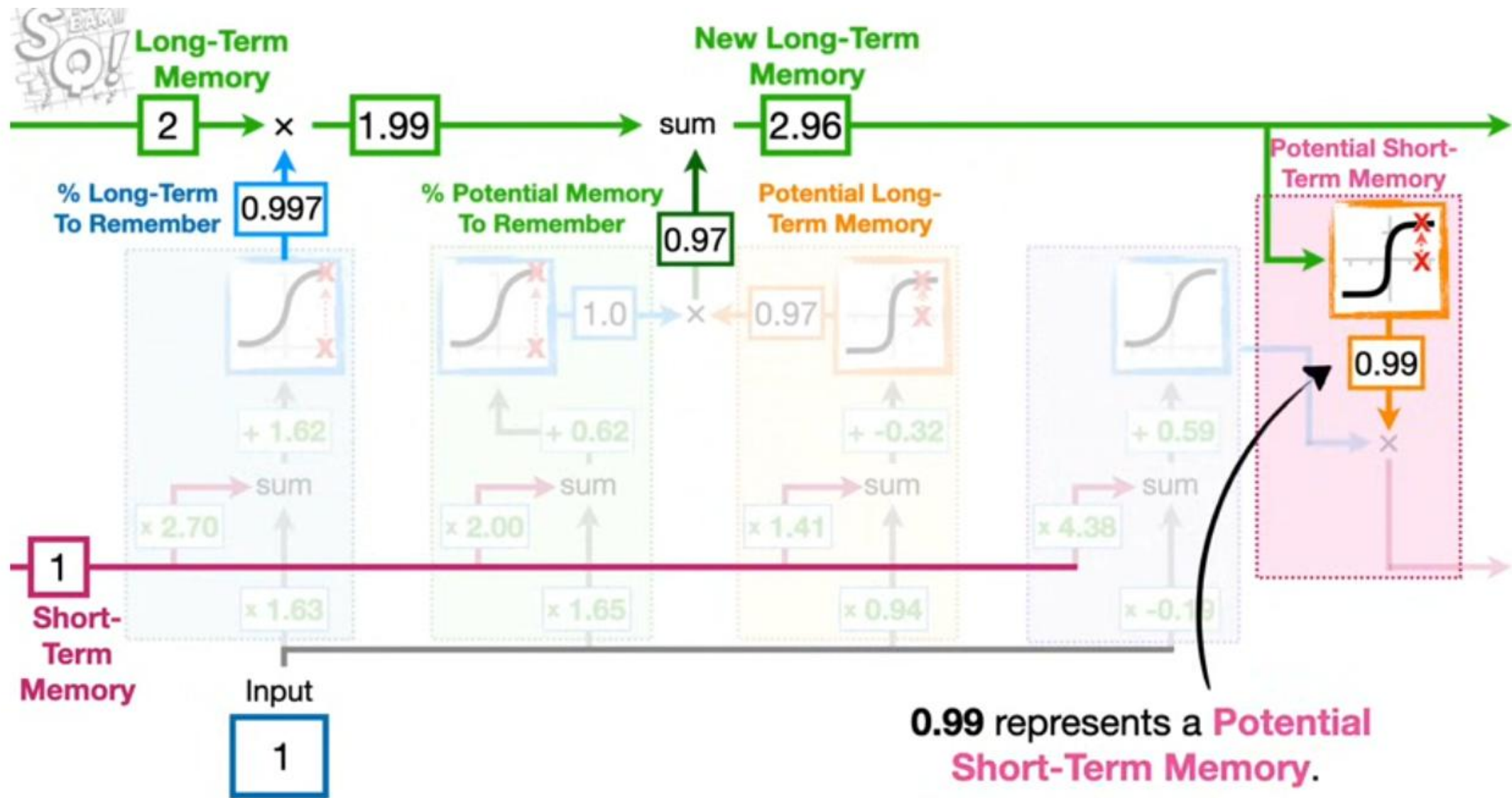
Now that we have a **New Long-Term Memory**...

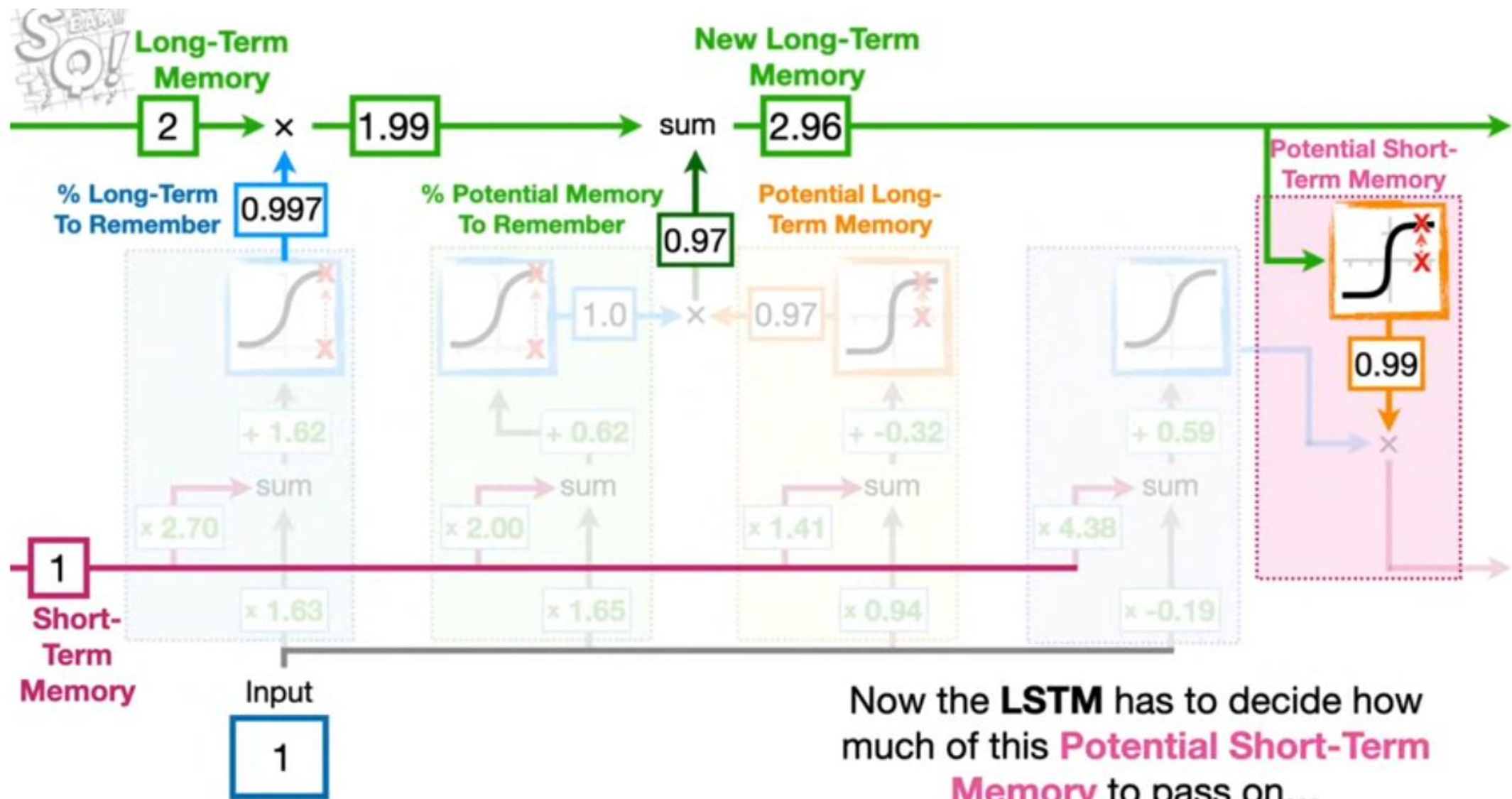


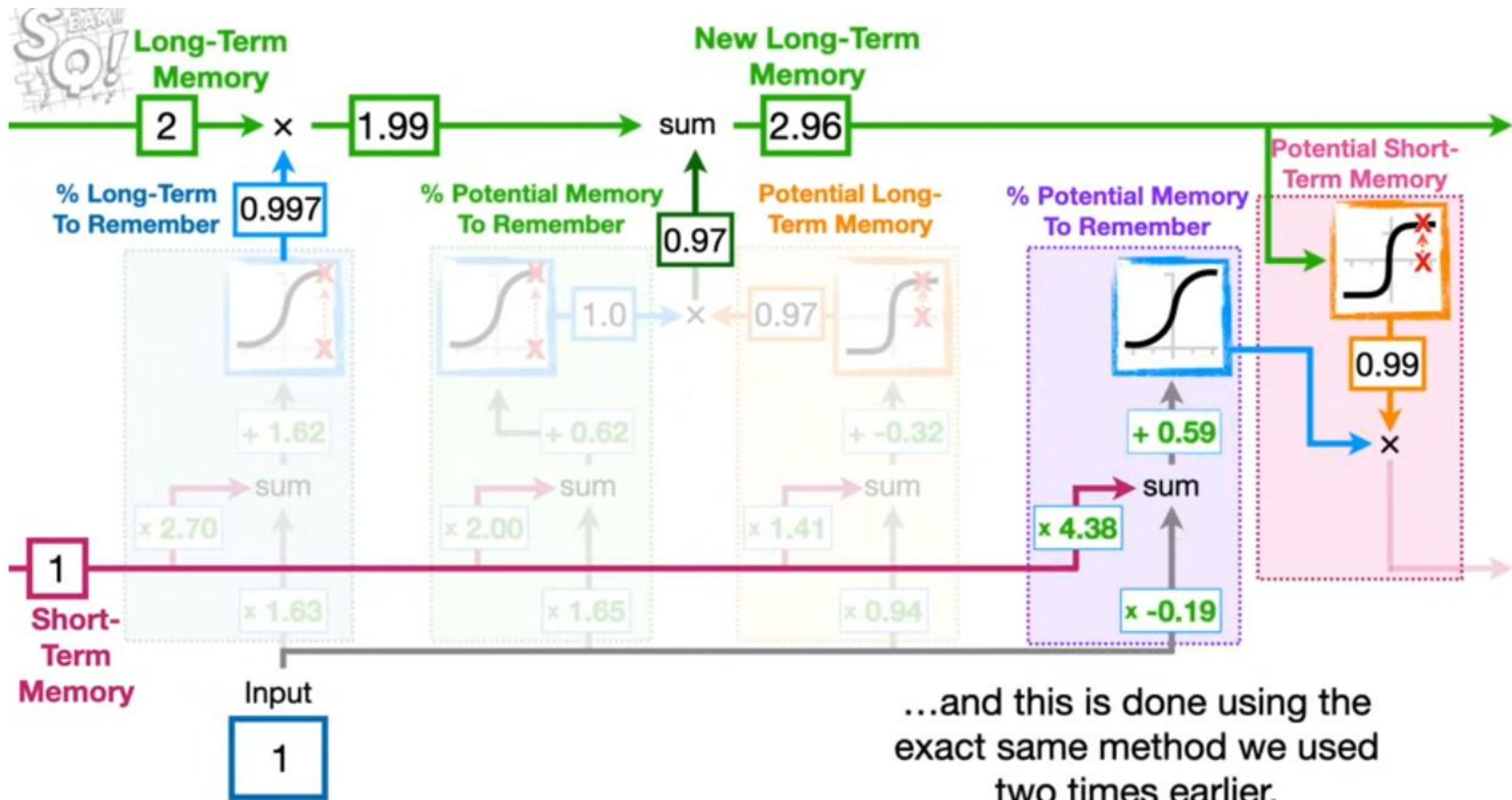


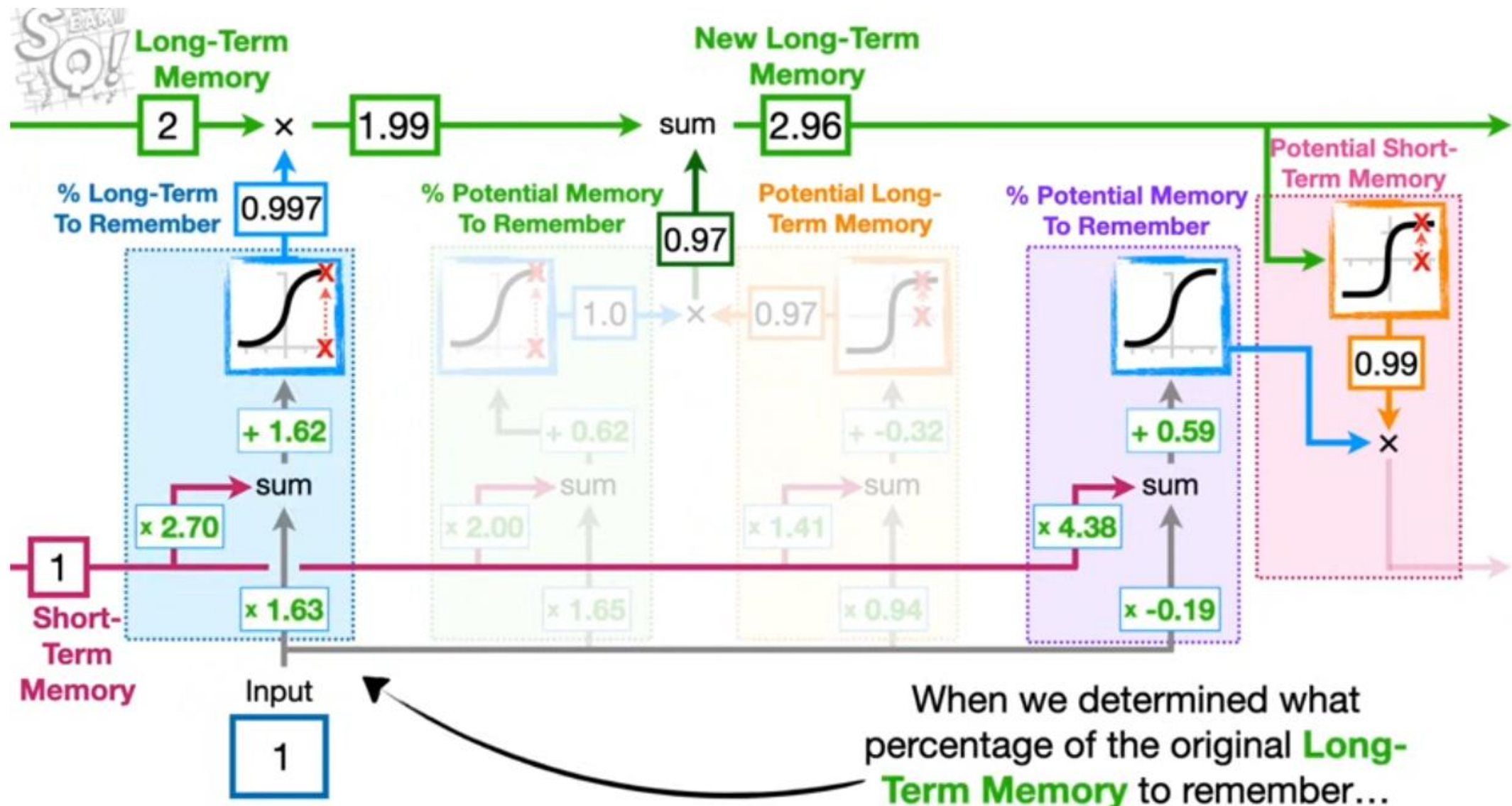


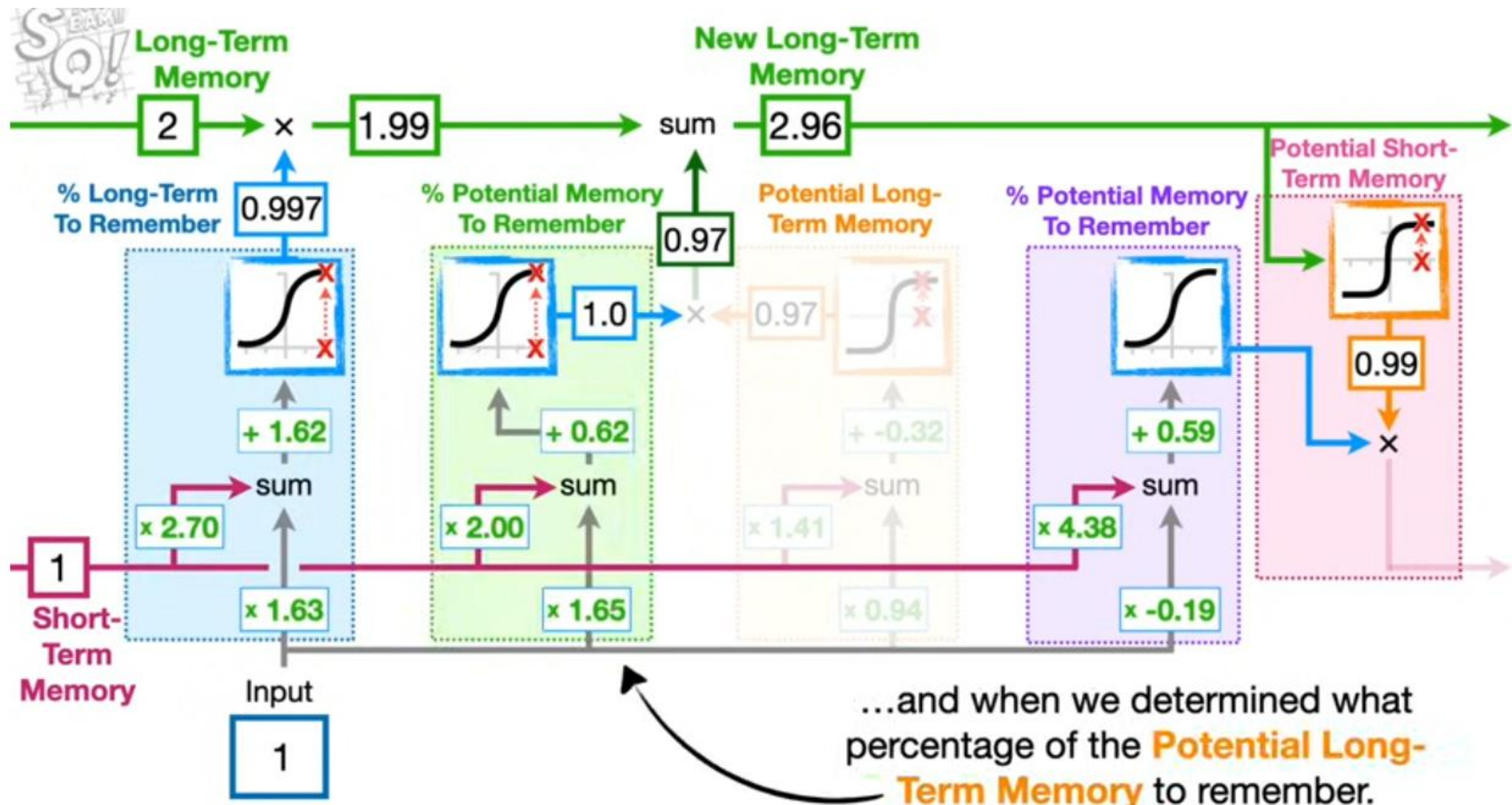


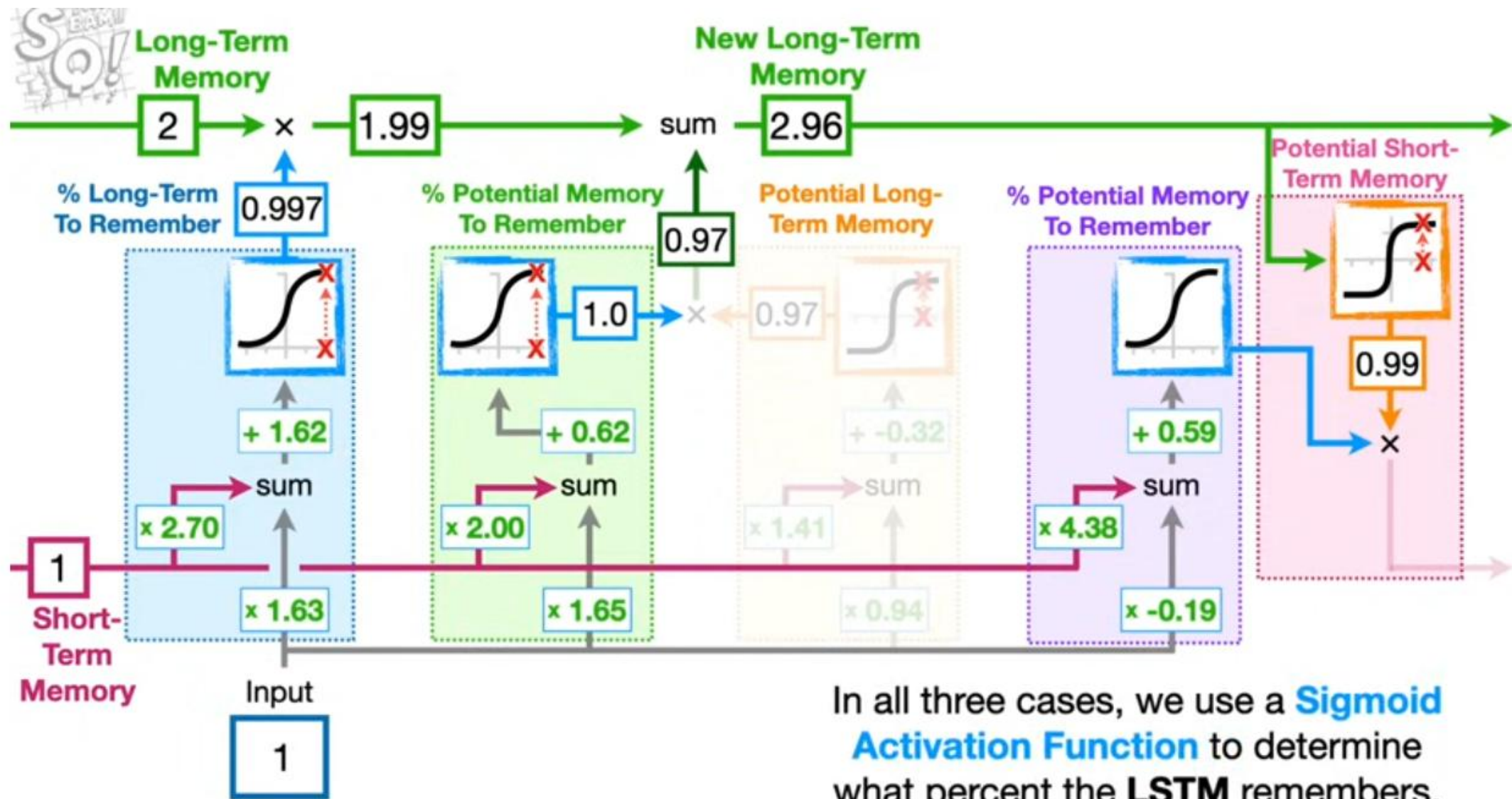












In all three cases, we use a **Sigmoid Activation Function** to determine what percent the **LSTM** remembers.

